PAM Reference Book

David Joyner; source documents co-authored with Claude

May 27, 2026

Contents

PAM — Pose And Motion

Stick-figure animation library for Manim, plus a Fountain screenplay converter — built for the *Too Nice to Die* animated sci-fi production pipeline.

PAM v0.9.18 · fountain2pam v0.9.13 Co-authored by David Joyner and Claude (Anthropic).

What PAM is

PAM (Pose And Motion) is a Python library that animates screenplay characters as stick figures, built on a graph-theoretic skeleton: joints are nodes, bones are edges, and every pose is a dict mapping joint names to (x, y) coordinates. Animation is keyframe-based — each action interpolates between named poses or programmatic targets.

PAM ships with:

- **Three figure types** — humanoid (with `male` / `female` / `child` gender presets), alien (split-torso skeleton, `male` / `female` variants), and dog (quadruped, with `facing` parameter).
- **Two prop-character types** — `DogGraph` (e.g. Chekov the robot dog) and `GovernorGraph` (autonomous dodecahedral background character).
- **A prop system** with ~30 registered builders, organized across five sub-modules: furniture, carried, flora, environment, accessories. Includes stateful props (`avatar_pod`, `elevator`, `pocket_door`, `tv_monitor`) with bound animation methods.
- **A keyframe pose library** (~50 named poses, ~11 cycles) covering standing, walking, running, jumping, sitting, reaching, lying flat, and emotional reaction beats.
- **A camera system** built on Manim's `MovingCameraScene` with a framing / MOVE vocabulary (`wide`, `medium`, `close`, `insert`; `pan-up`, `pan-down`, `tilt-up`, `tilt-down`, `dolly`).
- **A Fountain+ converter** (`fountain2pam.py`) that parses Fountain-format screenplays plus PAM-specific annotation keys and emits JSON action files plus per-subscene visual prompts.
- **A canonical character registry** (`tntd_characters.json`) carrying full color palettes, figure types, gender presets, and voice metadata for the TNTD cast.

PAM's design priority is **screenplay-driven blocking** — get the action timing, character positions, and dialogue beats correct quickly, so the output can be used as a reference for higher-fidelity downstream rendering (Blender, AI video) without re-deriving the choreography each time.

The production pipeline

```
Fountain screenplay (.fountain)
         |
         v
+---------------------+
|   fountain2pam.py   |   parses Fountain+ annotations
+---------------------+
         |
         +---> screenplay.json     (PAM action sequence)
         +---> prompts.json        (per-subscene visual prompts)
         +---> characters.txt      (synced from CHARACTER blocks)
         |
         v
+---------------------+
|    pam_player.py    |   Manim blocking animation
+---------------------+
         |
         v
    blocking MP4         -- used as timing/blocking reference
         |
         v
+---------------------+
|   pam2blender.py    |   exports scene layout to Blender
+---------------------+
         |
         v
Blender scene (camera, lights, props, character positions)
         |
         v
AI video generation (frame-by-frame, externally)
         |
         v
Final Cut Pro X (assembly, sound, final cut)
```

The blocking MP4 is the artifact most often regenerated during authoring — it answers "does the scene read?" before any expensive downstream rendering.

Installation

Requirements:

- Python 3.10+
- Manim Community (tested with 0.20.1 on the Cairo backend)
- screenplain — Fountain parser used by `fountain2pam.py`
- numpy, scipy (transitive via Manim)

```
pip install manim screenplain
# clone PAM into your project, then:
cd pam
pip install -e .
```

Optional:

- LaTeX (TeX Live or MacTeX) — needed only if you author scenes that use LaTeX math labels on characters (the dodecahedron `D`-style labels work without it).
- Blender 4.x — required only to run the `pam2blender.py` exporter.

Quick start

A two-character "hello" scene in JSON:

```
[
  {"action": "title", "text": "Quick Start", "subtitle": "PAM v0.9.13"},

  {"action": "cast", "characters": {
    "alice": {"figure_type": "human", "gender": "female",
              "offset": [-2.0, 0, 0],
              "style": {"head_label": "A", "edge_color": "#cc4477"}},
    "bob":   {"figure_type": "human", "gender": "male",
              "offset": [ 2.0, 0, 0],
              "style": {"head_label": "B", "edge_color": "#4477cc"}}
  }},

  {"action": "props", "items": {
    "table": {"type": "desk", "x": 0.0, "label": "T"}
  }},

  {"action": "fade_in", "who": "alice"},
  {"action": "fade_in", "who": "bob"},

  {"action": "say",  "who": "alice", "text": "Hi, Bob!"},
  {"action": "wave", "who": "bob", "cycles": 2},
  {"action": "say",  "who": "bob",   "text": "Hi, Alice!"},

  {"action": "walk_to", "who": "alice", "to_x": -0.5},
  {"action": "walk_to", "who": "bob",   "to_x":  0.5},

  {"action": "say", "who": "alice", "text": "Nice desk."}
]
```

Render it:

```
manim -pql pam_player.py PAMScene --json scene_quick_start.json
```

A working sample scene with the full feature set lives in `examples/scene_quick_start.json`.

Package layout

```
pam/
+-- __init__.py             public API exports
+-- pam_player.py           main Manim scene + action dispatcher
+-- characters.py           HumanGraph, AlienGraph, DogGraph,
|                            GovernorGraph
+-- poses.py                ~50 named poses + ~11 cycles +
|                            EXPRESSION_GLYPHS dict
+-- paired_poses.py         multi-character interaction templates
+-- actions.py              gesture and locomotion handlers
|                            (nod, shake_head, shrug, wave, ...)
+-- builds.py               figure proportions and joint scaling
|                            (default, narrow, broad, alien, ...)
+-- props.py                thin dispatcher - re-exports from
|                            sub-modules
+-- props_core.py           shared infrastructure
+-- props_furniture.py      chair, desk, pocket_door, elevator,
|                            avatar_pod, tv_monitor, desk_lamp
+-- props_carried.py        hat, briefcase, folder, phone (3 styles),
|                            backpack, laptop, bug
+-- props_flora.py          flower, floral_arrangement
+-- props_environment.py    building, sun, moon, dodecahedron,
|                            backdrop, solar_panel
+-- props_accessories.py    silver_hair builder
|                            (name_tag etc. are CAPTION-only)
+-- character_gallery.py    visual regression / documentation
+-- fountain2pam.py         Fountain -> PAM JSON converter
+-- pam2blender.py          PAM JSON -> Blender scene exporter
+-- tntd_characters.json    canonical character registry
```

`tntd_characters.json` is the **canonical** character registry — `characters.json` (older, present in some clones) is superseded and known to have at least one error: Factor's `figure_type` is `alien`, not human.

Core concepts

The skeleton graph

Each character is built from a graph: nodes are body joints (`head`, `neck`, `lshoulder`, `lelbow`, `lhand`, etc.) and edges are bones. Three skeleton variants ship with PAM:

Skeleton	Joints	Edges	Use
Human	12	11	`figure_type: "human"`

Skeleton	Joints	Edges	Use
Alien	14	14	`figure_type: "alien"` (split torso)
Dog	11	11	`figure_type: "dog"` (quadruped)

A **pose** is a Python dict mapping each joint name to an (x, y) tuple. PAM's animation engine interpolates between poses — actions like `walk_to` are implemented as keyframe-by-keyframe morphs over a walk cycle, with the figure translated in world space at each step.

Character classes

```
HumanGraph        humanoid skeleton, gender presets
AlienGraph        split-torso alien, gender variants
DogGraph          quadruped, with `facing` parameter
GovernorGraph     dodecahedral autonomous prop-character
```

Each accepts a `style` dict in the cast block carrying head_label, edge_color, node_color, node_stroke, head_color, head_stroke, and highlight_color.

Props

Props are Manim `VGroups` with three layers of metadata:

- **Flat attrs** (backward-compat): `.pam_name`, `.pam_type`, `.pam_x`, `.pam_y`, `.pam_surface_y`
- **Scene-graph node** (`.pam_node` dict): `name`, `kind`, `parent`, `attach`, `attrs`, `x`, `y`
- **Attachment points** (`.pam_attachments` dict): named 3-D positions on the geometry where child props or characters can connect

See `PROP_PROPERTIES.md` for the complete prop reference.

Actions

A PAM screenplay is a flat list of action dicts. Every action has an `"action"` key naming the handler. Handlers fall into five categories — see Categorized action reference below.

The PAM JSON screenplay format

A scene file is a JSON array of action dicts. The conventional order is:

1. `title` (optional) — opening title card
2. `cast` — declare all characters
3. `props` — declare all initial props
4. `_manifest_captions` (optional, documentation-only) — at-a-glance list of caption text and timing
5. Scene action sequence (`fade_in`, `say`, `walk_to`, ...)

Comment keys are silently skipped by `pam_player.py`:

Key prefix	Behavior
`_comment`	Skipped at any depth
`_hint`	Skipped; surfaces in debug logs
`_subscene_marker`	Used by `fountain2pam.py` to chunk prompts.json
`_shot_meta`	Camera framing hints for the subscene

Avoid the unprefixed keys `_manifest_captions`, `_note`, etc., at top level inside action dicts — `pam_player.py` does not skip those in all versions and may try to read `step["action"]` and fail. Either wrap them in `"_comment"` or place them at the array boundary between actions.

A minimal cast block

```
{"action": "cast", "characters": {
  "freydoon": {
    "figure_type": "human",
    "gender":      "male",
    "build":       "default",
    "pose":        "standing_front",
    "scale":       {"sy": 0.7, "sx": 0.7, "anchor": "lankle"},
    "offset":      [-4.5, 0, 0],
    "style": {
      "head_label":   "Freydoon",
      "edge_color":   "#e8a87c",
      "node_color":   "#7a4a2a",
      "node_stroke":  "#f0c090",
      "head_color":   "#5a3010",
      "head_stroke":  "#f5d0a0",
      "highlight_color": "#fde8c0"
    }
  }
}}
```

Full field reference: see `CHARACTER_PROPERTIES.md`.

A minimal props block

```
{"action": "props", "items": {
  "main_desk": {"type": "desk",     "x":  0.0, "label": "D"},
  "exit_door": {"type": "pocket_door", "x": 5.5}
}}
```

Full field reference: see `PROP_PROPERTIES.md`.

Categorized action reference

This section organizes PAM actions by *what they operate on*, so the right action is easy to find when authoring a scene. Detailed sub-key tables for each action live in `CHARACTER_PROPERTIES.md` (character-target actions) and `PROP_PROPERTIES.md` (prop-target actions).

Quick-lookup table

Action	Target(s)	Since	Parallel-safe?
`walk_to`	character	0.3	yes (different characters)
`run_to`	character	0.3	yes
`rush_to` / `rush_out`	character	0.9.5	yes
`squeeze_through`	character	0.9.6	no
`trot_to`	dog	0.9.7	yes
`turn`	character	0.3	yes
`morph`	character	0.3	yes
`nod` / `shake_head` / `shrug`	character	0.9.11	no
`wave`	character	0.3	yes
`jump_up`	character	0.9.5	yes
`dodge`	character	0.9.5	yes
`pat`	character	0.9.5	no
`react` / `express`	character	0.9.5	yes
`say`	character	0.3	no (blocks for read time)
`prop_say`	prop	0.7	no
`clear_bubble`	character or prop	0.9.10	yes
`clear_all_bubbles`	scene-level	0.9.10	yes
`change_uniform`	character (torso color)	0.9.9	yes
`focus` / `focus_reset`	scene-level	0.9.7	yes
`fade_in` / `fade_out`	character	0.3	yes
`flash`	character or prop	0.9.7	yes
`rotate`	character or prop	0.9.13	no
`group_translate`	multi-target	0.9.6	WARNING unsafe today
`reach_for`	character + prop	0.9.5	no
`grab`	character + prop	0.9.5	no
`punch_button`	character + prop	0.9.5	no
`place_on`	character + prop	0.9.5	no
`move_aside`	character + prop	0.9.5	no
`stick_to`	character + prop	0.9.5	no
`peel_from_hand`	character + prop	0.9.5	no
`snap_photo`	character + prop	0.9.5	no
`exit_through_doors`	character + prop	0.9.5	no
`open_doors` / `close_doors`	prop	0.9.5	yes
`open_lid` / `close_lid`	prop	0.9.5	yes
`spawn_prop` / `remove_prop`	prop	0.7	yes
`caption` / `persistent_caption`	scene-level	0.9.5	yes
`sound_cue`	scene-level	0.9.5	yes
`on_screen_text`	scene-level	0.5	yes

Action	Target(s)	Since	Parallel-safe?
`title`	scene-level	0.3	n/a (always first)
`wait`	scene-level	0.3	n/a
`parallel`	meta	0.7	n/a (it *is* the meta)
`zone_shift`	scene-level	0.9.5	yes

WARNING `group_translate` is the only known parallel-unsafe action that may plausibly appear inside a `parallel` block (walk-and-talk). v1.0.0 will either fix the safety issue or have the player reject it with a clear error.

A. Character-only actions

Locomotion, gestures, poses, dialogue — anything that operates on a single character without a prop target.

- **Locomotion:** `walk_to`, `run_to`, `rush_to`, `rush_out`, `squeeze_through`, `trot_to` (dogs)
- **Pose / direction:** `turn`, `morph`, `set_pose`
- **Gestures:** `nod`, `shake_head`, `shrug`, `wave`, `jump_up`, `dodge`, `pat`
- **Reactions:** `react`, `express` (smirk / roll_eyes glyphs)
- **Dialogue:** `say` (with `persist`, `duration`, `os` bubble styles)
- **Visibility:** `fade_in`, `fade_out`
- **Costume:** `change_uniform`

B. Prop-only actions

Operate on a single prop or change a prop's state.

- **Spawning / despawning:** `spawn_prop`, `remove_prop`
- **Stateful methods:** `open_doors`, `close_doors`, `open_lid`, `close_lid` (dispatched by `getattr` on the prop VGroup)
- **Visual cues:** `flash` (also accepts a character)
- **Dialogue:** `prop_say`

C. Character + prop interactions

Actions that require both a character (`who`) and a prop (`prop` or `target`). Most are sequential-only (not parallel-safe) because they mutate the character's hand/arm state.

- **Pointing:** `reach_for`, `punch_button`
- **Carrying:** `grab`, `place_on`, `move_aside`, `stick_to`, `peel_from_hand`, `snap_photo`
- **Exit:** `exit_through_doors` (walks to a door/elevator, pauses, fades out)

D. Scene-level actions

Operate on the scene, camera, or overlay text.

- **Captions:** `caption`, `persistent_caption`, `title`, `on_screen_text`
- **Diegetic cues:** `sound_cue`
- **Time control:** `wait`, `parallel`
- **Focus:** `focus`, `focus_reset`

- **Cleanup:** `clear_bubble`, `clear_prop_bubble` (alias), `clear_all_bubbles`
- **Zone:** `zone_shift` (sub-location marker for the prompts file)

E. Multi-target actions

- **`group_translate`** — synchronized translation of multiple characters and/or props. Used for walk-and-talk treadmill resets. Parallel-unsafe today (see warning above).

Fountain+ annotations

Fountain+ extends standard Fountain with bracketed annotation lines that drive PAM-specific behavior. All annotations live in `[[ ... ]]` note blocks scoped to the preceding beat or scene heading.

The full reference lives in **`fountain2pam-manual.md`** — this is a summary table of the keys.

Key	Purpose	Emits
`CAMERA`	Framing / move / subject / transition	`_shot_meta` keys on the next subscene marker
`MOOD`	Lighting / atmosphere	prompts.json hint
`KIND`	Scene type (interior / exterior / fantasy / phone-call)	prompts.json hint
`SCENE POPULATION`	Background extras	prompts.json hint
`NEGATIVE`	"Avoid this" terms	prompts.json hint
`CAPTION`	On-screen text card	`caption` action
`SOUND`	Diegetic sound flash	`sound_cue` action
`PHONE`	Intercut phone-call mode	`os_bubble` on `say`
`CHARACTER`	Per-scene character override	cast-block entry
`PRODUCTION NOTE`	Metadata only	(nothing — for humans)

Example:

```
INT. POLYMERCORP - CEO'S OFFICE - DAY

[[ CAMERA: FRAMING=medium | SUBJECT=ensemble | MOVE=static | TRANSITION=cut ]]
[[ MOOD: corporate, harsh fluorescent, late afternoon ]]
[[ CAPTION: Office of Mrs Tatiana Bosch, CEO of PolymerCorp. | lower-third | 5s |
↪  italic ]]

BRAD
(into phone)
We need that shipment Tuesday.

[[ PHONE: Brad on landline; intercut with Chava ]]
```

Reference documents

The README is intentionally a front door — it points to deeper docs for the details.

Document	Scope
`CHARACTER_PROPERTIES.md`	Complete character reference: every field a cast entry accepts, every action that targets a character, color authority chain, registry behavior, name resolution, the `hidden` flag, z-order, and ~22 documented footguns.
`PROP_PROPERTIES.md`	Complete prop reference: creation-time fields, prop metadata (flat attrs, `.pam_node`, `.pam_attachments`), registry methods, all prop-target actions, stateful-prop dispatch, walk-following, and prop-side gotchas.
`fountain2pam-manual.md`	Full Fountain+ reference: every annotation key with examples, CAMERA / LIGHTING vocabulary tables, action-interpreter pattern reference, output file structures, complete worked screenplay example.
`pam_player-manual.md`	(planned) Player-side reference: action handler internals, the camera system (`MovingCameraScene`, `_apply_camera`, framing / MOVE vocabulary), parallel-block semantics, the bubble registry.
`BACK_BURNER.md`	Items deferred to v1.0.0 or later: API normalization (`to_x`/`x`), the `StatefulProp` mixin refactor, walk-and-talk attached-bubble pattern, etc.

Extending PAM

Adding a new prop type

1. Implement `build_<yourtype>(name, x, y=..., **kwargs)` in the relevant `props_*.py` submodule, returning a `VGroup` with `_attach_pam_attrs` metadata.
2. Add an entry to `PROP_TYPES` in `props.py` mapping the type name (and any aliases) to the builder.
3. If the prop is stateful, attach `open_*`, `close_*`, or method-named handlers to the returned VGroup. See `build_pocket_door` or `build_avatar_pod` as templates.
4. Document in `PROP_PROPERTIES.md` §6.

Adding a new character build

1. Add a token -> build-name entry to `CHARACTER_BUILD_MAP` in `fountain2pam.py`.
2. If the build needs new proportions, add a row to the proportion table in `builds.py`.

3. If the build introduces a new figure type, see the `AlienGraph` / `DogGraph` implementations in `characters.py` as templates.

Adding a new action handler

1. Add the handler to `actions.py` (gesture/locomotion) or `pam_player.py` (scene-level or prop-state actions).
2. Add entries to `_BEAT_DURATION_MAP`, `_summarise_beats`, and the drama-scoring function in `fountain2pam.py` if the action should be inferred from prose.
3. Document the action in `CHARACTER_PROPERTIES.md` or `PROP_PROPERTIES.md` (whichever is the natural home) and add a row to the Quick-lookup table.

Adding a Fountain+ annotation key

1. Add the key's parser to `fountain2pam.py` alongside the existing `MOOD` / `KIND` / `CAMERA` parsers.
2. Decide whether it emits a PAM action or contributes to `prompts.json` metadata only.
3. Document in `fountain2pam-manual.md`.

Version history

A summary of headline changes per minor version. Full per-version notes live in `CHANGELOG.md`.

Version	Headline changes
0.9.18	`face_builder.py` per-character geometry overrides for hair and triangle hats. Hair: `hair_width`, `hair_height`, `hair_offset_y` tune the primary cap shape of any `hair_style`. Hats: `hat_width` and `hat_height` now also honor the `"triangle"` and `"triangle_inv"` styles (previously only `"square"` read them). All overrides are optional and default to the existing hardcoded values, so existing characters render unchanged. Alien-ear anchoring fix: `_build_ears` and `_build_earrings` accept a `head_rx` argument so ears sit at the species-correct head edge. `pam/tics.py` fragment registry expanded with `hand_twitch_l` (mirror of `hand_twitch_r`), `foot_tap_r`, and `foot_tap_l`, all applicable to `human` and `alien` figures. `GovernorGraph.say` bubble placement is now geometry-aware: bubble bottom sits a fixed gap above `self._y + self._radius` regardless of prop size, replacing the v0.9.x hardcoded `self._y + 0.3` that put the tail tip inside the dodecahedron's lower half. `prop_say` accepts a new `y_offset` key (`GovernorGraph` only) for per-call vertical nudging. Focus/focus_reset head-dot exclusion upgrade: the v0.9.17 end-state repair for face-attached figures is superseded by removing the head dot from the animation target entirely (rebuilt `VGroup(edge_group, *non_head_dots)`), eliminating the transient flicker visible at `rt < 0.05`. See §5.5 for the hair/hat default tables and syntax examples, §1.12 for the tic fragment registry, §3.1.2 for the `y_offset` parameter, and §8.4 for the focus head-dot exclusion.
0.9.13	`rotate` action for characters and props (uses joint coords + offset for character pivot; native Manim getters for props); `wave` direction-key bug fixed (now reads `direction` from JSON); `lying_flat_right` / `lying_flat_left` poses for horizontal speech-bubble anchoring; `closed` param on `build_laptop`; `paired_poses.py` for multi-character interaction templates.
0.9.11	`nod`, `shake_head`, `shrug` gesture actions.

Version	Headline changes
0.9.10	Persistent-bubble cleanup: `clear_bubble`, `clear_prop_bubble`, `clear_all_bubbles`. Documented the `focus_reset` overlay gotcha.
0.9.9	`change_uniform` action for mid-scene torso recolor. Cast-block `uniforms` dict for named variants.
0.9.8	Color authority chain: three-place merge (registry / cast block / Fountain+ annotation). `fountain2pam.py --characters` correctly applies palettes.
0.9.7	`props.py` split into six sub-modules. `build_backpack` (Fano-plane graph), `build_laptop`. `_drag_attached_props` for held-prop walk-following. `focus` / `focus_reset` scene-dimming.
0.9.6	`group_translate` for multi-target synchronized motion. `squeeze_through` for narrow-stance walks.
0.9.5	Major action expansion: `grab`, `place_on`, `move_aside`, `reach_for`, `punch_button`, `stick_to`, `snap_photo`, `peel_from_hand`, `jump_up`, `dodge`, `react`, `express`, `pat`, `search_drawers`, `caption`, `persistent_caption`, `sound_cue`, `zone_shift`, `rush_to`, `rush_out`, `exit_through_doors`. Fountain+ keys: `CAPTION`, `SOUND`, `PHONE`, `PRODUCTION NOTE`. `flash` action for temporary recolor.
0.9.4	`pam2blender.py` exporter. Blender workflow integration. Stateful props: `build_pocket_door`, `build_elevator`, `build_avatar_pod` with `open_doors/close_doors/open_lid/close_lid` methods.
0.9.3	Gender presets for humanoid and alien builds. Character registry (`characters.txt` / `tntd_characters.json`). Fountain+ `CHARACTER` annotation. `character_gallery.py`.
0.9.0	First public-facing release. Camera system on `MovingCameraScene`. Full Fountain+ CAMERA vocabulary. Manim -> Blender -> AI video pipeline.

License

PAM is a working artifact of the Too Nice to Die* production pipeline. Bug reports, suggestions, and pull requests welcome on GitHub: wdjoyner/pam.*

pam_player.py Reference Manual

PAM v0.9.13 — pam_player.py reference

Companion document to CHARACTER_PROPERTIES.md and PROP_PROPERTIES.md. This manual covers the *player* side: how pam_player.py loads a screenplay, dispatches actions, manages the camera, tracks persistent bubbles, and what the architectural split between pam_player.py and actions.py actually means in practice.

Where the property docs answer "what does this action do?", this manual answers "how does the player know what to do, and what happens when it does it?"

---

1. Overview

1.1 What pam_player.py is

pam_player.py is a Manim MovingCameraScene subclass (PAMPlayer) that loads a JSON screenplay from disk, iterates its action list, and emits a rendered video. It is the bridge between the authoring layer (Fountain -> JSON via fountain2pam.py, or hand-edited JSON) and Manim's rendering pipeline.

The class exposes a single public entry point — Manim's construct() method — which:

1. Reads PAM_SCRIPT from the environment.
2. Loads the JSON action list.
3. Sets up the camera (if PAM_CAMERA_MODE=1).
4. Iterates the action list, dispatching each step.
5. Lets Manim's renderer produce the MP4.

There is no command-line argument parsing of PAM's own — Manim owns the CLI, so PAM-specific configuration is passed through environment variables.

1.2 The split: pam_player.py vs actions.py

Before v0.9.5, _dispatch_one in pam_player.py was a ~350-line if/elif chain handling every action type. The v0.9.5 refactor moved character-targeting action handlers into a separate actions.py module with a registry-based dispatch:

```
from pam.actions import ACTION_REGISTRY

def _dispatch_one(step, name, *, collect_anims=False):
```

```
    act = step["action"]
    fig = _get_fig(name)
    if fig is None:
        return None

    # Special cases handled inline below: fade_in, say, trot_to
    # Everything else: dispatch via registry.
    handler = ACTION_REGISTRY.get(act)
    if handler:
        step_aug = dict(step, _collect_anims=collect_anims) \
                   if collect_anims else step
        return handler(fig, step_aug, self, name,
                       props=prop_registry, cast=cast)
```

After the refactor, `_dispatch_one` shrank to ~100 lines and only three character-targeting actions remain inline: `fade_in` (because it *creates* the figure), `say` (because it needs the `_pending_camera` mechanism, §7.5), and `trot_to` (because dogs are keyed in the prop registry, not the cast).

Everything else with a character target — `walk_to`, `run_to`, `morph`, `turn`, `wave`, `nod`, `grab`, `punch_button`, etc. — lives in `actions.py`.

What stays in `pam_player.py`:

- The main action loop (the iteration itself)
- The camera system (`_apply_camera`, `_camera_anim`, `_FRAMING_CAMERA`)
- The bubble registry (`_persistent_bubbles`, `_sweep_expired_bubbles`)
- Cast registration (`cast` action)
- Prop registration (`props`, `spawn_prop`, `remove_prop`)
- The `parallel` meta-dispatcher
- The three inline special cases above
- Scene overlays (`title`, `caption`, `sound_cue`, `wait`)
- `_subscene_marker` and `zone_shift` (camera triggers)

What lives in `actions.py`:

- All character-targeting handlers that don't need player-internal state (~30+ actions)
- The `ACTION_REGISTRY` dict mapping action names to handlers
- The `KNOWN_ACTIONS` frozenset (used by `fountain2pam.py` for validation)
- The `_CANNOT_PARALLEL` frozenset (used by `pam_player.py`'s parallel handler)

1.3 File responsibilities

File	Owns
`pam_player.py`	Main loop, camera, bubbles, cast/prop registries, parallel dispatch, the three inline handlers
`actions.py`	Character-targeting action handlers, `ACTION_REGISTRY`, `_CANNOT_PARALLEL`
`characters.py`	`HumanGraph`, `AlienGraph`, `DogGraph`, `GovernorGraph` classes — including `say()` and `morph_to()` methods

File	Owns
`figure.py`	Shared figure infrastructure (bubble layout, scale anchors) — referenced by character classes
`poses.py`	Named pose dicts and walk/run/jump cycles
`paired_poses.py`	Multi-character interaction pose templates
`props.py` + `props_*.py`	Prop builders and registry

When debugging an action, the question "where does this code live?" follows the rule: if the action targets a character only and doesn't need camera or bubble state, it's in `actions.py`; if it touches the camera, the bubble registry, the cast/prop registries, or the `parallel` block, it's in `pam_player.py`.

---

2. Running pam_player

2.1 Required: `PAM_SCRIPT`

The path to the JSON screenplay file. Required — `pam_player.py` raises a clear error if it's unset or points to a missing file.

```
PAM_SCRIPT=scene_25_hospital.json manim -pql pam_player.py PAMPlayer
```

The path is resolved relative to the working directory at run time.

2.2 Optional environment variables

`PAM_CAMERA_MODE` — gates the entire camera system.

Value	Effect
unset / `0`	No camera moves. `_subscene_marker` steps are silently skipped. Frame stays at Manim's default.
`1`	Camera system active. `_subscene_marker` triggers reframes per `_shot_meta`. Initial wide/ensemble reset fires before the action loop.

Crucial gotcha: without `PAM_CAMERA_MODE=1`, every `_subscene_marker` in the JSON is a no-op. If a scene has an `insert` framing on a phone that doesn't seem to fire, this is the first thing to check.

`PAM_PROMPTS` — path to the prompts JSON written by `fountain2pam.py` alongside the action JSON. Only consulted when `PAM_CAMERA_MODE=1`. Provides per-subscene metadata (lighting hints, freeform notes) that complement `_shot_meta`. Default: `<PAM_SCRIPT stem>_prompts.json`.

`PAM_SHOW_CLOCK` — set to `1` to render a live elapsed-time display in the upper-right corner. Format: `M:SS.ss` (minutes, seconds, hundredths). Intended for low-quality preview renders only; the clock is a Manim mobject and consumes real render time.

```
PAM_SCRIPT=scene.json PAM_SHOW_CLOCK=1 manim -pql pam_player.py PAMPlayer
```

2.3 The `pam-render` wrapper

A shell script (typically in the repo root or `~/bin`) wraps the environment-variable plumbing:

```
#!/bin/bash
# pam-render - render a PAM screenplay
SCRIPT="${1:-screenplay.json}"
OUTPUT="${2:-PAMPlayer}"
QUALITY="${3:-l}"

PAM_SCRIPT="$SCRIPT" manim -pq"$QUALITY" \
  -o "${OUTPUT}.mp4" pam_player.py PAMPlayer
```

Usage:

```
./pam-render scene_25.json scene_25_preview        # 480p
./pam-render scene_25.json scene_25_final     h    # 1080p
./pam-render scene_25.json scene_25_master    k    # 4K
```

Extended versions of `pam-render` may accept `--camera`, `--show-clock`, and `--prompts` flags that set the corresponding environment variables before invoking Manim.

2.4 Manim flags and quality presets

Flag	Quality	Resolution	FPS
`-ql`	low	480p	15
`-qm`	medium	720p	30
`-qh`	high	1080p	60
`-qk`	4K	2160p	60
`-p`	preview	(opens player when done)	—

Authoring iteration usually runs at `-pql` (preview + low) for fast feedback; final renders use `-qh` or `-qk`.

---

3. World coordinates

3.1 The reference frame

PAM characters at the default `scale=0.7` sit in a coordinate frame where:

Landmark	Approximate y
Floor	-2.6
Ankle / feet	-2.0
Waist	-0.5
Shoulders	0.5

Landmark	Approximate y
Head	1.2
Governor (hovering)	1.5

X coordinates run left (negative) to right (positive). The **stage** — the area characters should occupy — spans roughly `x in [-6, 6]`.

3.2 Stage extents and safe zones

The default Manim frame width is 14.2 units; the visible screen extends to `x in [-7.1, 7.1]` and `y in [-4, 4]`.

Speech bubbles extend ~2.5 units from the speaker's head. Safe x ranges for speakers:

- `side: "right"` bubble — speaker must be at `x <= 4.0`
- `side: "left"` bubble — speaker must be at `x >= -4.0`

Characters parked off-camera can sit at `x approx. ±6.5` (just inside the stage edge).

3.3 Default frame extents

The initial camera state (set both by Manim's defaults and by the explicit initial reset in `construct()` when camera mode is active):

Property	Value
Frame width	14.2
Frame center	`(0.0, -0.5, 0.0)`

This corresponds to `framing: "wide"`, `subject: "ensemble"`.

---

4. The action dispatcher

4.1 The main loop

The heart of `pam_player.py` is the `for step in actions:` loop in `construct()`. Each iteration does roughly:

```
for step in actions:
    if "_comment" in step or "_hint" in step:
        continue

    if _persistent_bubbles:
        _sweep_expired_bubbles()

    act = step.get("action")
```

```
# Main-loop handlers (cast, props, spawn_prop, parallel,
# _subscene_marker, fade_in, say, prop_say, trot_to, ...)
if act == "cast": ...
elif act == "props": ...
elif act == "parallel": ...
elif ...
elif "_subscene_marker" in step:
    ...
else:
    # Resolve targets and dispatch via the registry
    for name in _targets(step):
        _dispatch_one(step, name)
```

The main loop's structure mixes:

1. **Annotation skip** — `_comment` / `_hint` keys cause `continue`.
2. **Bubble bookkeeping** — `_sweep_expired_bubbles()` at the top.
3. **Inline special cases** — handlers that need player-level state.
4. **Registry dispatch** — for everything else, via `_dispatch_one`.

4.2 Skipped keys (`_comment`, `_hint`)

Any step containing `_comment` or `_hint` is skipped entirely. This allows scene authors to annotate the JSON:

```
{"_comment": "REVIEW: this beat may need a beat of silence first"},
{"action": "say", "who": "freydoon", "text": "...wait."}
```

Gotcha: because `_comment` is checked at the top of the loop, *any* step containing `_comment` is skipped — even if it also has an `"action"` key. An action with a `_comment` key inside it is a no-op. If you need both, place the comment in a separate dict *before* the action.

Other underscore-prefixed top-level keys are **not** skipped:

- `_subscene_marker` is handled specially (§5.8)
- `_shot_meta` is consumed by the subscene marker handler
- `_manifest_captions`, `_note`, etc. — **not** skipped, and the player will try to read `step["action"]` and fail. Wrap these in `_comment` if you need documentation blocks.

4.3 Target resolution: `_targets`, `_get_fig`, `_get_prop`

Target resolution happens in three helpers:

`_targets(step)` — returns a list of character names from a step's `who` key. Accepts:

`who` value	Returns
`"freydoon"`	`["freydoon"]`
`["freydoon", "bevers"]`	`["freydoon", "bevers"]`
`"all"`	every key in `cast`
(missing)	`[]`

_get_fig(name) — returns the live figure object for a character, or `None` if the character isn't faded in. Reads `cast[name]["fig"]`.

_get_prop(pname) — returns the live prop VGroup, or `None`. Reads from the prop registry.

4.4 _dispatch_one and the ACTION_REGISTRY

For any step that isn't an inline special case, the main loop calls `_dispatch_one(step, name)` per resolved character name. Internally:

```
def _dispatch_one(step, name, *, collect_anims=False):
    act = step["action"]
    fig = _get_fig(name)
    if fig is None:
        return None

    # Inline cases (fade_in, say, trot_to) handled here...

    # Otherwise: ACTION_REGISTRY dispatch
    handler = ACTION_REGISTRY.get(act)
    if handler is None:
        return None    # silent no-op for unknown actions
    return handler(fig, step, self, name,
                   props=prop_registry, cast=cast)
```

The registry is built in `actions.py` as a dict mapping action names to handler functions:

```
ACTION_REGISTRY: dict[str, callable] = {
    "walk_to":   act_walk_to,
    "run_to":    act_run_to,
    "turn":      act_turn,
    "morph":     act_morph,
    "wave":      act_wave,
    "nod":       act_nod,
    "shake_head": act_shake_head,
    "shrug":     act_shrug,
    "grab":      act_grab,
    "punch_button": act_punch_button,
    "reach_for": act_reach_for,
    # ... ~30+ entries ...
}
```

Unknown-action behavior. If an action name doesn't appear in the registry and isn't a main-loop special case, `_dispatch_one` returns `None` silently. This is the failure mode when a typo in the JSON produces nothing on screen. Validate authored JSON against `KNOWN_ACTIONS` (also exported by `actions.py`) to catch these at parse time.

4.5 Handler signature convention

Every handler in `actions.py` follows the same signature:

```
def act_walk_to(fig, step, scene, name="I.G. NoreMe",
                *, props=None, cast=None):
    """Walk a character to a target x position."""
    to_x = step.get("to_x", step.get("x"))  # accept either
    rt   = step.get("rt", 0.6)
    ...
    if step.get("_collect_anims"):
        return fig._walk_anims(to_x, rt=rt)
    fig.walk_to(scene, to_x, rt=rt)
    return None
```

The signature elements:

Arg	Meaning
`fig`	The live figure object (already resolved)
`step`	The action dict
`scene`	The PAMPlayer instance (Manim scene)
`name`	Character key (for state updates)
`props=`	The prop registry (for actions that target props)
`cast=`	The cast registry (for actions touching others)

The `name="I.G. NoreMe"` default is a sentinel — handlers that require a real name check for it explicitly.

4.6 The `_collect_anims` thread

When a sub-action runs inside a `parallel` block, the dispatcher passes `collect_anims=True`. This is threaded into the step dict as `step["_collect_anims"] = True`, so handlers see it without a signature change.

A handler that supports parallel collection returns a list of Manim animations instead of playing them itself:

```
if step.get("_collect_anims"):
    return fig._pose_anims(target_pose, new_offset)
fig.morph_to(scene, target_pose, new_offset=new_offset, rt=rt)
return None
```

The `parallel` handler then plays all collected animations concurrently in a single `scene.play(*all_anims, run_time=rt)` call.

Handlers that don't support `_collect_anims` (the parallel-unsafe set) ignore the flag and just run. The parallel handler routes them to a sequential fallback path — see §6.

---

5. Special main-loop handlers

These handlers live inline in `construct()`'s main loop, not in `actions.py`, because they touch player-level state (registries, camera, bubbles, scene clock).

5.1 `cast` — character registration

Registers all characters at scene start. Reads the `characters` sub-dict and populates the `cast` dict with one entry per character:

```
cast[cname] = {
    "fig":         None,          # populated on fade_in
    "figure_type": "human",       # or "alien", "dog", "governor"
    "pose":        "standing_front",
    "offset":      [-2.0, 0, 0],
    "style":       {"head_label": "F", "edge_color": "#e8a87c", ...},
    "build":       "default",
    "scale":       {"sy": 0.7, "sx": 0.7, "anchor": "lankle"},
    "spawn":       {},            # for prop-characters
}
```

The `fig` slot stays `None` until `fade_in` actually builds the figure (§5.4). This separation lets `cast` declare characters without committing to their appearance — they may be revealed only later in the scene.

5.2 `props` — initial prop registration

Same pattern for props:

```
{"action": "props", "items": {
  "main_desk": {"type": "desk", "x": 0.0, "label": "D"},
  "exit_door": {"type": "pocket_door", "x": 5.5}
}}
```

Each entry is built immediately via `build_prop(name, **spec)` and added to the prop registry. Props declared in `props` appear at opacity 1 from scene start unless they carry `"hidden": true`.

5.3 `spawn_prop` / `remove_prop`

Mid-scene appearance and despawn. `spawn_prop` builds the prop and fades it in:

```
prop = build_prop(name, **spawn_spec)
prop_registry.add(name, prop)
scene.play(FadeIn(prop), run_time=rt)
```

`remove_prop` does the inverse:

```
prop = prop_registry.get(name)
scene.play(FadeOut(prop), run_time=rt)
prop_registry.remove(name)
```

See `PROP_PROPERTIES.md` §3 for the full registry behavior, including the symmetric-clear gotcha when a prop has been parent-attached.

5.4 `fade_in` — figure construction

`fade_in` stays inline because it *creates* the figure object:

```
if act == "fade_in":
    entry = cast[name]
    if entry["figure_type"] == "human":
        fig = HumanGraph(**entry["style"], build=entry["build"])
    elif entry["figure_type"] == "alien":
        fig = AlienGraph(**entry["style"], gender=entry.get("gender"))
    elif entry["figure_type"] == "dog":
        fig = DogGraph(facing=entry.get("facing", "right"))
    elif entry["figure_type"] == "governor":
        fig = GovernorGraph(**entry["style"])

    fig.pose = POSES[entry["pose"]]
    fig.offset = np.array(entry["offset"])
    if entry.get("scale"):
        fig.apply_scale(**entry["scale"])

    fig.fade_in(self, rt=step.get("rt", 0.5))
    entry["fig"] = fig
    continue
```

After `fade_in`, the figure is in the cast registry and discoverable by `_get_fig(name)`. All subsequent actions can target it.

5.5 say / prop_say — bubble plus concurrent camera

The `say` handler stays inline because it implements the **`_pending_camera` mechanism** (§7.5): if a `_subscene_marker` just fired with an animated camera move, the move is held in `self._pending_camera` and included in the `say`'s `scene.play(...)` call so the camera glides during the bubble fade-in:

```
if act == "say":
    fig = _get_fig(name)
    text = step["text"]
    extra_anims = []
    if self._pending_camera is not None:
        extra_anims.append(self._pending_camera)
        self._pending_camera = None

    fig.say(text, scene=self,
            extra_anims=extra_anims,
            ...)
```

`prop_say` follows the same pattern but builds a bubble anchored above the prop's `pam_x` / `pam_y` rather than the character's head.

Persistent bubbles. Either handler, when given `persist: true` or `duration: t`, registers the bubble in `_persistent_bubbles`:

```
if step.get("persist") or step.get("duration"):
    expires_at = (self._scene_clock[0] + step["duration"]
```

```
                if step.get("duration") else None)
    self._persistent_bubbles[name_or_pname] = {
        "bubble": bubble,
        "expires_at": expires_at,
    }
```

See §8 for the full bubble lifecycle.

5.6 `parallel` — meta-dispatch

The `parallel` handler is structurally a sub-loop:

```
if act == "parallel":
    sub_actions = step.get("do", [])
    rt = step.get("rt", 0.4)

    # Partition sub-actions into:
    #  - locomotion (walk_to, run_to, ...) - special keyframe path
    #  - simple (everything parallel-safe) - collect anims
    locomotion = [s for s in sub_actions if s["action"] in _LOCO]
    simple     = [s for s in sub_actions if s["action"] not in _LOCO]

    # Pass 1: locomotion - interpolated keyframes across all movers
    if locomotion:
        ... _walk_anims for each character, played together ...

    # Pass 2: simple parallel-safe sub-actions
    if simple:
        all_anims = []
        for sub in simple:
            for tname in _targets(sub):
                anims = _dispatch_one(sub, tname, collect_anims=True)
                if anims:
                    all_anims.extend(anims)
        if all_anims:
            scene.play(*all_anims, run_time=rt)

    # Post-pass: update fig.pose for turn, clear cast for fade_out
    ...
    continue
```

The two-pass structure exists because locomotion sub-actions need keyframe-by-keyframe coordination (so two walkers stay in step), but single-pose sub-actions like `wave` or `nod` can just have their animation lists concatenated.

5.7 `trot_to` — DogGraph routing

`trot_to` is special because `DogGraph` instances live in the **prop registry**, not the cast. When a step has `"action": "trot_to"` and a `"prop"` key (not `"who"`), the main loop routes it directly to a DogGraph-aware path:

```
if act == "trot_to" and "prop" in step and "who" not in step:
    _dispatch_one(step, step["prop"])
    continue
```

`_dispatch_one` then resolves the dog via the prop registry instead of the cast.

Known inconsistency: `trot_to` reads `step["x"]` while `walk_to` reads `step["to_x"]`. Flagged for v1.0.0 normalization.

5.8 `_subscene_marker` — camera reframe trigger

A `_subscene_marker` step has the form:

```
{
  "_subscene_marker": "scene_25_kitchen_ss03",
  "_shot_meta": {
    "framing":    "medium",
    "subject":    "freydoon",
    "move":       "push",
    "transition": "cut",
    "lighting":   null,
    "freeform":   false
  }
}
```

When `PAM_CAMERA_MODE=1`, the main loop:

1. Looks up the marker ID in `_subscene_index` (built from `PAM_PROMPTS`), falling back to `step["_shot_meta"]` if not found.
2. Calls `_apply_camera(meta, scene, char_x_positions)` for instant reframes, or stores `_camera_anim(...)` in `_pending_camera` for animated moves to fire with the next `say`.

When `PAM_CAMERA_MODE` is unset, the entire subscene-marker block is skipped silently.

5.9 `zone_shift` — sub-location marker

A `zone_shift` step marks a sub-location within a scene:

```
{"action": "zone_shift", "zone": "secretary_s_pod",
 "label": "Secretary's pod"}
```

Used by `fountain2pam.py` to chunk per-subscene prompts; in `pam_player.py` it's an instantaneous camera-region shift (zero animation cost) and does not close the current subscene.

5.10 Scene overlays

Action	Effect
`title`	Opening title card. Should be the first action.
`caption`	On-screen text with `position` (`bottom`/`top`/`lower-third`), `duration`, `style` (`normal`/`italic`/`bold`). Fades in, holds, fades out.

Action	Effect
`persistent_caption`	Same as `caption` but doesn't auto-dismiss.
`overlay_caption`	v0.9.11 — like `caption` but with a `position: "center"` that places the bar at `_frame_cy + 0.5` (above standing characters).
`sound_cue`	Flashes a diegetic sound label on screen.
`wait`	Pauses the scene for `t` seconds.
`on_screen_text`	Multi-line text card with `hold` parameter.
`clear_bubble` / `clear_all_bubbles`	See §8.3.

---

6. The parallel block

6.1 The `_CANNOT_PARALLEL` set

`actions.py` exports a frozenset of action names that cannot run inside a `parallel` block:

```
_CANNOT_PARALLEL = frozenset({
    "walk_to", "run_to", "sit_down", "stand_up", "wave",
    "wave_left", "wave_right", "carry", "exit_through",
    "exit_through_doors", "rush_to", "rush_out", "squeeze_through",
    "jump_up", "pat", "search_drawers", "pick_up_phone", "hang_up",
    "grab", "punch_button", "reach_character", "peel_from_hand",
    "group_translate", "nod", "shake_head", "shrug",
    "grab_arm", "twist_arm_behind", "release_arm", "rotate",
})
```

When the parallel handler encounters one of these inside a `parallel` block, it prints a console warning and runs the sub-action sequentially (outside the `scene.play` batch).

Locomotion is the exception. `walk_to` and `run_to` are technically in `_CANNOT_PARALLEL` because they can't be `_collect_anims`'d generically, but the parallel handler has a *dedicated path* (§6.3) that interpolates their keyframes in lockstep.

6.2 Two-pass collection

The parallel handler partitions sub-actions into two buckets:

1. **Locomotion** — handled via the keyframe-by-keyframe special path
2. **Simple parallel-safe** — collected as Manim animation lists and played in a single `scene.play(*anims, run_time=rt)` call

Each bucket is played in its own `scene.play` invocation; they don't interleave.

6.3 Locomotion sub-actions

The locomotion path zips per-character walk keyframes:

```
loco_steps = max(fig._walk_steps(to_x) for to_x in destinations)

for step_i in range(loco_steps):
    frame_anims = []
    for mover, dest in zip(movers, destinations):
        pose, new_off = mover._walk_keyframe(step_i, dest)
        frame_anims.extend(mover._pose_anims(pose, new_off))
        mover.pose, mover.offset = pose, new_off
    scene.play(*frame_anims, run_time=loco_rt, rate_func=smooth)
```

This is the mechanism that lets Lucy and Lenny run together while Chekov trots behind — every character's footfall lands on the same beat. The `rt_per_kf` parameter on `parallel` controls the per-keyframe duration (default 0.12s).

6.4 Post-pass state updates

After the `scene.play(...)` call completes, the parallel handler runs a post-pass to update figure state for actions whose state changes happen *after* the animation rather than within it:

```
for sub in sub_actions:
    if sub["action"] == "turn":
        for tname in _targets(sub):
            fig = _get_fig(tname)
            if fig:
                fig.pose = _resolve_pose(sub.get("pose"),
                                         fig._bp["standing_side"])
    elif sub["action"] == "fade_out":
        for tname in _targets(sub):
            cast[tname]["fig"] = None
```

Without this post-pass, a `turn` inside a `parallel` block would animate the figure to the side pose but leave `fig.pose` set to `standing_front`, breaking the next `walk_to`.

---

7. The camera system

7.1 `MovingCameraScene` foundations

`PAMPlayer` inherits from Manim's `MovingCameraScene`, which exposes `self.camera.frame` — a `ScreenRectangle` whose `width` and `center` control the visible viewport. PAM's camera system manipulates this frame:

- **Width** controls zoom (smaller width = more zoomed in)
- **Center** controls pan (x for horizontal, y for vertical)

The camera is purely 2D — there's no perspective, no depth-of-field, no real "tilt." The `pan-up` action approximates a tilt by panning the frame upward while applying a shear to tall props (§7.8).

7.2 The `_FRAMING_CAMERA` dict

Module-level constant mapping framing names to `(frame_width, frame_center_y)`:

```
_FRAMING_CAMERA: dict[str, tuple] = {
    "wide":         (14.2, -0.5),   # full stage
    "medium":       (8.0,  -0.2),   # waist-up
    "medium-close": (5.5,   0.3),   # chest-up
    "close":        (3.5,   0.9),   # face and shoulders
    "ots-left":     (7.0,   0.0),
    "ots-right":    (7.0,   0.0),
    "oneshot":      (5.0,   0.3),   # single character
    "insert":       (3.0,   1.5),   # prop / detail level
}
```

Smaller width = tighter framing. `center_y` should track the visual center of the framed subject — head-level for `close`, mid-body for `medium`.

7.3 The `_MOVE_RT` dict

Maps move names to animation durations:

```
_MOVE_RT: dict[str, float] = {
    "static":     0.0,    # instant cut
    "push":       0.8,
    "pull":       0.8,
    "pan-follow": 0.5,
    "drift":      1.5,
}
```

`static` moves bypass `scene.play` entirely — the frame is repositioned instantly between actions. All other moves animate over their listed duration.

7.4 `_apply_camera` — instant and animated reframes

The authoritative camera reposition function:

```
def _apply_camera(meta, scene, char_x_positions):
    framing = meta.get("framing", "wide").lower()
    move    = meta.get("move",    "static").lower()
    subject = meta.get("subject", "ensemble").lower()

    target_w, target_y = _FRAMING_CAMERA[framing]
    rt                 = _MOVE_RT.get(move, 0.0)

    # Resolve subject to a target x
    if subject in ("ensemble", "none", ""):
        target_x = 0.0
    else:
        raw_x  = char_x_positions.get(subject, 0.0)
        half_w = target_w / 2
```

```
        target_x = float(np.clip(raw_x,
                                 -7.1 + half_w, 7.1 - half_w))

    frame = scene.camera.frame
    if rt > 0:
        scene.play(
            frame.animate.set_width(target_w)
                  .move_to([target_x, target_y, 0]),
            run_time=rt, rate_func=smooth)
    else:
        frame.set_width(target_w)
        frame.move_to([target_x, target_y, 0])
```

The clamp on `target_x` keeps the frame inside the visible stage so that following a character near the edge doesn't reveal off-stage space.

7.5 _camera_anim and _pending_camera

A variant that **returns** an animation instead of playing it. Used when a camera move should run *concurrently* with a speech bubble:

```
def _camera_anim(meta, scene, char_x_positions):
    """Return a camera animation (or None) - does NOT play it."""
    ...
    if rt == 0.0:
        return None    # static - handled by _apply_camera elsewhere
    return frame.animate.set_width(target_w).move_to(
        np.array([target_x, target_y, 0]))
```

The `_pending_camera` mechanism in `construct()`:

```
self._pending_camera = None

# When a _subscene_marker fires with an animated move:
anim = _camera_anim(meta, self, char_x_positions)
self._pending_camera = anim

# When the next `say` fires:
extra_anims = []
if self._pending_camera is not None:
    extra_anims.append(self._pending_camera)
    self._pending_camera = None
fig.say(text, scene=self, extra_anims=extra_anims, ...)
```

This makes camera moves invisible *as moves* — they feel like the camera is gliding toward the speaker during their first beat of dialogue. Without `_pending_camera`, the camera would jump, pause, then the bubble would appear separately.

If no `say` follows the marker before the next marker, the pending camera is consumed by the next marker's `_apply_camera` call directly.

7.6 The `_shot_meta` dict

Every `_subscene_marker` carries an inline `_shot_meta` dict with six keys:

Key	Values	Default
`framing`	`wide` / `medium` / `medium-close` / `close` / `ots-left` / `ots-right` / `oneshot` / `insert`	`wide`
`subject`	character key / prop key / `ensemble` / `none`	`ensemble`
`move`	`static` / `push` / `pull` / `pan-follow` / `pan-up` / `pan-down` / `drift`	`static`
`transition`	`cut` / `hold` / `hold-empty` / `smash`	`cut`
`lighting`	(lighting vocab)	`null`
`freeform`	bool	`false`

`null` values mean "inherit current state" — they're not the same as explicit defaults. A subscene with `framing: null` after a `pan-follow` will inherit the panned frame. A subscene with `framing: "wide"` will reset to wide. This distinction matters when a previous shot moved the camera.

7.7 Initial camera reset

When `PAM_CAMERA_MODE=1`, `construct()` forces the camera to wide/ensemble *before* the action loop:

```
if _camera_mode:
    init_frame = self.camera.frame
    init_frame.width = 14.2
    init_frame.move_to(np.array([0.0, -0.5, 0]))
```

Without this reset, the camera could inherit a zoomed or off-center state from a previous render in the same Manim process.

7.8 `pan-up` / `pan-down` and the tilt implementation

`pan-up` is implemented by `_execute_pan_up`, which:

1. Zooms in to `medium-close` on the named subject.
2. Pans the frame upward at the configured tilt duration (2.5s).
3. Optionally applies a vertical shear to the target building prop to fake keystone convergence.
4. Holds briefly at the top.

The target prop must have a `pam_height` attribute — `build_building` sets this, but other tall props may not. If the subject doesn't have `pam_height`, the camera tilts to a default upper-frame position.

`pan-down` mirrors the implementation, starting high and panning to ground level.

7.9 The `insert` framing for prop close-ups

`insert` is the tightest framing (frame width 3.0, center_y 1.5) intended for prop detail shots:

```
[[ CAMERA: FRAMING=insert | SUBJECT=smartphone | MOVE=null | TRANSITION=cut ]]
```

When `subject` resolves to a prop name (rather than a character), the camera centers on the prop's `pam_x` / `pam_y` instead of char_x_positions. Used for phone screens, monitor displays, audio_video_bug shots, and similar tiny-prop moments.

---

8. The bubble registry

8.1 `_persistent_bubbles` structure

A class-level dict on `PAMPlayer`:

```
_persistent_bubbles: dict[str, dict] = {}
# Structure:
#   key   = character name OR prop registry name
#   value = {"bubble": VGroup, "expires_at": float | None}
```

An entry is added whenever a `say` or `prop_say` includes either `persist: true` (no expiry — clear explicitly) or `duration: t` (expires at scene-clock + t seconds).

8.2 `_scene_clock` and `_sweep_expired_bubbles`

The scene clock is a mutable holder:

```
_scene_clock: list = [0.0]
```

It's incremented after every `scene.play(...)` and `scene.wait(...)` call by the run_time, tracking wall-clock seconds of scene time.

At the top of each main-loop iteration:

```
def _sweep_expired_bubbles(self):
    now = self._scene_clock[0]
    expired = [k for k, e in self._persistent_bubbles.items()
               if e["expires_at"] is not None and e["expires_at"] <= now]
    for k in expired:
        bubble = self._persistent_bubbles.pop(k)["bubble"]
        self.play(FadeOut(bubble), run_time=0.25)
```

This is what makes `duration: 2.5` work — the bubble persists until its expiry passes, at which point the next main-loop iteration fades it out automatically.

8.3 `clear_bubble` / `clear_prop_bubble` / `clear_all_bubbles`

Explicit-dismissal actions for `persist: true` bubbles (which never auto-expire).

```
{"action": "clear_bubble", "who": "thalia", "rt": 0.25}
{"action": "clear_bubble", "prop": "phone1"}
{"action": "clear_all_bubbles", "rt": 0.25}
```

Each looks up the entry in `_persistent_bubbles` by `who` or `prop`, plays a `FadeOut`, and removes the entry. `clear_all_bubbles` iterates the dict and fades everything concurrently in a single `scene.play` call.

`clear_prop_bubble` is a v0.9.9 alias for `clear_bubble` retained for scenes authored before the rename.

8.4 The `focus` / `focus_reset` re-paint gotcha

The general rule. `focus` and `focus_reset` animate `fig.group.animate.set_opacity(x)` on each affected figure. Because `fig.group` is a property returning `VGroup(edge_group, dot_group)`, the animation walks every edge and every joint dot — including any dot whose opacity was previously set to `0` by an attach-style action. Two consequences fire together at the end of the `play()` call:

1. Anything you had hidden by setting its opacity to `0` inside `fig.group` is re-revealed at the new target opacity.
2. Manim's painter's algorithm re-asserts the animated mobjects above anything that was painted on top of them between when they were originally added and when `focus` fired.

So any decoration that lives *above* an opacity-hidden part of `fig.group` is at risk: the part underneath comes back into view *and* jumps in front of the decoration. The rule has two known instances.

Example A: persistent speech bubbles (v0.9.10, unfixed).

Symptom. A persistent bubble (from `say` or `prop_say` with `persist: true` or `duration: t`) shows on screen. A `focus_reset` fires, restoring all characters to full opacity. The bubble is now visually overlaid by the character mobjects.

Cause. The bubble was added to the scene after the characters. `focus_reset` re-plays the characters, and the painter's algorithm puts them on top of the bubble.

Workaround. Issue a `clear_bubble` or `clear_all_bubbles` before the `focus_reset`:

```
{"action": "clear_all_bubbles"},
{"action": "focus_reset"}
```

Or tune the bubble's `duration` so it expires before the reset fires. This case is not patched in-handler because the right semantic answer ("should bubbles always render above characters?") is part of the v1.0.0 bubble-lifecycle migration — see `BACK_BURNER.md` and §8.5 (world-space anchoring), which is the larger refactor this gotcha is waiting on.

Example B: attached faces (v0.9.15, fixed in v0.9.17).

Symptom. A character has had `attach_face` applied, so a face PNG is showing on the head. A `focus` fires with `bright: 1.0` (or a `focus_reset`) targeting that character. The face PNG is replaced by the underlying head dot — a dark oval with the character's label, e.g. "Bevers".

Cause. `attach_face` hides the head dot by setting `fig.dots["head"].opacity = 0` and adds a `head_face ImageMobject` above it. The head dot is a child of `fig.dots`, hence of `fig.group`, so

the focus animation drives its opacity back to `1.0` and the play also re-orders it above `head_face`. The labeled dot reappears, clobbering the face. (At `dim` opacity the same mechanism applies but the dot only animates to `0.30`, so visually the dimmed face still dominates — which is why the bug shows asymmetrically on the brightened characters only.)

Fix (in-handler, v0.9.17, end-state repair). After each focus/reset `play()`, the handler walks every figure it touched (both brightened and dimmed) and, for any figure with a live `head_face`, reasserts `fig.dots["head"].set_opacity(0)` synchronously and calls `self.bring_to_front(fig.head_face)`. This corrected the final frame but left intermediate animation frames showing the dot peek through — visible flicker (5–10 frames at 24/30fps) on focus calls with `rt < 0.05`.

Upgrade (v0.9.18, transient elimination). The handler now selects its animation target conditionally for each figure. For figures with no `head_face`, it animates `fig.group` exactly as before. For figures with a live `head_face`, it animates a rebuilt `VGroup(fig.edge_group, *non_head_dots)` that EXCLUDES `fig.dots["head"]` entirely. The head dot is never in the animation, so no interpolated frame can show it above the face. The v0.9.17 post-play repair is retained as defensive belt-and-suspenders — its `set_opacity(0)` call is now a no-op (the dot was never touched), but its `bring_to_front(head_face)` is still meaningful for z-order reassertion if any unrelated later animation re-paints over the face. See the `_animation_group` helper in `pam_player.py`'s focus/focus_reset handler. No screenplay changes are required.

Refinement (v0.9.18.1, residual neck/shoulder flicker). Excluding the head dot left a subtler artifact: the animated group still contains `fig.edge_group` and the non-head dots, which include the neck edge and the shoulder dots/struts that a head-and-shoulders face bust covers in the static frame. Manim's `play()` re-adds the animated group to the front of the scene each frame, so those covered body elements paint ABOVE the face for the duration of the animation, then snap back behind it when the post-play `bring_to_front` runs — a brief flash of neck/shoulder lines over the face. Rather than enumerate exactly which edges and dots the bust covers (build-specific, and the set would drift as the art evolves), the handler installs a per-frame updater on each touched figure's `head_face` for the span of the `play()`. The updater re-asserts `bring_to_front(head_face)` every frame, holding the face on top regardless of what is re-added behind it. The updaters are removed immediately after the `play()`; steady-state z-order is then handled by `_repair_face_overlays` as before. Because nothing persistent changes, there is no interaction with bubble or caption z-order outside the focus window. See the `_install_face_front_keepers` / `_remove_face_front_keepers` helpers in the focus handler.

Why the two examples are treated differently. The face case has one unambiguous correct rendering: the face PNG was put there deliberately to obscure the dot, so any reveal of the dot is a bug. The bubble case is genuinely ambiguous: a persistent bubble may or may not be intended to outlast a focus reset, and the bubble's anchoring story (§8.5) is itself in flux. The face fix can be a local handler patch; the bubble fix needs the larger refactor.

Generalization to future attach methods. The mechanism is not face-specific. Any future `attach_*` method that hides part of `fig.group` by setting a child's opacity to `0` and painting something on top will collide with this rule. Two options for new code:

1. Teach the v0.9.18 `_animation_group` helper to also exclude the hidden child for any figure that has the new attached attribute. Same exclusion pattern, parameterized by the attached attribute name.

2. Avoid the hide-with-opacity pattern entirely: `scene.remove` the hidden child instead, and re-add it on `detach`.

Option (2) is the cleaner long-term design but breaks any code that expects the hidden joint dot to remain in `fig.dots` for geometry queries (`lshoulder`-style lookups, the prop attachment machinery in `_drag_attached_props`, etc.). Option (1) is what v0.9.17 chose.

8.5 Bubble world-space anchoring

Bubbles attach to a fixed world-space position at creation, then **do not track** the speaker or prop afterward. Implications:

- `say` with `persist: true` followed by `walk_to` leaves the bubble behind at the character's starting position.
- `prop_say` followed by moving the prop leaves the bubble behind.

This is why walk-and-talk dialogue cannot use `persist` — the bubble would visibly detach. The v1.0.0 bubble-lifecycle migration (figures own their bubbles, see `BACK_BURNER.md`) is intended to fix this by making the bubble follow the head joint per-frame.

For walk-and-talk today, break dialogue into short non-persisted `say` beats per walk step, or design the dialogue around standing rest points (treadmill pattern: walk-and-talk -> bench rest -> group-translate reset).

---

9. Internals quick reference

9.1 Module-level constants

Constant	Purpose
`_FRAMING_CAMERA`	framing -> (frame_width, frame_center_y)
`_MOVE_RT`	move -> animation duration in seconds
`_WIDE_W`	14.2 — full-stage frame width
`_WIDE_Y`	-0.5 — default frame center y
`_DEFAULT`	sentinel for single-character mode
`BG_COLOR`	scene background
`LABEL_COLOR`	default character label color
`PADDING_WAIT`	post-bubble pause (configurable)
`_LOCO`	set of locomotion action names

9.2 Module-level helpers

Helper	Purpose
`_resolve_pose(name, default, fig=None)`	Look up a pose by name in `POSES`
`_targets(step)`	Resolve `who` to a list of character names
`_get_fig(name)`	Look up a live figure in the cast
`_get_prop(pname)`	Look up a prop in the registry

Helper	Purpose
`_apply_camera(meta, scene, char_x_positions)`	Instant or animated reframe
`_camera_anim(meta, scene, char_x_positions)`	Returns an animation (does not play)
`_execute_pan_up(meta, scene, props, char_x_positions)`	Tilt-up implementation
`_sweep_expired_bubbles()`	Bubble bookkeeping (instance method)

9.3 Environment-variable summary

Variable	Required?	Effect
`PAM_SCRIPT`	yes	Path to JSON screenplay
`PAM_CAMERA_MODE`	no	`1` enables camera system; otherwise subscene markers are skipped
`PAM_PROMPTS`	no	Path to prompts JSON; default `<PAM_SCRIPT stem>_prompts.json`
`PAM_SHOW_CLOCK`	no	`1` displays elapsed-time overlay

9.4 Output paths

Manim writes by default to:

`./media/videos/pam_player/<quality>/<SceneName>.mp4`

. . . where `<quality>` is `480p15`, `720p30`, `1080p60`, or `2160p60` depending on the `-q` flag, and `<SceneName>` is `PAMPlayer` unless overridden with `-o`.

For convenience, the `pam-render` wrapper accepts an output name as its second argument and stages the final MP4 alongside the script.

---

10. Runtime additions in the attached v0.9.14–v0.9.X sources

This section records runtime behavior present in the attached source files but not fully covered by the earlier `pam_player.py` reference. It is intentionally practical: each entry gives the screenplay syntax, the registry affected, and the failure mode to expect while debugging.

10.1 Scene-object teardown: `remove_scene_object` and `clear_all_scene_objects`

`scene_objects` are large, camera-addressable background objects stored in the player's private `_scene_objects` registry. They are deliberately separate from interactive props in the `PropRegistry`; this avoids name collisions and keeps stage dressing from being treated as grabbable, parentable, or speech-capable props.

Two teardown verbs complete that separation:

Action	Target registry	Purpose
`remove_scene_object`	`_scene_objects` only	Fade out one named scene object.
`clear_all_scene_objects`	`_scene_objects` only	Fade out every registered scene object concurrently.

`remove_scene_object` uses the sub-key `prop` for symmetry with `remove_prop`, but it does **not** search the prop registry. If the named object is not in `_scene_objects`, the action is a silent no-op. It does not remove characters, dogs, Governor figures, or interactive props.

```
[
  {"action": "scene_objects", "items": {
    "warehouse-wall": {"type": "wall", "x": 0, "y": 0},
    "skyline":        {"type": "backdrop", "x": 0, "y": 1.5}
  }},

  {"action": "remove_scene_object", "prop": "warehouse-wall", "rt": 0.4},
  {"action": "clear_all_scene_objects", "rt": 0.5}
]
```

Use `remove_prop` for interactive props and `remove_scene_object` for non-interactive scene dressing. A common authoring error is to declare a background item under `scene_objects` and later try to delete it with `remove_prop`; the result is that nothing happens, because the item is not in the prop registry.

10.2 Render-time preview clock

The player supports a preview clock overlay controlled by `PAM_SHOW_CLOCK=1`, or by the `pam-render --show-clock` wrapper. The overlay appears near the title bar in the upper-right and shows elapsed scene time as `M:SS.ss`. It is meant for low-quality preview renders, especially when tuning dialogue holds, camera drift, pauses, or synchronized entrances.

```
PAM_SCRIPT=my_scene.json PAM_SHOW_CLOCK=1 manim -pql pam_player.py PAMPlayer
./pam-render --script my_scene.json --show-clock
```

Do not use the clock for final renders; it is a timing diagnostic.

10.3 Path C speaker resolution and dual registration

The attached `actions.py` adds `_resolve_speaker(step, cast, props)`, a small but important target-resolution helper. It reads `who` first, then `prop`, and looks the resulting name up in both registries. The purpose is to make dog-like figures work consistently whether an action reaches them through the cast side or the prop side.

The practical rule is:

Object kind	Cast registry	Prop registry	Notes
Human / alien character	yes	no	Normal biped figure.
Plain prop	no	yes	Desk, phone, door, button panel, etc.
DogGraph / quadruped figure	yes	yes	Dual-registered under the same key.

For authoring, this means a dog can be addressed consistently by `who` in character-like actions and by `prop` in prop-like or speech actions:

```
[
  {"action": "fade_in",  "who": "chekov"},
  {"action": "trot_to",  "who": "chekov", "x": 2.0},
  {"action": "prop_say", "prop": "chekov", "text": "Woof."},
  {"action": "react",    "who": "chekov"}
]
```

The helper is deliberately quiet on misses. Individual action handlers decide whether a missing target is an error, a warning, or a no-op.

10.4 Explicit bubble colors on `prop_say` and figure speech

The attached player and figure code allow speech bubbles to be colored explicitly. For ordinary character `say`, the figure style may define `bubble_color` and `bubble_text_color`. For prop-style speech, the step can supply explicit colors.

```
{
  "action": "cast",
  "characters": {
    "nona": {
      "figure_type": "alien",
      "style": {
        "edge_color": "#8a6030",
        "bubble_color": "#8a6030",
        "bubble_text_color": "#201408"
      }
    }
  }
}
```

```
{"action": "prop_say", "prop": "governor",
 "text": "Processing.",
 "bubble_color": "#382050",
 "text_color": "#ffffff",
 "border_color": "#c8a020"}
```

If no explicit speech-bubble color is provided, character speech falls back to the figure's `edge_color`. The human/alien `say()` implementation uses the bubble color as a saturated border and derives a

pale tinted-white fill so that dark text remains legible.

End of pam_player-manual.md v0.9.13 draft. Companion to `CHARACTER_PROPERTIES.md` *and* `PROP_PROPERTIES.md`*. Revision pass expected after the v1.0.0 cleanup of* `to_x/x` *normalization,* `group_translate` *parallel-safety, and the bubble-lifecycle migration.*

Character Properties Reference

Overview

Version basis: PAM v0.9.15.

A complete reference for everything that can be attached to or done with a character in PAM. Use this document as the single source of truth for character-side authoring; the main README links here rather than duplicating these details.

The reference is organized by **lifecycle**:

1. **Creation-time properties** — fields on a cast-block entry. What a character *is*.
2. **Modifier actions** — change character state without locomotion or dialogue.
3. **Action targets** — verbs that take a character as primary target.
4. **Meta** — name resolution, the `hidden` flag, z-order, the color authority chain, registry behavior.
5. **Face & Appearance** — face data, expression variants, `attach_face`, hats, layering, and face debugging.

This document is intended to evolve into the v1.0.0 frozen reference. Where current behavior is provisional or under active design, the relevant subsection is flagged.

Character Properties Sections

1. Creation-time properties

Characters are declared in a `cast` action block, typically at the top of a screenplay:

```
{
  "action": "cast",
  "characters": {
    "bevers": {
      "figure_type": "alien",
      "build": "alien",
      "gender": "male",
      "pose": "standing_front",
      "offset": [3, 0, 0],
      "scale": {"sy": 0.7, "sx": 0.7, "anchor": "lankle"},
      "color": "#44bb66",
      "torso_color": "#2a6090",
```

```
      "style": {"head_label": "Bevers"},
      "tic_profile": [{"fragment": "hand_twitch_r", "triggers": ["react"]}]
    }
  }
}
```

Every key inside a character entry except the cast-key itself (`"bevers"` above) is optional in principle, but `figure_type` should always be set explicitly to avoid the default-human fallback misclassifying alien or quadruped characters.

The same fields are also accepted in a single-character `fade_in` step, which spawns a character mid-scene without a prior `cast` entry:

```
{"action": "fade_in", "who": "athena",
 "figure_type": "human", "build": "narrow", "gender": "female",
 "offset": [-2, 0, 0]}
```

For multi-scene projects, store canonical character entries once in `tntd_characters.json` (or your project's equivalent) and let `fountain2pam.py` merge them into each scene's cast block at conversion time. See §[?].

1.1 `figure_type`

Type: `str` · *Default:* `"human"`

The figure class used to render the character. Determines skeleton topology, locomotion vocabulary, and which actions are meaningful.

Accepted values:

- `"human"` — `HumanGraph`. 15-vertex biped skeleton. Full action vocabulary including walk, run, gesture, restraint, prop interaction.
- `"alien"` — `AlienGraph`. Subclass of `HumanGraph` with a split-torso joint structure (extra `torso_left`/`torso_right` joints). All human actions work; in addition, alien-specific poses live in the build's pose registry. Use `build="alien"` or `build="alien_female"` to match.
- `"dog"` — `DogGraph`. 19-joint quadruped, side-view default. Locomotion is `trot_to` (not `walk_to` / `run_to`); `say` is a no-op for dogs — use `prop_say` keyed on the dog's name. Path C: dual-registered as both a cast member and a prop, so `who: chekov` and `prop: chekov` both resolve.
- `"dodecahedron"` — `GovernorGraph`. Procedurally-animated dodecahedron (Schlegel diagram or 3D spinning, depending on `style`). Used for AI characters like the Governor. Rotates autonomously; `say` is a no-op — use `prop_say`.

Default behavior if omitted: treated as `"human"`. This is a known footgun for alien characters — always set `figure_type` explicitly when declaring an alien.

Example:

```
"chekov":   {"figure_type": "dog"},
"governor": {"figure_type": "dodecahedron"},
"thalia":   {"figure_type": "alien", "build": "alien_female", "gender": "female"}
```

1.2 `build`

Type: `str` · *Default:* derived from `gender`

The proportions preset. Controls skeleton dimensions (shoulder width, leg length, torso height) and certain build-specific poses. Builds are defined in `pam/builds.py`.

Accepted values:

- `"default"` — neutral male-coded proportions. Default for `gender="male"`.
- `"narrow"` — narrower shoulders, slightly shorter. Default for `gender="female"` and `gender="child"`.
- `"broad"` — wider shoulders, heavier frame.
- `"alien"` — alien proportions with split torso. Use with `figure_type="alien"`, `gender="male"`.
- `"alien_female"` — alien proportions, narrower variant. Use with `figure_type="alien"`, `gender="female"`.

Interaction with `gender`: if `build` is omitted, the value is inferred from `gender` via `GENDER_DEFAULTS` (in `figure.py`). Setting `build` explicitly always wins.

For aliens: the human builds (`default`, `narrow`, `broad`) have no `torso_left`/`torso_right` joints, so they cannot render alien poses. An alien character with `build="default"` will render with human proportions but break on alien-specific morphs. Always pair `figure_type="alien"` with `build="alien"` or `build="alien_female"`.

Example:

```
"chava":  {"figure_type": "human", "build": "narrow", "gender": "female"},
"factor": {"figure_type": "alien", "build": "alien",  "gender": "male"},
"thalia": {"figure_type": "alien", "build": "alien_female", "gender": "female"}
```

1.3 `gender`

Type: `str` · *Default:* `"male"`

Drives the build inference (see [**?**]) and is consumed by downstream tooling (e.g. voice casting in any future text-to-speech integration). Does not directly affect rendering unless `build` is also omitted.

Accepted values: `"male"`, `"female"`, `"child"`.

Default warning: the converter prints a warning if `gender` is missing and defaults to `"male"`. Always set explicitly.

Example:

```
"bart": {"figure_type": "human", "gender": "child"}
```

1.4 `color`

Type: `str` (hex string) · *Default:* build's `edge_color`

Single-color shorthand for the character's edge/stroke color. Sets `edge_color` on the style palette; the remaining palette keys (`node_color`, `node_stroke`, `head_color`, `head_stroke`, `highlight_color`) are derived from `color` via `_style_from_color` in `figure.py`.

Interaction with `style`: explicit values inside `style` override anything derived from `color`. Common pattern: set `color` and `torso_color` at the top level, then add `head_label` (and only `head_label`) in `style`.

Authority: when `tntd_characters.json` (or another canonical registry) is in play, the registry's `color` is authoritative — `fountain2pam.py` re-derives the full palette from the canonical `color` and overwrites any per-scene values during conversion. See §1.13.

Example:

```
"bevers": {
  "color": "#44bb66",
  "torso_color": "#2a6090",
  "style": {"head_label": "Bevers"}
}
```

1.5 `torso_color`

Type: `str` (hex string) · *Default:* none (single-zone rendering)

Two-zone color (v0.9.4). When present, the torso edges (`lshoulder`–`torso`, `rshoulder`–`torso`, `lhip`–`torso`, `rhip`–`torso`) and torso fill are drawn in `torso_color` instead of the main `edge_color`. Reads as a uniform, shirt, or sash.

Omit for: single-color rendering — character drawn entirely in `edge_color` / derived palette.

Example:

```
"chava":  {"color": "#e8943a", "torso_color": "#7a4010"},
"thalia": {"color": "#e055aa", "torso_color": "#2a6090"}
```

Pair with `uniforms` ([?]) for named torso variants that swap via the `change_uniform` action.

1.6 `style`

Type: `dict` · *Default:* `DEFAULT_STYLE` (or derived from `color`)

Explicit palette and label overrides. Every key inside `style` is optional; missing keys fall through to the derived palette.

Sub-keys:

Key	Type	Default	Description
`edge_color`	`str` (hex)	`"#3a7bd5"`	Bone/edge color. Overridden by top-level `color` if present, then by `style.edge_color` if present (style wins).
`node_color`	`str` (hex)	`"#1e3a5f"`	Joint fill color.
`node_stroke`	`str` (hex)	`"#5b9cf6"`	Joint outline color.
`head_color`	`str` (hex)	`"#0d2340"`	Head circle fill.
`head_stroke`	`str` (hex)	`"#7ec8ff"`	Head circle outline.
`head_label`	`str`	`"v_0"`	Text inside the head circle. Multi-character labels work but are visually cramped; one-to-two characters reads cleanest at typical scales.
`head_font`	`str`	`"Courier New"`	Font family for the head label *and* speech bubbles.
`head_font_sz`	`int`	`14`	Font size for the head label.
`head_radius`	`float`	`0.28`	Head circle radius in world units. `DogGraph` defaults to `0.22`.
`node_radius`	`float`	`0.14`	Joint circle radius. `DogGraph` defaults to `0.11`.
`edge_width`	`float`	`2.5`	Stroke width for edges.
`highlight_color`	`str` (hex)	`"#7ec8ff"`	Color used by `highlight_edges()` and the v0.9.15 `attach_face` halo (not implemented in current MVP).

DogGraph-additional sub-keys:

Key	Type	Default	Description
`far_edge_color`	`str` (hex)	matches `edge_color`	Far-side legs (perspective cue).
`far_edge_opacity`	`float`	`0.35`	Opacity for far-side legs.
`far_node_opacity`	`float`	`0.30`	Opacity for far-side joints.

Key	Type	Default	Description
`far_edge_width`	`float`	`1.5`	Stroke width for far-side legs.

Example — full palette:

```
"bevers": {
  "style": {
    "head_label": "Bevers",
    "edge_color": "#44bb66",
    "node_color": "#2a6090",
    "node_stroke": "#6ce38e",
    "head_color": "#08170c",
    "head_stroke": "#80f7a2",
    "highlight_color": "#94ffb6"
  }
}
```

Example — label only (palette derived from `color`):

```
"chava": {
  "color": "#e8943a",
  "torso_color": "#7a4010",
  "style": {"head_label": "Chava"}
}
```

1.7 `pose`

Type: `str` · *Default:* `"standing_front"`

The initial pose at fade-in. Must be a key in the `POSES` registry (`pam/poses.py`) or in the character's build-specific pose dict.

Common values:

- `"standing_front"` — facing camera. Default. Required for `say` to read correctly in scenes where the bubble lands above the head.
- `"standing_side"` — side-view. Required for walking, running, and any locomotion verb. `say` works in `standing_side` without constraint.
- `"sitting_mid"`, `"sitting_down"` — mid-sit and fully-seated. Used after `sit_down`.
- `"lying_flat_right"`, `"lying_flat_left"` — horizontal poses (v0.9.14). Speech bubble anchoring works correctly only with these named poses; an ad-hoc rotation will mis-anchor bubbles.
- `"at_attention"` — feet together, arms at sides. Formal stance.
- `"look_up"`, `"look_up_point"` — head tilted up. Used for sky-watching beats.
- Dog: `"dog_standing"`, `"dog_trot_a"`, `"dog_trot_b"`.

Full pose registry: see `POSES` in `pam/poses.py`. Categories: standing · grappled · sitting · lying · wave (right-arm) · lwave (left-arm) · walk · run · carry (chest) · side_carry (briefcase

low) · look · attention · reach · rush · squeeze · fall · dodge · jump · pat · dog.

Build awareness: if the character's build registers its own poses (e.g. alien builds with split-torso variants), those override the global `POSES` entry of the same name. An alien character in `"standing_front"` gets the alien-skeleton standing pose, not the human one.

Example:

```
"freydoon": {"pose": "standing_front"},
"sergeant": {"pose": "at_attention"},
"chekov":   {"figure_type": "dog", "pose": "dog_standing"}
```

1.8 `offset`

Type: `list` of 2 or 3 floats · *Default:* `[0, 0, 0]`

World-space position at fade-in, as `[x, y, z]`. The third component is conventionally `0` (PAM is 2D); a 2-element list is also accepted.

Coordinate system:

- `x` ranges roughly `-7.1` to `+7.1` for a default-framing camera. Negative is left, positive is right.
- `y = 0` is the ground line. Positive `y` raises the character. For a character standing on a raised surface (a step, a platform), set `y` to the surface height.
- `z` is unused in current rendering but reserved for future depth.

Relation to `walk_to` / `run_to`: locomotion verbs update the figure's *runtime* offset but do not write back to the cast spec. A character who walks left and then is faded out + faded back in returns to their original cast-spec `offset`, not their walked-to position. Authors who want continuity across fade cycles must `walk_to` the desired x explicitly after fade-in.

Example:

```
"bevers":  {"offset": [3, 0, 0]},
"freydoon": {"offset": [-5.5, 0, 0]},
"narrator": {"offset": [0, 2.5, 0]}
```

1.9 `scale`

Type: `dict` with keys `sy`, `sx`, `anchor` · *Default:* `1.0` uniform

Initial size, expressed as separate y and x scale factors around an anchor joint. Different from the `scale` *action* (which animates a scale change live).

Sub-keys:

Key	Type	Description
`sy`	`float`	Vertical scale. `1.0` = full size; `0.7` is canonical for TNTD's reference framing.

Key	Type	Description
`sx`	`float`	Horizontal scale. Typically equal to `sy` for uniform scaling; setting `sx < sy` produces a thinner silhouette.
`anchor`	`str`	Joint name that stays fixed during scaling. `"lankle"` (left ankle) keeps the character standing on the ground line as size changes. Other useful anchors: `"torso"` (centroid), `"head"` (for "looming" effects).

Shorthand: a bare float is accepted and expanded to `{"sy": v, "sx": v, "anchor": "lankle"}`. This is the form `fountain2pam.py` writes when `scale=0.48` is set in a Fountain+ `CHARACTER` annotation.

TNTD canonical values:

- `0.7` — adult character, standard framing.
- `0.65` — slightly smaller for narrower or background characters.
- `0.48`, `0.32` — distant or miniaturized (e.g. Chava observed through a window).

Node radius and scaling: prior to v0.9.x patches, joint node radii were set at construction time and did not scale with `sy`/`sx`, producing oversized nodes on a scaled-down character. This is fixed in current `figure.py`; node radius scales with the character.

Example:

```
"athena":  {"scale": {"sy": 0.65, "sx": 0.65, "anchor": "lankle"}},
"freydoon": {"scale": {"sy": 0.7,  "sx": 0.7,  "anchor": "lankle"}},
"distant_chava": {"scale": {"sy": 0.32, "sx": 0.32, "anchor": "lankle"}}
```

1.10 `facing` (dog only)

Type: `str` · *Default:* `"right"` · *Applies to:* `figure_type: "dog"`

Direction the dog faces at spawn time. Affects which side of the skeleton is drawn solid vs. dashed (near-side legs solid, far-side dashed).

Accepted values: `"left"`, `"right"`.

Has no effect on `HumanGraph` / `AlienGraph` / `GovernorGraph` — those use the `pose` field to express facing (`standing_side` is right-facing by default; use `pose="standing_side"` plus a `turn` action to face left).

Example:

```
"chekov": {"figure_type": "dog", "facing": "left",
           "offset": [-2, 0, 0]}
```

1.11 `uniforms`

Type: `dict` mapping name -> uniform-spec · *Default:* none

Named torso-color variants for the `change_uniform` action. Each uniform may set `torso_color` (required), `color` (optional, overrides the character's main edge color), and `description` (free-form annotation for authoring; ignored at runtime).

Reserved uniform name: `"default"` is the uniform applied at fade-in. If you declare a `uniforms` dict, also declare `"default"` so the character has a baseline to return to.

Example:

```
"chava": {
  "color": "#e8943a",
  "torso_color": "#7a4010",
  "uniforms": {
    "default":  {"torso_color": "#7a4010",
                 "description": "Default spy look - dark brown torso"},
    "delivery": {"torso_color": "#6b4226", "color": "#c88040",
                 "description": "Delivery disguise - warm brown uniform"},
    "casual":   {"torso_color": "#e8b830",
                 "description": "Off-duty - bright golden yellow torso"}
  }
}
```

Authors switch uniforms mid-scene with:

```
{"action": "change_uniform", "who": "chava", "uniform": "delivery"}
```

Full `change_uniform` parameters are documented in the section on modifier actions.

1.12 `tic_profile`

Type: `list` of fragment-spec dicts · *Default:* none

Speech-tic profile (v0.9.14). A list of small motion fragments that fire on specified triggers — a fidget on `react`, a tail-wag at `sentence_end`, etc.

Spec structure: each entry is a dict with:

Key	Type	Description
`fragment`	`str`	Fragment name from `TIC_FRAGMENTS` in `pam/tics.py`. See the registry table below for current fragments and their body-type applicability.

Key	Type	Description
`triggers`	`list[str]`	Trigger names that fire this fragment. Built-in triggers: `"react"` (fired by the explicit `react` action), `"sentence_end"` (fired automatically after every `say` / `prop_say`).

TIC_FRAGMENTS registry: the full set of fragments currently defined in `pam/tics.py`. Each fragment names the joint it animates, a delta-based cycle, and the figure types it applies to.

Fragment	Joint	Applies to	Description
`"hand_twitch_r"`	`rwrist`	`human`, `alien`	Right-hand fidget: small outward jiggle with a slight up-lift, then inward overshoot return. Reads as "fidget", not "flail."
`"hand_twitch_l"`	`lwrist`	`human`, `alien`	Left-hand fidget. x-mirror of `hand_twitch_r` (v0.9.18). Use for left-handed characters or staging beats where the right hand is occupied.
`"foot_tap_r"`	`rankle`	`human`, `alien`	Right-foot tap: pure-vertical lift then drop with a small overshoot below the rest position before auto-return (v0.9.18).
`"foot_tap_l"`	`lankle`	`human`, `alien`	Left-foot tap. Same vertical motion as `foot_tap_r` on the opposite ankle (v0.9.18).
`"tail_wag"`	`tail`	`dog`	Quadruped tail back-and-forth swing. Three keyframes (up-forward, down-back, up-forward) before auto-return for a clearly oscillating motion.

Body-type filtering: the tic system silently skips fragments whose `applies_to` list doesn't include the character's `figure_type`. A `tail_wag` declared on a biped is a no-op; a `hand_twitch_r` declared on a dog is a no-op.

Per-step opt-out: any individual `say` / `prop_say` / `react` step can set `"suppress_tics": true` to skip the otherwise-automatic `sentence_end` trigger for that step.

Example — Bevers's anxious-hand-fidget:

```
"bevers": {
  "figure_type": "alien",
```

```
  "tic_profile": [
    {"fragment": "hand_twitch_r", "triggers": ["react"]}
  ]
}
```

Example — left-handed variant for a hand-occupied beat. If a scene stages the right hand off camera or holding a prop, swap to `hand_twitch_l` so the fidget reads:

```
"bevers": {
  "figure_type": "alien",
  "tic_profile": [
    {"fragment": "hand_twitch_l", "triggers": ["react"]}
  ]
}
```

Example — a foot-tap on every sentence end. The `foot_tap_*` fragments fire on `sentence_end` as well as `react`, so a character can develop an impatience read across a whole dialogue scene without per-line authoring:

```
"thalia": {
  "figure_type": "alien",
  "tic_profile": [
    {"fragment": "foot_tap_r", "triggers": ["sentence_end"]}
  ]
}
```

Example — combining two fragments on one character. A character can have multiple entries in `tic_profile`, each with its own trigger set:

```
"freydoon": {
  "figure_type": "alien",
  "tic_profile": [
    {"fragment": "hand_twitch_r", "triggers": ["react"]},
    {"fragment": "foot_tap_l", "triggers": ["sentence_end"]}
  ]
}
```

Example — Chekov's tail-wag on every sentence end:

```
"chekov": {
  "figure_type": "dog",
  "tic_profile": [
    {"fragment": "tail_wag", "triggers": ["sentence_end", "react"]}
  ]
}
```

See `pam/tics.py` for fragment definitions and the design rationale for delta-based fragments.

1.13 Field precedence and the canonical registry

In multi-scene projects, characters typically have a canonical definition in `tntd_characters.json` (or your project's equivalent registry file). `fountain2pam.py` merges this into each scene's cast block at conversion time.

Canonical (registry-stored) fields:

- `figure_type`, `build`, `gender`
- `color`, `torso_color`, `style`
- `default_scale` (the default scale at which the character renders; stored in the registry as a convenience)

Scene-specific (per-scene only) fields:

- `offset`, `scale`, `pose`
- `facing` (dog only)

The registry is loaded *before* conversion: missing canonical fields in a scene's cast block are filled in from the registry. The scene always wins for scene-specific fields — a character can stand at different `offset`s in different scenes.

Color authority (v0.9.8 fix): if the registry has a `color` field, the converter re-derives the full six-key palette from it and stamps it into the character's `style` dict, overwriting any per-scene values during conversion. This means: to change a character's canonical color, edit the registry file directly. Per-scene `color` and `style.edge_color` overrides are not durable across registry-driven conversions.

Registry path: `fountain2pam.py` looks for `characters.json` next to the `.fountain` file by default. Override with `--characters /path/to/tntd_characters.json`.

Example registry entry:

```
{
  "bevers": {
    "figure_type": "alien",
    "build": "alien",
    "gender": "male",
    "color": "#44bb66",
    "torso_color": "#2a6090",
    "style": {"head_label": "Bevers"},
    "default_scale": {"sy": 0.7, "sx": 0.7, "anchor": "lankle"},
    "tic_profile": [
      {"fragment": "hand_twitch_r", "triggers": ["react"]}
    ]
  }
}
```

A per-scene cast block that references `"bevers": {"offset": [3, 0, 0]}` will receive all canonical fields above merged in automatically.

Sections 2 (modifier actions), 3 (action targets), and 4 (meta) are delivered in subsequent substeps. This document will be consolidated into a single `CHARACTER_PROPERTIES.md` when all sections are complete.

2. Modifier actions

A *modifier action* changes character state without locomotion or dialogue. This section covers visibility (`fade_in`, `fade_out`), transforms (`morph`, `scale`, `rotate`), decoration (`change_uniform`, `attach_face`, `detach_face`), framing (`focus`, `focus_reset`), and bubble cleanup (`clear_bubble`, `clear_all_bubbles`).

Throughout, each action's heading carries three flags:

- **Parallel-safe** — whether the action may appear inside a `parallel` block. Unsafe actions fire sequentially even when nested inside `parallel`, with a console warning.
- **Animation** — `none` (instant state change), or `timed` (the action consumes scene time via a `play()` call). Timed actions accept `rt` / `duration` to override the default.
- **Required keys** — minimum JSON shape that does something useful.

Cross-references to actions documented in §3 (`walk_to`, `say`, etc.) use forward links rather than duplicating parameters.

2.1 `fade_in`

Parallel-safe: no · *Animation:* timed · *Required keys:* `who`

Bring a character onto the screen with an animated entrance (edges draw first, then joints grow). Constructs the appropriate figure class based on `figure_type` and writes the live figure into `cast[name]["fig"]`.

JSON shape (minimum, character already in cast):

```
{"action": "fade_in", "who": "bevers"}
```

JSON shape (mid-scene spawn, character not in cast): all the creation-time fields from §1 may be supplied inline.

```
{"action": "fade_in", "who": "athena",
 "figure_type": "human", "build": "narrow", "gender": "female",
 "offset": [-2, 0, 0], "pose": "standing_front",
 "scale": {"sy": 0.7, "sx": 0.7, "anchor": "lankle"},
 "color": "#5b9cf6",
 "style": {"head_label": "Athena"}}
```

Parameters:

Key	Type	Required	Default	Description

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key from the cast block, or a new name to spawn fresh.
`figure_type`	`str`	no	from cast, else `"human"`	See §1.1.
`build`	`str`	no	from cast, else `"default"`	See §1.2.
`gender`	`str`	no	from cast, else `None`	See §1.3.
`offset`	`list[float]`	no	from cast, else `[0,0,0]`	World-space spawn position (3-element).
`x`	`float`	no	—	Convenience: sets `offset = [x, 0, 0]`. Overrides `offset` if both are present.
`pose`	`str`	no	from cast, else `"standing_front"`	Initial pose name. See §1.7.
`scale`	`dict`	no	from cast	`{sy, sx, anchor}` dict. See §1.9.
`style`	`dict`	no	from cast	Full style palette. See §1.6.
`color`	`str`	no	from cast	Single-color shorthand. See §1.4.
`torso_color`	`str`	no	from cast	Two-zone torso color. See §1.5.
`facing`	`str`	no	from cast, else `"right"`	Dog-only direction. See §1.10.
`rt`	`float`	no	figure default	Edge fade-in time in seconds.
`duration`	`float`	no	—	Synonym for `rt`.

Behavior:

- All canonical fields (figure_type, build, gender, color, torso_color, style) fall through from the cast spec if not given on the `fade_in` step.
- Scene-specific fields (offset, scale, pose, facing) likewise fall through from the cast spec.

- The `tic_profile` from the cast spec is wired onto the live figure as `fig.tic_profile` so trigger handlers can read it without a cast-side lookup.
- Dog figures (Path C, v0.9.14) are dual-registered: a `DogGraph` faded in via `fade_in` is also added to the prop registry under the same name, so `who: chekov` and `prop: chekov` both resolve. Skipped if a prop with that name already exists.

Defaults derive from the cast spec, not the cast registry file: if `tntd_characters.json` has Bevers's color but Bevers is missing from the cast block of this scene, `fade_in` won't find anything — the registry merge happens at `fountain2pam.py` conversion time, not at `pam_player.py` runtime.

Re-fade-in: calling `fade_in` for a character who has been faded out is supported. The figure is rebuilt fresh from the cast spec; any prior runtime state (walked-to position, attached face, applied uniform) is gone. This is the documented mechanism for non-persistence of face attachment (see §2.7).

2.2 `fade_out`

Parallel-safe: yes · *Animation:* timed · *Required keys:* `who`

Fade the character off-screen with a synchronized fade of edges and joints, and clear their cast slot. Any attached face is torn down concurrently with the body (v0.9.15).

JSON shape:

```
{"action": "fade_out", "who": "bevers", "rt": 0.8}
```

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key. `"all"` is supported as a convenience to fade out every active cast member sequentially.
`rt`	`float`	no	`1.0`	Fade-out duration in seconds.
`duration`	`float`	no	—	Synonym for `rt`.

Behavior:

- `cast[name]["fig"]` is cleared to `None` after the fade.
- If `fig.head_face` is set (v0.9.15 face attachment), the updater is removed and `FadeOut(face)` is added to the same `play()` call as the body fades — single concurrent animation.
- The figure object itself is discarded; opacity state on its head dot is not restored. A subsequent `fade_in` builds a fresh figure.

Parallel-safe because the action collects animations via the `_collect_anims` path and returns a list of `FadeOut` mobjects for the parallel composer.

2.3 `morph`

Parallel-safe: yes · *Animation:* timed · *Required keys:* `who`, `pose`

Animate the character from its current pose to a new named pose, with an optional position offset. The character's `fig.pose` and `fig.offset` are updated to match.

JSON shape:

```
{"action": "morph", "who": "bevers", "pose": "standing_side", "rt": 0.4}
```

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`pose`	`str`	yes	—	Target pose name from `POSES` registry (see §1.7). Build-specific poses override global registry.
`rt`	`float`	no	`0.4`	Morph duration in seconds.
`dx`	`float`	no	`0.0`	Add to the figure's current x-offset. Useful for poses where the visual center shifts.
`dy`	`float`	no	`0.0`	Add to the figure's current y-offset.

Examples:

```
{"action": "morph", "who": "freydoon", "pose": "look_up"}
{"action": "morph", "who": "thalia",   "pose": "standing_side", "rt": 0.3}
{"action": "morph", "who": "brad",     "pose": "lying_flat_right", "dy": -0.3}
```

Notes:

- For initial poses at fade-in, use the cast spec's `pose` field (§1.7) rather than a separate `morph` step.
- For posture transitions (`sit_down`, `stand_up`), prefer the dedicated action (§3) which handles the multi-keyframe cycle correctly.

- Morphing back to `standing_front` undoes any prior `rotate` (since `rotate` doesn't update internal pose; see §2.5).

2.4 `scale`

Parallel-safe: yes · *Animation:* timed · *Required keys:* `who`

Apply a persistent scale to the figure. Unlike the creation-time `scale` dict (§1.9), this is an action that animates the scale change live. The new scale persists until the next `scale` action.

JSON shape:

```
{"action": "scale", "who": "chava", "sy": 0.48, "sx": 0.48,
 "anchor": "lankle", "rt": 0.6}
```

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`sy`	`float`	no	`1.0`	New vertical scale. **Not a multiplier** — replaces the previous `sy` outright.
`sx`	`float`	no	`1.0`	New horizontal scale.
`anchor`	`str`	no	`"lankle"`	Joint name to keep fixed during the rescale. See §1.9.
`rt`	`float`	no	`0.8`	Scale duration in seconds.

Note on the "not a multiplier" semantics: if Chava is at scale 0.7 and you write `{"action": "scale", "who": "chava", "sy": 0.5}`, her new scale is 0.5 (not 0.35). To halve a character, compute the target value yourself.

Use cases:

- Distance suggestion — drop a character to scale 0.32 to read as "across the room" or "outside the window."
- Growth or shrinking effects — sci-fi avatar transitions.
- Establishing-shot to close-up transitions when paired with a camera move.

2.5 `rotate`

Parallel-safe: no · *Animation:* timed · *Required keys:* `who` or `prop`, plus an angle

Rotate a character or a prop around a chosen pivot point. Useful for laying a character flat on a stretcher, knocking a figure off-balance, falling, opening a pod lid, or swinging a door on its hinge.

JSON shape:

```
{"action": "rotate", "who": "freydoon",
 "angle_deg": -90, "pivot": "bottom", "rt": 0.6}
```

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	one of	—	Character key. Mutually exclusive with `prop`; if both are given, `prop` wins.
`prop`	`str`	one of	—	Prop registry key.
`angle_deg`	`float`	one of	—	Rotation angle in degrees. Preferred.
`angle`	`float`	one of	—	Rotation angle in radians. Used if `angle_deg` is absent.
`pivot`	`str` or `list`	no	`"bottom"`	One of `"bottom"`, `"center"`, `"top"`, `"left"`, `"right"`, or an explicit `[x, y]` world point. For characters, `"bottom"` is the feet; for props, the floor line.
`rt`	`float`	no	`0.5`	Animation duration. `0` produces an instant non-animated rotation.

Examples:

```
{"action": "rotate", "who": "freydoon", "angle_deg": -90, "pivot": "bottom"}
{"action": "rotate", "prop": "bevers_pod", "angle_deg": 70, "pivot": "left"}
{"action": "rotate", "prop": "stretcher",  "angle_deg": 4,  "rt": 0.15}
```

Caveat — characters only: rotation is a visual-only transform. The figure's internal `pose` and `offset` are not updated. If you `morph`, `walk_to`, or otherwise reanimate a rotated character, the next `morph_to` snaps it back to upright. Issue a counter-rotation (`angle_deg: +90` after a `-90` lay-flat) before further animation.

Props do not have this caveat — their geometry is rotated permanently and subsequent `move_aside` / `group_translate` / `remove_prop` operations compose correctly with the rotation.

Interaction with `attach_face` (v0.9.15): the face mobject tracks head position but does not rotate with the body. A character rotated to lie flat will have a horizontal body and an upright face — cartoon convention. See §2.7 for the design note.

2.6 `change_uniform`

Parallel-safe: no · *Animation:* timed · *Required keys:* `who`

Apply a named uniform variant from the character's `uniforms` dict (§1.11). Updates the torso color (and optionally the edge color) live; the variant persists until the next `change_uniform`.

JSON shape:

```
{"action": "change_uniform", "who": "chava", "uniform": "delivery", "rt": 0.3}
```

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`uniform`	`str`	yes (or override)	—	Variant name from the character's `uniforms` dict.
`torso_color`	`str` (hex)	no	—	Inline override. Used if `uniform` is absent or the named variant doesn't define `torso_color`.
`color`	`str` (hex)	no	—	Inline override for the edge color. Updates all non-torso edges and joints.

Key	Type	Required	Default	Description
`rt`	`float`	no	`0.3`	Hold time after applying. Not a true animation — the color change is currently instant; `rt` is a `wait()` so nearby actions have breathing room. Pass `0` to skip the wait.

Resolution order for `torso_color`:

1. The named uniform's `torso_color`.
2. Inline `torso_color` on the step.
3. Cast spec's top-level `torso_color`.

If none of the three resolves, the action prints a warning and skips.

Examples:

```
{"action": "change_uniform", "who": "chava", "uniform": "delivery"}
{"action": "change_uniform", "who": "nona",  "uniform": "formal"}
{"action": "change_uniform", "who": "chava", "uniform": "default"}
```

Reserved uniform name: `"default"` is the baseline — declare it in `uniforms` if you want a stable name to return to.

2.7 `attach_face`

Parallel-safe: yes (returns empty animation list) · *Animation:* none · *Required keys:* `who`, `image` · *Version:* v0.9.15

Attach a PNG face image to a character's head dot. The image is loaded as an `ImageMobject`, sized via the `scale` parameter, positioned at the head dot's current center, and given a Manim updater so it tracks the head every frame (through walks, morphs, scale changes, rotations). The head dot is hidden via opacity-0 so the face replaces it visually.

JSON shape:

```
{"action": "attach_face", "who": "freydoon",
 "image": "bevers_neutral_face.png", "scale": 0.4}
```

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.

Key	Type	Required	Default	Description
`image`	`str`	yes	—	Filename in `pam/assets/`. PNG, JPG, or JPEG. Resolved to `<pam package dir>/assets/<filename>`.
`scale`	`float`	no	`1.0`	Explicit scale factor applied to the loaded image. Tune per face graphic.

Behavior:

- The head dot's VGroup opacity is set to 0 — both circle and label are hidden. The dot itself persists as the position anchor for the updater and for `fig.say()` bubble anchoring.
- The face mobject is stored on `fig.head_face`, with the updater on `fig.head_face_updater`.
- If a face is already attached, the old one is torn down (updater removed, mobject removed from scene, head dot opacity restored) before the new one is attached. This gives `change_face` semantics for free — re-call `attach_face` with a different filename to swap.
- No fade-in animation — the face snaps in instantly. Parallel-safe.

Silent skips (graceful no-ops with console warning):

- Character has no `"head"` entry in `fig.dots` (quadrupeds, future body types).
- `image` key missing.
- File extension not in PNG/JPG/JPEG.
- File not found at the resolved path.

Persistence: faces do **not** persist across `fade_out` + `fade_in`. `fade_out` tears the face down concurrently with the body fade; a subsequent `fade_in` builds a fresh figure with no face. Re-call `attach_face` after `fade_in` if you want the face back.

Rotation: the face tracks head *position* but not head *orientation*. A character rotated via §2.5 keeps an upright face by design (cartoon convention). Out-of-scope for the v0.9.15 MVP; an opt-in `rotate_with_body: true` parameter is on the back burner.

Examples:

```
{"action": "attach_face", "who": "freydoon",
 "image": "bevers_neutral_face.png", "scale": 0.4}

{"action": "attach_face", "who": "freydoon",
 "image": "bevers_worried_face.png", "scale": 0.4}

{"action": "attach_face", "who": "thalia",
 "image": "thalia_smirk.png", "scale": 0.5}
```

Asset path convention: drop face PNGs into `pam/assets/`; reference them by bare filename. The handler resolves to `<pam pkg dir>/assets/<filename>`.

Known Manim limitation (Manim CE 0.20.1, Cairo backend): RGBA PNGs with transparent corners render with a visible alpha-channel artifact instead of compositing cleanly with the scene background. Workaround: produce opaque PNGs with the corners filled in the scene's `BG_COLOR` (`#0a0e1a` by default). See the demo's `make_demo_faces.py` for the pattern.

2.8 `detach_face`

Parallel-safe: yes (returns empty animation list) · *Animation:* none · *Required keys:* `who` · *Version:* v0.9.15

Remove an attached face: stop the updater, remove the image mobject from the scene, restore head dot visibility. No-op if no face is attached.

JSON shape:

```
{"action": "detach_face", "who": "freydoon"}
```

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.

Use cases:

- Consciousness-departs-but-body-remains beat (the avatar leaves Freydoon's body, but Freydoon stays on screen).
- Test teardown — useful in demos where you want to verify the head dot returns to full visibility.

Most authoring doesn't need `detach_face` — `fade_out` auto-removes any attached face.

2.9 `focus`

Parallel-safe: no · *Animation:* timed · *Required keys:* `on` or `dim`

Dim background characters and/or brighten foreground ones to direct audience attention. Animates each character's group opacity. Parsed from Fountain+ `FOCUS:` keys by `fountain2pam.py`.

JSON shape:

```
{"action": "focus", "on": ["bevers"], "dim": "all_others", "rt": 0.4}
```

Parameters:

Key	Type	Required	Default	Description
`on`	`list[str]` or `["all"]`	one of	`[]`	Character keys to keep bright. The literal `["all"]` triggers a reset (equivalent to `focus_reset`).
`dim`	`list[str]` or `"all_others"`	one of	`[]`	Character keys to dim. The string `"all_others"` expands to every cast member not in `on`.
`opacity`	`float`	no	`0.30`	Target opacity for dimmed characters.
`bright`	`float`	no	`1.0`	Target opacity for focused characters.
`rt`	`float`	no	`0.4`	Transition duration.

Behavior:

- Each character's resulting opacity is stored on `cast[name]["opacity"]` so subsequent `focus` calls can diff against it.
- The animation is a single `play()` call combining all character opacity animations concurrently.

Examples:

```
{"action": "focus", "on": ["thalia"], "dim": "all_others"}
{"action": "focus", "on": ["bevers", "athena"], "dim": ["freydoon", "brad"]}
{"action": "focus", "on": ["nona"], "dim": "all_others", "opacity": 0.15, "rt":
↪  0.6}
```

Important interaction with speech bubbles: see §2.13.

2.10 `focus_reset`

Parallel-safe: no · *Animation:* timed · *Required keys:* none

Restore all characters to full opacity in a single concurrent animation.

JSON shape:

```
{"action": "focus_reset", "rt": 0.4}
```

Parameters:

Key	Type	Required	Default	Description
`rt`	`float`	no	`0.4`	Restore duration.

Equivalent to `{"action": "focus", "on": ["all"]}`. `cast[name]["opacity"]` is reset to `1.0` for every cast member.

Important interaction with persistent speech bubbles: see §2.13.

2.11 `clear_bubble`

Parallel-safe: no · *Animation:* timed · *Required keys:* `who` or `prop`

Dismiss a single persistent bubble (one that was created by a `say` or `prop_say` step with `"persist": true` or a non-zero `"duration"`).

JSON shape:

```
{"action": "clear_bubble", "who": "bevers", "rt": 0.25}
```

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	one of	—	Character whose persistent bubble should be cleared.
`prop`	`str`	one of	—	Prop whose persistent bubble should be cleared. An alias `clear_prop_bubble` is also accepted.
`rt`	`float`	no	per-bubble `pam_rt_out`, else `0.25`	Fade-out duration.
`run_time`	`float`	no	—	Synonym for `rt`.

Behavior:

- The named bubble is removed from the internal `_persistent_bubbles` dict and faded out.
- Silent no-op if no bubble is registered under the given key — does not crash a scene with a stale or doubled clear.
- The bubble registry is global to the scene; bubbles persist across subscene boundaries unless explicitly cleared.

No `who`/`prop`: the action silently no-ops rather than crashing.

2.12 `clear_all_bubbles`

Parallel-safe: no · *Animation:* timed · *Required keys:* none

Dismiss every persistent bubble currently on screen.

JSON shape:

```
{"action": "clear_all_bubbles", "rt": 0.25}
```

Parameters:

Key	Type	Required	Default	Description
`rt`	`float`	no	`0.25`	Fade-out duration. Applied uniformly — per-bubble `pam_rt_out` is ignored because the fades run concurrently.
`run_time`	`float`	no	—	Synonym for `rt`.

Use cases:

- Scene or subscene boundaries. Subscene transitions do not auto-clear persistent bubbles, so an explicit `clear_all_bubbles` at the end of a beat is the documented cleanup pattern.
- Before any `focus_reset` if persistent bubbles are on screen — see §2.13.

2.13 The bubble / `focus_reset` ordering gotcha

Documented in v0.9.10. Resolved-status: footgun pending fix.

`focus_reset` (and `focus` with `on: ["all"]`) restores character opacity and z-order in a single `play()` call. When a *persistent* bubble is on screen at the moment of reset, the restored character can be drawn on top of the bubble, partially or fully occluding it.

The mechanism: persistent bubbles are added to the scene via `scene.add(bubble)` before `focus_reset` runs. `focus_reset` animates opacity on figure groups, which Manim treats as updating the existing mobject in-place. If the figure's z-order index is higher than the bubble's, the figure draws on top.

Authoring workaround: clear the bubble before the focus reset.

```
{"action": "clear_bubble", "who": "bevers"},
{"action": "focus_reset"}
```

...or use `clear_all_bubbles` if multiple persistent bubbles are active:

```
{"action": "clear_all_bubbles", "rt": 0.2},
{"action": "focus_reset", "rt": 0.4}
```

Alternative workaround — `say` with `duration`: if the persistent bubble's purpose is "stay visible during the focus shot," set `"duration"` on the `say` step to a value that expires *before* the `focus_reset`. Avoids needing an explicit `clear_bubble`.

Why not `say` with `persist` in walk-and-talk: persistent bubbles anchor to world-space coordinates at the moment they're created. A character that walks under a persistent bubble leaves the bubble behind — it doesn't track the head dot like `attach_face` does. This is a separate limitation; the head-anchored-bubble approach is on the back burner.

Section 3 (action targets) and Section 4 (meta) are delivered in subsequent substeps.

3. Action targets

A character can serve as the target of any of the verbs documented in this section. Coverage is organized functionally:

- **§3.1** — Speech: `say`, `prop_say`.
- **§3.2** — Locomotion (single character): `walk_to`, `run_to`, `rush_to`, `rush_out`, `trot_to`, `walk_to_prop`, `run_to_prop`, `squeeze_through`, `exit_through`, `exit_through_doors`, `jump_up`, `fall_down`, `dodge`.
- **§3.3** — Posture transitions: `sit_down`, `stand_up`.
- **§3.4** — Orientation: `turn`, `face`.
- **§3.5** — Expressive gestures: `wave`, `wave_left`, `wave_right`, `nod`, `shake_head`, `shrug`, `point_at`, `express`, `react`.
- **§3.6** — Prop interaction: `grab`, `pick_up`, `pick_up_phone`, `hang_up`, `place_on`, `put_down`, `peel_from_hand`, `punch_button`, `reach_for`, `search_drawers`, `snap_photo`, `stick_to`, `move_aside`, `carry`.
- **§3.7** — Two-figure interaction: `kiss`, `hold_hands`, `pat`, `pat_head`, `hand_to`, `grab_arm`, `twist_arm_behind`, `release_arm`, `reach_character`.
- **§3.8** — Multi-target: `group_translate`.

Section conventions established in §2 carry forward: each action shows *Parallel-safe*, *Animation*, and *Required keys*. Where an action wraps an internal animation, the **default `rt`** is documented per action and is usually overridable.

Shared conventions

- **`who`** — always a character key from the cast block. When omitted from a step's docs below, it is required.
- **`prop`** — a key from the prop registry. Used when the action targets a prop (`grab`, `walk_to_prop`, etc.).
- **`target`** — either a prop key or a character key, depending on the action. Some actions accept both (e.g. `reach_for`, `face`); others restrict to props only (`stick_to`) or characters only (`reach_character`).
- **`rt`** — animation run time in seconds. Defaults vary per action; documented per action below.
- **`hold`** — pause time in seconds after the main animation, before returning to a rest pose.

The parallel-unsafe set

The following actions print a console warning and run sequentially when nested inside a `parallel` block (from `_CANNOT_PARALLEL` in `actions.py`):

`walk_to`, `run_to`, `sit_down`, `stand_up`, `wave`, `wave_left`, `wave_right`, `carry`, `exit_through`, `exit_through_doors`, `rush_to`, `rush_out`, `squeeze_through`, `jump_up`, `pat`, `search_drawers`, `pick_up_phone`, `hang_up`, `grab`, `punch_button`, `reach_character`, `peel_from_hand`, `group_translate`, `nod`, `shake_head`, `shrug`, `grab_arm`, `twist_arm_behind`, `release_arm`, `rotate`

Every other character-targeting action is parallel-safe in principle (returns animations via `_collect_anims`). The `parallel` + `say` constraint is documented separately in §3.1.

3.1 Speech

3.1.1 `say` *Parallel-safe:* **no** · *Animation:* timed · *Required keys:* `who`, `text`

Display a speech bubble above a character's head. The bubble's box is automatically sized to fit the text, clamped to stay within frame margins, and anchored to the head dot's current position at fade-in.

JSON shape:

```
{"action": "say", "who": "bevers", "text": "Hello, world.",
 "hold": 1.5, "side": "right"}
```

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`text`	`str`	yes	—	Bubble text. Multi-line is supported with `\n`.
`hold`	`float`	no	`1.2`	Seconds to display the bubble before fade-out.
`rt_in`	`float`	no	`0.4`	Fade-in duration.
`rt_out`	`float`	no	`0.3`	Fade-out duration.
`side`	`str`	no	`"right"`	Bubble placement relative to head: `"left"` or `"right"`.
`font_size`	`int`	no	`20`	Text font size.

Key	Type	Required	Default	Description
`style`	`str`	no	`"normal"`	`"normal"` (solid bubble) or `"os"` / `"phone"` (dashed border for off-screen / phone dialogue).
`persist`	`bool`	no	`false`	If true, bubble stays on screen until a `clear_bubble` action. See §2.11.
`duration`	`float`	no	—	If set, bubble stays for `duration` seconds (measured from fade-in start), then auto-clears at the next action.

Behavior:

- The bubble anchors to the head dot at the moment of fade-in. It does **not** track a moving figure — see "parallel + say" below.
- If `persist` or `duration` is set, the bubble is added to the `_persistent_bubbles` registry under `char:{who}`. Subsequent `say` calls on the same character fade the prior bubble out first to avoid stacking.
- Tic system integration (v0.9.14): a `say` step automatically fires `sentence_end` triggers in the character's `tic_profile`. Set `"suppress_tics": true` on the step to skip.

Parallel + say: `say` is not in `_CANNOT_PARALLEL`, but it cannot be nested inside a `parallel` block at the screenplay level — the player's parallel composer treats `say` specially and refuses it. This is tribal knowledge worth a formal docs callout (back-burner).

Speech bubble anchoring caveat: the bubble is positioned in world space at the moment of creation. A character who walks while a `persist` bubble is up will leave the bubble behind. For walk-and-talk, interleave `walk_to` and short non-persist `say` steps rather than running a long persistent bubble; the head-anchored-bubble design for walk-and-talk is on the back burner.

O.S. / phone bubbles:

```
{"action": "say", "who": "freydoon",
 "text": "(over the phone) I'm on my way.", "style": "os"}
```

The dashed border reads as off-screen / disembodied dialogue. Use sparingly — readable but visually distinct.

Bubble for a horizontal character: the named poses `lying_flat_right` and `lying_flat_left`

(v0.9.14) are required for correct bubble anchoring when a character is lying down. An ad-hoc `rotate` to horizontal will mis-anchor the bubble.

3.1.2 `prop_say` *Parallel-safe:* no · *Animation:* timed · *Required keys:* `prop`, `text`

Display a speech bubble next to a prop. Used for prop-characters (`GovernorGraph`, `DogGraph`, news anchors on TV monitors) that have no head joint for `say` to anchor to.

JSON shape:

```
{"action": "prop_say", "prop": "governor",
 "text": "Approved.", "hold": 1.2}
```

Parameters:

Key	Type	Required	Default	Description
`prop`	`str`	yes	—	Prop registry key. Path C dogs (`who: chekov` or `prop: chekov`) resolve via either key.
`text`	`str`	yes	—	Bubble text.
`hold`	`float`	no	`1.2`	Seconds to display.
`style`	`str`	no	`"normal"`	`"normal"` or `"os"` (dashed border).
`y_offset`	`float`	no	`0.0`	*(v0.9.18)* Vertical nudge applied on top of the geometry-aware default. Currently consumed only by `GovernorGraph`; `DogGraph` and the generic-prop branch ignore it. Positive lifts the bubble, negative lowers it.
`persist`, `duration`, `rt_in`, `rt_out`, `font_size`	—	—	—	Same as `say`.

Behavior:

- For `GovernorGraph` and `DogGraph`, `prop_say` uses the figure's own bubble layout (these classes implement their own bubble geometry).
- *(v0.9.18)* `GovernorGraph` bubble placement is now geometry-aware. The bubble's bottom edge sits `0.10` units above the dodecahedron's top (`self._y + self._radius`), regardless of prop size, so the bubble grows away from the prop instead of overlapping it. The tail tip lands inside the top portion of the dodecahedron, which is the comic-book convention for "bubble points at speaker." Previously the bubble center was hardcoded at `self._y + 0.3`, which for a standard-sized Governor put the tail tip below the prop's centre. Use `y_offset` to nudge the result up or down if a specific scene still reads off.
- For generic props (tv_monitor, cellphone, etc.), the bubble is built manually with placement near the prop's top surface. Bubble color resolves in priority order: explicit `bubble_color` / `text_color` on the step -> parent character's palette (if the prop has `pam_follows`) -> prop's `screen_color` -> default tan.
- Word wrapping in `prop_say` is currently implemented for `DogGraph` and `GovernorGraph` only — generic props receive un-wrapped text.

Examples:

```
{"action": "prop_say", "prop": "chekov",  "text": "Woof!"}
{"action": "prop_say", "prop": "athena_laptop",
 "text": "WiFi scan complete.", "hold": 1.8}
{"action": "prop_say", "prop": "news_anchor", "text": "Breaking news.",
 "style": "os"}
```

Example — Governor with bubble nudged up (v0.9.18). A camera angle that places the Governor near the bottom of the frame sometimes leaves the default bubble too close to the lower edge. `y_offset` lifts it without changing the prop's position:

```
{"action": "prop_say", "prop": "governor",
 "text": "Decision rendered.",
 "y_offset": 0.3}
```

3.2 Locomotion (single character)

3.2.1 `walk_to` *Parallel-safe:* no · *Animation:* timed · *Required keys:* `who`, `x`

Walk the figure to an x position. Drives the four-keyframe walk cycle for each step's worth of horizontal distance; pose must be a side-view at start (or the character first morphs to `standing_side` automatically — confirm against `figure.py`).

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`x`	`float`	yes	—	Target x in world coordinates. Negative for left, positive for right.

Behavior:

- Walk speed is derived from the figure's internal stride length; `walk_to` figures out how many steps to take based on the x delta.
- Attached props update their positions via `_drag_attached_props` so a character carrying a laptop keeps the laptop with them.
- `walk_to` updates `fig.offset` to the new x but does **not** write back to `cast[name]["offset"]`. After a `fade_out` + `fade_in`, the character returns to the original cast-spec offset.

Example:

```
{"action": "walk_to", "who": "bevers", "x": -2.5}
```

3.2.2 `run_to` *Parallel-safe:* no · *Animation:* timed · *Required keys:* `who`, `x`

Same as `walk_to`, but uses the run cycle (faster, more dynamic posture).

```
{"action": "run_to", "who": "freydoon", "x": 4.0}
```

3.2.3 `rush_to` *Parallel-safe:* no · *Animation:* timed · *Required keys:* `who`, `x`

Fast walk with a forward-lean pose. Visually reads as "in a hurry" without the full sprint of `run_to`.

```
{"action": "rush_to", "who": "thalia", "x": 3.0}
```

3.2.4 `rush_out` *Parallel-safe:* no · *Animation:* timed · *Required keys:* `who`, `x`

Alias of `rush_to` — identical mechanics, name only differs for readability when the destination is off-screen.

```
{"action": "rush_out", "who": "sidel", "x": 7.5}
```

3.2.5 `trot_to` *Parallel-safe:* yes (handled specially in `parallel`) · *Animation:* timed · *Required keys:* `who` or `prop`, `x` · *Applies to:* `figure_type: "dog"`

Dog-only locomotion. Path C dual-registration (v0.9.14) means a dog can be addressed via `who: chekov` or `prop: chekov`; `trot_to` resolves to the same `DogGraph` either way.

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	one of	—	Dog's character key.
`prop`	`str`	one of	—	Dog's prop key (same name under dual registration).

Key	Type	Required	Default	Description
`x`	`float`	yes	—	Target x in world coordinates.
`stride`	`float`	no	`0.14`	Step length per trot keyframe. Shorter for slower, more delicate trot; longer for faster gait.

Example:

```
{"action": "trot_to", "who": "chekov", "x": 1.5, "stride": 0.18}
```

Note: there is no equivalent `walk_to_dog` or `run_to_dog` — dogs use a single locomotion verb (`trot_to`) with stride control. An author needing a slow dog should reduce `stride`; for a fast dog, increase it.

3.2.6 `walk_to_prop` *Parallel-safe:* yes · *Animation:* timed · *Required keys:* `who`, `prop`

Walk to the x position of a named prop. Convenience wrapper — equivalent to `walk_to` with `x` computed from the prop's `pam_x`.

```
{"action": "walk_to_prop", "who": "sidel", "prop": "desk"}
```

Note: the character ends up at the prop's x — not "next to" it. For an offset stand-near-prop position, follow with a `walk_to` to a slightly different x.

3.2.7 `run_to_prop` *Parallel-safe:* yes · *Animation:* timed · *Required keys:* `who`, `prop`

Same as `walk_to_prop` but using the run cycle.

```
{"action": "run_to_prop", "who": "thalia", "prop": "exit_door"}
```

3.2.8 `squeeze_through` *Parallel-safe:* no · *Animation:* timed · *Required keys:* `who`, `x`

Narrow-stance walk through a tight space (doorway, crowd gap). Uses the `squeeze_walk_a` / `squeeze_walk_b` poses with arms held close to the body.

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`x`	`float`	yes	—	Target x.

Key	Type	Required	Default	Description
`rt`	`float`	no	per cycle	Per-keyframe run time. Tune for tightness pacing.

```
{"action": "squeeze_through", "who": "freydoon", "x": 2.5}
```

3.2.9 `exit_through` *Parallel-safe:* no · *Animation:* timed · *Required keys:* `who`, `prop`

Walk to a door (or other exit prop), then fade out. The character disappears at the door's x position.

```
{"action": "exit_through", "who": "bevers", "prop": "exit_door"}
```

3.2.10 `exit_through_doors` *Parallel-safe:* no · *Animation:* timed · *Required keys:* `who`, `prop`

Walk to a door (typically an elevator), pause for the door's open/close animation, then disappear off-screen. Difference from `exit_through`: this version respects elevator-style props with explicit door state.

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`prop`	`str`	yes	—	Elevator or door prop with open/close animation.
`rt`	`float`	no	per stage	Per-stage run time.

```
{"action": "exit_through_doors", "who": "thalia", "prop": "elevator_1"}
```

3.2.11 `jump_up` *Parallel-safe:* no · *Animation:* timed · *Required keys:* `who`

Eager reactive jump: crouch -> peak -> land. Used for affirmative reactions ("got it!", "yes!").

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`height`	`float`	no	`0.4`	Peak height in world units.

Key	Type	Required	Default	Description
`rt`	`float`	no	`0.4`	Total cycle run time (crouch + peak + land).

```
{"action": "jump_up", "who": "bart", "height": 0.6}
```

3.2.12 `fall_down` *Parallel-safe:* yes · *Animation:* timed · *Required keys:* `who`

Animate a character falling from standing to on-hands-and-knees.

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`style`	`str`	no	`"stumble"`	Variant: `"stumble"` (catch-and-go-down), `"fall_catch"` (catch-mid-fall, recover), or a direct pose name.
`rt`	`float`	no	per stage	Per-stage run time.

```
{"action": "fall_down", "who": "brad", "style": "stumble"}
```

3.2.13 `dodge` *Parallel-safe:* yes · *Animation:* timed · *Required keys:* `who`

Lateral sidestep away from another character's path. Short, sharp displacement followed by return to upright.

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`direction`	`str`	no	`"right"`	`"left"` or `"right"`.
`distance`	`float`	no	`0.4`	Lateral displacement in world units.
`rt`	`float`	no	`0.3`	Sidestep duration.

```
{"action": "dodge", "who": "thalia", "direction": "left", "distance": 0.5}
```

3.3 Posture transitions

3.3.1 `sit_down` *Parallel-safe:* no · *Animation:* timed · *Required keys:* `who`

Lower the figure from standing to seated. Drives a three-keyframe cycle through `sitting_mid` to `sitting_down`. The character must be near a chair prop for the seated position to read correctly — `sit_down` does not change x position.

```
{"action": "sit_down", "who": "freydoon"}
```

Use pattern: walk to chair, then sit:

```
{"action": "walk_to_prop", "who": "freydoon", "prop": "chair_1"},
{"action": "sit_down", "who": "freydoon"}
```

3.3.2 `stand_up` *Parallel-safe:* no · *Animation:* timed · *Required keys:* `who`

Inverse of `sit_down`.

```
{"action": "stand_up", "who": "freydoon"}
```

3.4 Orientation

3.4.1 `turn` *Parallel-safe:* yes · *Animation:* timed · *Required keys:* `who`

Turn the figure to a side or named pose. Convenience wrapper around `morph` for orientation-only changes.

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`pose`	`str`	no	`"standing_side"`	Target pose. Common values: `"standing_front"`, `"standing_side"`.

```
{"action": "turn", "who": "thalia", "pose": "standing_side"}
```

3.4.2 `face` *Parallel-safe:* yes · *Animation:* timed · *Required keys:* `who`, `target`

Turn the figure to face a named prop or character. Computes the appropriate side-view orientation based on the target's x position relative to the character's.

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`target`	`str`	yes	—	Prop or character key. If target's x > character's x, face right; else face left.

```
{"action": "face", "who": "thalia", "target": "bevers"}
{"action": "face", "who": "athena", "target": "desk"}
```

3.5 Expressive gestures

3.5.1 `wave` *Parallel-safe:* no · *Animation:* timed · *Required keys:* `who`

Raise one arm and wag it.

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`direction`	`str`	no	`"right"`	`"right"` or `"left"` — which arm waves. Synonym: `"hand"`.
`hand`	`str`	no	`"right"`	Synonym for `direction` (matches `figure.py`'s `wave(hand=...)` parameter name).
`cycles`	`int`	no	`2`	Number of wag oscillations.
`rt_lift`	`float`	no	`0.4`	Arm raise/lower run time.
`rt_wag`	`float`	no	`0.24`	Per-wag-keyframe run time.

```
{"action": "wave", "who": "bevers", "direction": "left", "cycles": 3}
```

3.5.2 `wave_left` / `wave_right` *Parallel-safe:* no · *Animation:* timed · *Required keys:* `who`

Convenience aliases. `wave_left` is equivalent to `wave` with `direction: "left"`; `wave_right` is

equivalent to `wave` with `direction: "right"`.

```
{"action": "wave_left",  "who": "short_emt"},
{"action": "wave_right", "who": "tall_emt"}
```

3.5.3 `nod` *Parallel-safe:* no · *Animation:* timed · *Required keys:* `who`

Nod the head up-and-down (affirmative).

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`cycles`	`int`	no	`2`	Number of nod oscillations.
`amp`	`float`	no	`0.08`	Vertical head displacement amplitude.
`rt_per_step`	`float`	no	`0.18`	Per-keyframe run time.

```
{"action": "nod", "who": "freydoon", "cycles": 3}
```

3.5.4 `shake_head` *Parallel-safe:* no · *Animation:* timed · *Required keys:* `who`

Shake the head side-to-side (negation). Same parameters as `nod` but the displacement is horizontal.

```
{"action": "shake_head", "who": "thalia"}
```

3.5.5 `shrug` *Parallel-safe:* no · *Animation:* timed · *Required keys:* `who`

Raise both shoulders (arms follow), hold briefly, lower.

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`amp`	`float`	no	`0.15`	Shoulder rise amplitude.
`rt`	`float`	no	`0.4`	Rise/lower duration per direction.
`hold`	`float`	no	`0.3`	Hold duration at peak.

```
{"action": "shrug", "who": "bevers", "amp": 0.2, "hold": 0.5}
```

3.5.6 `point_at` *Parallel-safe:* yes · *Animation:* timed · *Required keys:* `who`, `target`

Extend arm and point at a named prop or character.

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`target`	`str`	yes	—	Prop or character key.
`hold`	`float`	no	`0.6`	Hold time at full extension before retracting.

```
{"action": "point_at", "who": "nona", "target": "factor"}
{"action": "point_at", "who": "athena", "target": "desk"}
```

3.5.7 `express`

Parallel-safe: yes · *Animation:* timed · *Required keys:* `who`, `expression`

Flash a reaction glyph above the character's head. The glyph is a small Unicode/text symbol drawn just above the head dot.

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`expression`	`str`	yes	—	Key from `EXPRESSION_GLYPHS` in `pam/poses.py`. Current registry: `"smirk"`, `"roll_eyes"`.
`hold`	`float`	no	per-glyph default	Display duration.
`rt_in`	`float`	no	`0.2`	Fade-in.
`rt_out`	`float`	no	`0.2`	Fade-out.
`color`	`str` (hex)	no	per-glyph default	Override the glyph color.

```
{"action": "express", "who": "bevers", "expression": "smirk"}
{"action": "express", "who": "thalia", "expression": "roll_eyes", "hold": 1.5}
```

Note: `EXPRESSION_GLYPHS` is small (2 entries currently). Each entry specifies its own default `glyph`, `dx`, `dy`, `font_size`, `hold`, and `color`. The action's `hold` and `color` parameters override the glyph defaults.

3.5.8 `react`

Parallel-safe: yes · *Animation:* timed · *Required keys:* `who`

Fire any tics in `fig.tic_profile` whose triggers include `"react"`. No-op for figures with no tic

profile or no react-triggered fragments.

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`suppress_tics`	`bool`	no	`false`	If `true`, the action no-ops (useful for marking a beat without actually firing the tic).

```
{"action": "react", "who": "bevers"}
```

Tic fragment registry: see `pam/tics.py` and §1.12 for the full table. Current fragments: `hand_twitch_r`, `hand_twitch_l`, `foot_tap_r`, `foot_tap_l` (biped, `human` and `alien`), and `tail_wag` (quadruped, `dog`). Body-type compatibility is checked at play time — a `tail_wag` declared on a biped silently no-ops, as does a `hand_twitch_*` or `foot_tap_*` declared on a dog.

Trigger interaction: `say` and `prop_say` also fire tics under the `"sentence_end"` trigger automatically. A tic_profile that triggers on both `react` and `sentence_end` fires on every `say` step *and* every explicit `react`.

3.6 Prop interaction

The following actions all take a character as `who` and a prop (or surface) as the direct object. Common patterns:

- `target` — the prop being approached or operated on (most actions).
- `prop` — the prop being carried, picked up, or set down (state-changing actions).
- `arm` — which arm to use: `"right"` (default) or `"left"`.

3.6.1 `reach_for` *Parallel-safe:* yes · *Animation:* timed · *Required keys:* `who`, `target`

Extend one arm toward a target prop, hold briefly, retract. Used as a non-committal gesture — character indicates the prop without acting on it.

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`target`	`str`	yes	—	Prop key.
`arm`	`str`	no	`"right"`	`"right"` or `"left"`.
`hold`	`float`	no	`0.4`	Hold time at full extension.
`rt`	`float`	no	`0.4`	Extend/retract per direction.

```
{"action": "reach_for", "who": "freydoon", "target": "doorknob"}
```

3.6.2 `grab` *Parallel-safe:* no · *Animation:* timed · *Required keys:* `who`, `prop` (or `target`)

Decisive `reach_for` + retract with the prop now held. Reads as more urgent than `pick_up` — faster, no kneel. After `grab`, the character is holding the prop in carry pose.

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`prop`	`str`	one of	—	Prop to grab.
`target`	`str`	one of	—	Synonym for `prop`.
`arm`	`str`	no	`"right"`	`"right"` or `"left"`.
`rt`	`float`	no	`0.3`	Grab speed.

```
{"action": "grab", "who": "chava", "prop": "folder"}
```

3.6.3 `pick_up` *Parallel-safe:* yes · *Animation:* timed · *Required keys:* `who`, `prop`

Reach down, pick up a prop, return to carry-hold pose. Slower and more deliberate than `grab` — used for ground-level or table-level objects.

```
{"action": "pick_up", "who": "sidel", "prop": "briefcase"}
```

3.6.4 `put_down` *Parallel-safe:* yes · *Animation:* timed · *Required keys:* `who`, `prop`

Lower a held prop onto a target surface or the floor.

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`prop`	`str`	yes	—	Currently-held prop.
`on`	`str`	no	—	Target surface prop (desk, table). Omit to put down on the floor.

```
{"action": "put_down", "who": "sidel", "prop": "briefcase", "on": "desk"}
{"action": "put_down", "who": "freydoon", "prop": "phone"}
```

3.6.5 `place_on` *Parallel-safe:* yes · *Animation:* timed · *Required keys:* `who`, `prop`, `target`

Set a carried prop onto a target prop's surface. Difference from `put_down`: `place_on` always requires a target surface and animates a more deliberate placement (slightly higher arc).

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`prop`	`str`	yes	—	Currently-held prop.
`target`	`str`	yes	—	Target surface prop.
`rt`	`float`	no	`0.5`	Placement duration.

```
{"action": "place_on", "who": "athena", "prop": "laptop", "target": "desk"}
```

3.6.6 `peel_from_hand`

Parallel-safe: no · *Animation:* timed · *Required keys:* `who`, `prop`

Peel a tiny prop (e.g. an audio/video bug) off the character's palm. The prop detaches from the carry pose and remains at the peeled-off position.

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`prop`	`str`	yes	—	Held prop to peel off.
`arm`	`str`	no	`"right"`	`"right"` or `"left"`.
`rt`	`float`	no	`0.4`	Peel duration.

```
{"action": "peel_from_hand", "who": "agent", "prop": "av_bug"}
```

3.6.7 `punch_button`

Parallel-safe: no · *Animation:* timed · *Required keys:* `who`, `target`

Sharp jab at a prop (button panel, elevator call, doorbell) then retract. Faster and more aggressive than `reach_for`.

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`target`	`str`	yes	—	Target prop.
`arm`	`str`	no	`"right"`	`"right"` or `"left"`.
`offset_x`	`float`	no	`0.0`	Fine-tune jab landing point relative to target's center.
`offset_y`	`float`	no	`0.0`	As above, vertical.

Key	Type	Required	Default	Description
`hold`	`float`	no	`0.1`	Brief hold at impact.
`rt`	`float`	no	`0.2`	Jab and retract duration.

```
{"action": "punch_button", "who": "athena", "target": "call_button"}
{"action": "punch_button", "who": "freydoon", "target": "elevator_panel",
 "offset_x": 0.1, "offset_y": -0.2}
```

3.6.8 `search_drawers` *Parallel-safe:* no · *Animation:* timed · *Required keys:* `who`, `target`

Rummaging macro: repeated reach-for with downward pose variants. Reads as opening and searching drawers in a desk.

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`target`	`str`	yes	—	Desk or chest prop.
`cycles`	`int`	no	`3`	Number of search cycles.
`rt`	`float`	no	`0.4`	Per-cycle run time.

```
{"action": "search_drawers", "who": "thalia", "target": "desk", "cycles": 4}
```

3.6.9 `snap_photo` *Parallel-safe:* yes · *Animation:* timed · *Required keys:* `who`, `target`

Point a held phone at a target and flash. Requires the character to be holding a phone (via `pick_up` or `grab`).

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`target`	`str`	yes	—	Subject of the photo (prop or character).
`flash_rt`	`float`	no	`0.15`	Flash overlay duration.
`hold`	`float`	no	`0.3`	Total hold time at the photo-taking pose.

```
{"action": "snap_photo", "who": "thalia", "target": "bevers"}
```

3.6.10 `stick_to` *Parallel-safe:* yes · *Animation:* timed · *Required keys:* `who`, `prop`, `target`

Attach a small prop (e.g. audio/video bug) to a target prop surface. After the action, the small prop is parented to the target and tracks with it.

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`prop`	`str`	yes	—	Held prop to stick.
`target`	`str`	yes	—	Target prop (must declare a `shade`, `surface`, or similar attachment point).
`rt`	`float`	no	`0.4`	Stick duration.

```
{"action": "stick_to", "who": "agent", "prop": "av_bug",
 "target": "desk_lamp"}
```

3.6.11 `move_aside` *Parallel-safe:* yes · *Animation:* timed · *Required keys:* `who`, `prop`

Push a prop laterally without picking it up.

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`prop`	`str`	yes	—	Prop to push.
`direction`	`str`	no	`"right"`	`"left"` or `"right"`.
`distance`	`float`	no	`0.5`	Lateral displacement.
`rt`	`float`	no	`0.4`	Push duration.

```
{"action": "move_aside", "who": "freydoon", "prop": "chair_2",
 "direction": "left", "distance": 0.8}
```

3.6.12 `pick_up_phone` *Parallel-safe:* no · *Animation:* timed · *Required keys:* `who`, `prop`

Specialized variant of `pick_up` for landline desk phones. Raises the handset to ear height in a specific carry pose (not the generic `carry_hold`).

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`prop`	`str`	yes	—	Phone prop (must be `landline_desk` style).
`arm`	`str`	no	`"right"`	`"right"` or `"left"`.
`rt`	`float`	no	`0.4`	Raise duration.

```
{"action": "pick_up_phone", "who": "sidel", "prop": "office_phone"}
```

3.6.13 `hang_up`

Parallel-safe: no · *Animation:* timed · *Required keys:* `who`, `prop`

Return a held phone handset to its cradle. Inverse of `pick_up_phone`.

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`prop`	`str`	yes	—	Held phone handset. Synonym: `target`.
`target`	`str`	optional	—	Synonym for `prop`.
`rt`	`float`	no	`0.4`	Lower duration.

```
{"action": "hang_up", "who": "sidel", "prop": "office_phone"}
```

3.6.14 `carry`

Parallel-safe: no · *Animation:* timed · *Required keys:* `who`, `prop`, `x`

Walk while carrying a named prop, with optional head companions (other props that ride along on the head).

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`prop`	`str`	yes	—	Prop being carried (must be already held — call `pick_up` first).
`x`	`float`	yes	—	Walk destination.

Key	Type	Required	Default	Description
`companions`	`list[str]`	no	`[]`	Other prop keys that follow the head joint while walking.
`torso_companions`	`list[str]`	no	`[]`	Other prop keys that follow the torso joint.
`size`	`float`	no	`1.0`	Visual scale multiplier applied during the carry walk.
`color`	`str` (hex)	no	—	Optional uniform/torso color recolor during the carry.

```
{"action": "carry", "who": "chava", "prop": "briefcase", "x": 2.5,
 "torso_companions": ["folder"]}
```

3.7 Two-figure interaction

The following actions take two character keys: `who` (the acting character) and `target` (the other character). Both must be in the cast and faded in. Most actions read the target via `cast[target]["fig"]` and skip with a console warning if the target is not live.

3.7.1 `pat` *Parallel-safe:* no · *Animation:* timed · *Required keys:* `who`

Short repeated tapping gesture toward a prop or body area. Single-character action — taps the air at a target position.

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key.
`cycles`	`int`	no	`3`	Number of pat oscillations.
`rt`	`float`	no	per cycle	Per-cycle run time.

```
{"action": "pat", "who": "thalia", "cycles": 3}
```

3.7.2 `pat_head` *Parallel-safe:* yes · *Animation:* timed · *Required keys:* `who`, `target`

Pat another character or dog on the head. The acting figure extends an arm to the target's head position and does a gentle up-down oscillation (like `pat`, but targeting another character's head joint).

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Acting character.
`target`	`str`	yes	—	Character or dog prop whose head receives the pat.
`cycles`	`int`	no	`3`	Number of pat oscillations.
`rt`	`float`	no	`0.2`	Per-oscillation run time.
`arm`	`str`	no	`"auto"`	`"r"`, `"l"`, or `"auto"`. `"auto"` picks the arm on the side closest to the target.

```
{"action": "pat_head", "who": "thalia", "target": "chekov"}
{"action": "pat_head", "who": "bevers", "target": "chekov", "cycles": 5, "arm":
↪  "l"}
```

3.7.3 `hand_to` *Parallel-safe:* yes · *Animation:* timed · *Required keys:* `who`, `target`, `prop`

Hand a prop to another character: the giver extends an arm with the prop toward the target, the target reaches to receive, and the prop transfers ownership. After the action, the prop is parented to the target.

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Giver.
`target`	`str`	yes	—	Receiver.
`prop`	`str`	yes	—	Prop being transferred.
`rt`	`float`	no	`0.35`	Morph speed for both characters' arms.
`hold`	`float`	no	`0.3`	Dwell at the hand-off point before transfer.

```
{"action": "hand_to", "who": "freydoon", "prop": "phone01",
 "target": "chava", "rt": 0.35}
```

3.7.4 `kiss` *Parallel-safe:* yes · *Animation:* timed · *Required keys:* `who`, `target`

Two characters lean forward, touch heads, and show a <3 glyph with a pink dashed `(kiss)` bubble. Both characters morph to a lean-forward pose (heads tilted toward each other), hold, then morph back.

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Acting character.
`target`	`str`	yes	—	Recipient.
`hold`	`float`	no	`1.5`	Kiss duration in seconds.
`rt`	`float`	no	`0.4`	Lean-in / lean-back morph speed.
`glyph`	`str`	no	`"<3"`	The glyph character displayed during the kiss.
`color`	`str` (hex)	no	`"#ff69b4"`	Glyph and bubble color.
`bubble`	`bool`	no	`true`	Show the `(kiss)` bubble. Set `false` to suppress for a silent kiss.

```
{"action": "kiss", "who": "nona", "target": "factor", "hold": 2.0}
{"action": "kiss", "who": "freydoon", "target": "athena",
 "glyph": "*", "color": "#ffdd55", "bubble": false}
```

3.7.5 `hold_hands` *Parallel-safe:* yes · *Animation:* timed · *Required keys:* `who`, `target`

Two adjacent characters extend their inner arms until wrists meet at a midpoint. The action figures out which character is left vs. right of the other, then morphs each figure's inner arm (the left character's right arm; the right character's left arm) to a midpoint handhold pose.

After the `hold` duration, both characters return to their pre-handhold rest poses. Not a persistent state — for a sustained handhold, run a longer `hold` or repeat the action.

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Acting character.
`target`	`str`	yes	—	Other character. Order doesn't matter; the action picks left/right based on x positions.

Key	Type	Required	Default	Description
`hold`	`float`	no	`2.0`	Hold duration before release.
`rt`	`float`	no	`0.35`	Arm-extend morph speed. Release uses `rt * 0.8`.

```
{"action": "hold_hands", "who": "thalia", "target": "bevers", "hold": 2.0}
```

3.7.6 `grab_arm` *Parallel-safe:* no · *Animation:* timed · *Required keys:* `who`, `target`

Grab another character's arm from behind: the actor reaches forward and seizes the target's wrist, locking that arm in place.

The action does NOT auto-return either character to a rest pose — the constraint persists until `release_arm` is called. The target's seized arm is recorded on `target_fig._restrained_arm` (`"r"` or `"l"`), which `twist_arm_behind` reads to know which arm to fold and which other actions can check to skip the constrained character.

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Acting character.
`target`	`str`	yes	—	Character whose arm is grabbed.
`arm`	`str`	no	`"auto"`	Which of the *target's* arms to seize: `"r"`, `"l"`, or `"auto"` (picks the arm on the side closest to the actor).
`rt`	`float`	no	`0.3`	Morph speed for the grab.

```
{"action": "grab_arm", "who": "chava", "target": "brad", "arm": "r"}
{"action": "grab_arm", "who": "chava", "target": "brad"}
```

3.7.7 `twist_arm_behind` *Parallel-safe:* no · *Animation:* timed · *Required keys:* `who`, `target`

Escalate a `grab_arm` into an arm-lock: fold the target's seized arm behind their back and tilt their torso forward under the pressure. Pairs with the `restrained_side` / `grip_behind` poses (see §1.7).

Must be called *after* `grab_arm` on the same target. Reads `target_fig._restrained_arm`

to know which arm to fold; if no arm is currently grabbed, the action prints a warning and skips.

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Acting character.
`target`	`str`	yes	—	Character being restrained (must already be grabbed).
`tilt`	`float`	no	`0.12`	How far the target's torso tilts forward, in unscaled pose units. Higher values read as more painful / more aggressive lock.
`rt`	`float`	no	`0.35`	Morph speed.

```
{"action": "twist_arm_behind", "who": "chava", "target": "brad", "tilt": 0.15}
```

3.7.8 `release_arm`

Parallel-safe: no · *Animation:* timed · *Required keys:* `who`, `target`

Release a grabbed arm: both characters return to their pre-grab rest poses, and the `target_fig._restrained_arm` constraint state is cleared.

Target must currently be restrained. If `release_arm` is called on a target with no active grab, the action prints a warning and skips. The full restraint sequence is therefore: `grab_arm` -> (optional `twist_arm_behind`) -> `release_arm`.

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Actor who originally grabbed.
`target`	`str`	yes	—	Character being released.
`rt`	`float`	no	`0.3`	Morph speed for the release.

```
{"action": "release_arm", "who": "chava", "target": "brad"}
```

3.7.9 `reach_character`

Parallel-safe: no · *Animation:* timed · *Required keys:* `who`, `target`

Jab one arm toward a body zone on another cast member, then retract. Similar mechanics to `punch_button`, but with a character as the target.

Parameters:

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Acting character.
`target`	`str`	yes	—	Target character.
`zone`	`str`	no	`"torso"`	Body zone of target: `"torso"`, `"head"`, `"larm"`, `"rarm"`, etc. Mapped to joint name on the target.
`arm`	`str`	no	`"right"`	Acting character's arm.
`offset_x`	`float`	no	`0.0`	Fine-tune landing point.
`offset_y`	`float`	no	`0.0`	As above.
`hold`	`float`	no	`0.2`	Hold at full extension.
`rt`	`float`	no	`0.3`	Reach and retract duration.

```
{"action": "reach_character", "who": "freydoon", "target": "brad",
 "zone": "torso", "arm": "right"}
```

Note: the cast registry lookup for `target` goes through `cast[target_name]["fig"]` (the live `HumanGraph` object), not the cast spec dict. If `target` isn't in cast or hasn't been faded in, the action skips with a warning.

3.8 Multi-target

3.8.1 `group_translate` *Parallel-safe:* no · *Animation:* timed · *Required keys:* `who`

Move multiple characters (and optionally props) simultaneously along a vector. Used for camera-like "treadmill" effects in walk-and-talk: the world translates underneath the characters, suggesting continuous motion without the characters changing scene position.

Parameters:

Key	Type	Required	Default	Description
`who`	`list[str]` or `str`	yes	—	Character keys to translate. Either a list of names or `"all"`.
`dx`	`float`	no	`0.0`	Horizontal translation.
`dy`	`float`	no	`0.0`	Vertical translation.

Key	Type	Required	Default	Description
`props`	`list[str]`	no	`[]`	Prop keys to also translate.
`rt`	`float`	no	`1.0`	Translation duration.

```
{"action": "group_translate",
 "who": ["bevers", "thalia"],
 "props": ["bench_1", "bench_2"],
 "dx": -2.0, "rt": 1.5}
```

Parallel-safety: currently marked unsafe. A clean fix is on the back-burner list (walk-and-talk wants this inside `parallel` for the talk-during-translate pattern). Workaround: split the talk into stationary beats interleaved with translation beats.

Section 4 (meta — name resolution, the ***`hidden`*** *flag, z-order, color authority chain, registry behavior) is the final substep.*

4. Meta

Cross-cutting concerns that don't fit cleanly inside any single field or action: how names resolve to figures and props, where the `hidden` flag does and doesn't apply, how z-order works, the color authority chain across the registry / scene / annotation layers, registry maintenance, and a consolidated gotchas appendix.

4.1 Name resolution

A step's target is identified by either `who` or `prop`. Both are string keys into separate registries — *characters* in the cast registry, *props* in the prop registry — but for some character types the same name resolves through both.

The cast registry is the dict `cast[name]` keyed on character names from the `cast` action block. Each entry holds the canonical spec plus a `"fig"` field that points to the live figure instance after fade-in:

```
cast["bevers"] = {
    "figure_type": "alien",
    "build": "alien",
    "fig": <AlienGraph instance>,  # populated by fade_in
    # ... all the creation-time fields from §1
}
```

`cast[name]["fig"]` is `None` between fade-out and the next fade-in for the same character.

The prop registry is the `PropRegistry` instance (`props` in handler signatures). Each entry is the prop's Manim `VGroup` decorated with `pam_*` attributes (`pam_name`, `pam_type`, `pam_x`, `pam_y`, etc.).

Path C dual registration DogGraph instances live in both registries simultaneously (v0.9.14+). When a dog is declared in a `cast` block, `pam_player`'s `fade_in` branch:

1. Builds the `DogGraph` and assigns it to `cast[name]["fig"]`.
2. Wraps it as a prop: binds `dog.group` to a local, stamps `pam_dog = fig` plus the other `pam_*` attributes on the group, and calls `props.add(name, dog_group)`.
3. Skips step 2 if a prop with that name already exists (e.g. `spawn_prop` ran first).

The result: `{"who": "chekov"}` and `{"prop": "chekov"}` both refer to the same `DogGraph`. Locomotion (`trot_to`) and speech (`prop_say`) resolve correctly from either side.

Why the `dog.group` local-bind matters: `DogGraph.group` is a `@property` that returns a fresh `VGroup` on every access. Setting attributes on `dog.group` directly would stamp them onto a throwaway object and leave the registered group bare. The pattern is:

```
dog_group = fig.group      # bind once
dog_group.pam_name = name
dog_group.pam_dog  = fig
# ...
props.add(name, dog_group)
```

Same pattern is used in both `spawn_prop`'s dog branch and `fade_in`'s dual-registration branch — symmetric registry maintenance.

The `_resolve_speaker` canonical helper `pam.actions._resolve_speaker(step, cast=cast, props=props)` is the canonical name-to-figure lookup. It reads `step.get("who")` first, then falls back to `step.get("prop")`, and returns a `ResolvedSpeaker` namedtuple:

```
ResolvedSpeaker(
    name: str | None,
    fig:  HumanGraph | AlienGraph | DogGraph | GovernorGraph | None,
    prop: VGroup | None,
)
```

Both `fig` and `prop` may be populated simultaneously for dual-registered entries. Callers select whichever they need.

The legacy fallthrough: if the cast lookup misses but a prop with `pam_dog` exists, `_resolve_speaker` surfaces the `DogGraph` as `fig`. This handles `spawn_prop`'d dogs cleanly even before they're cast-registered. The fallthrough preserves backward compatibility with pre-Path-C screenplays.

Resolution failure: if neither `who` nor `prop` resolves, the action handler typically prints a console warning and returns `None` (silent no-op). This is the documented behavior for safety — a malformed step should not crash a render.

Resolution priority (action-specific) Some actions accept both `who` and `prop` and have explicit priority rules:

- **`rotate`** — if both are present, `prop` wins. Documented in §2.5.
- **`grab`** — accepts `prop` or `target` as synonyms for the held-item key. `prop` wins if both are present.
- **`hang_up`** — accepts `prop` or `target` as synonyms for the phone-handset key.

These rules are per-action and not centralized; check the action's docstring in §3 for the specific resolution path.

4.2 The `hidden` flag

The `hidden` flag is a prop concept, not a character concept.

For props, `"hidden": true` in a `props` (or `scene_props`) action declares the prop but doesn't add it to the scene visibly. The prop is registered (so `parent` / `attach` references can resolve it) but rendered at opacity 0. A subsequent `spawn_prop` action reveals it via a fade-in.

```
{
  "action": "props",
  "items": {
    "athena_laptop": {"type": "laptop", "x": 4.5, "y": 0.4,
                      "parent": "athena", "hidden": true}
  }
}
```

Later in the screenplay:

```
{"action": "spawn_prop", "prop": "athena_laptop"}
```

For *characters*, visibility is controlled by `fade_in` / `fade_out`. There is no character-side `hidden` flag — a character is either in the cast block (and faded in to become visible) or not. Setting `"hidden": true` on a cast entry is silently ignored by `pam_player`.

Why this is worth flagging: the v1.0.0 back-burner note tentatively listed a character-side `hidden` flag. The implementation didn't land — character visibility remained tied to `fade_in` / `fade_out`. Authors who want a character on-stage but invisible (e.g. for an instant reveal) should `fade_in` with a very short `rt` rather than reach for a non-existent flag.

4.3 Z-order

PAM's z-order is implicit. The order in which mobjects are added to the scene determines their painter's-algorithm layering: later additions draw on top of earlier ones. There is no explicit `z-order` field on cast entries or actions.

Order observed in a typical scene:

1. **Scene props** — declared in the `scene_props` or `props` action block at the top. Drawn first; sit in the background.
2. **Characters** — added during `fade_in`. Drawn on top of props. Multiple characters layer in the order they're faded in.
3. **Mid-scene-spawned props** (`spawn_prop`) — drawn on top of characters that already faded in. This is how a held laptop or carried briefcase renders in front of the character's body.
4. **Attached faces** (`attach_face`, v0.9.15) — added per-character at attach time. Drawn on top of the head dot.

5. **Speech bubbles** (`say`, `prop_say`) — added at speech time. Drawn on top of everything below them at that moment.

Where z-order matters:

- **Face on top of head dot.** `attach_face` adds the face mobject *after* the figure was faded in, so the face draws on top. The head dot is set to opacity 0 underneath; the face replaces it visually.
- **Bubble above face above head dot.** A `say` step adds the bubble after `attach_face`, so the bubble draws above the face. Visually: dot (hidden) -> face -> bubble, stacked correctly.
- **Foreground-character occlusion.** If character A (with an attached face) walks behind character B (no face), the face draws on top of B because it was added later. In practice this is rare in TNTD blocking; flagged for future awareness in §2.7.
- **The `focus_reset` gotcha.** See §2.13: persistent bubbles can be drawn over by figures restored by `focus_reset` because Manim's opacity restoration interacts with z-order. Workaround documented there.

No explicit override: there is currently no action like `bring_to_front` or `send_to_back` for characters. An author who needs a specific layering should `fade_in` characters in the desired back-to-front order. This is on the v1.0.0 wishlist.

4.4 Color authority chain

A character's color can be specified in three places:

1. The canonical registry file (e.g. `tntd_characters.json`).
2. The scene's cast block (the `{"action": "cast", "characters": {...}}` step).
3. A `[[ CHARACTER: ... ]]` Fountain+ annotation in the `.fountain` source.

These are merged by `fountain2pam.py` at conversion time. The merge order and authority are:

```
[[ CHARACTER: ]] annotation     (lowest authority; applied first)
        down then
canonical registry file         (highest authority for canonical fields)
        down then
scene cast block                (scene-specific overrides only)
```

What's canonical, what's scene-specific From §1.13 (repeated here for convenience):

Canonical (registry-stored)	Scene-specific (never registry-stored)
`figure_type`	`offset`
`build`	`scale`
`gender`	`pose`
`color`	`facing` (dog only)
`torso_color`	
`style`	
`default_scale`	
`tic_profile`	
`uniforms`	

The canonical registry is authoritative for color. If the registry has `"bevers": {"color": "#44bb66", ...}`, then every scene Bevers appears in renders him at that color, regardless of what the per-scene cast block or the `[[ CHARACTER: ]]` annotation says.

The v0.9.8 fix in detail Before v0.9.8, `fountain2pam.py` would apply a round-robin color palette to any character missing an explicit color, then refuse to overwrite it from the registry. The result: Bevers might appear in green in one scene and amber in another, depending on alphabetical order of the cast block.

The fix re-derives the full six-key palette from the canonical `color` at conversion time and stamps it into the character's `style` dict, overwriting whatever the round-robin assigned. This is why:

- Editing the registry is the only durable way to change a character's color.
- Per-scene `color` / `style.edge_color` overrides are not durable — the next conversion stomps them.
- A `[[ CHARACTER: color=... ]]` annotation in the Fountain file is applied during conversion *before* the registry merge, so it can seed a new character but cannot durably override an existing canonical color.

Practical workflow

- **To add a new character:** write a `[[ CHARACTER: ]]` annotation in the Fountain file with `color=`, `torso_color=`, `gender=`, etc. After the first conversion, the canonical entry exists in the registry; subsequent scenes inherit it automatically.
- **To change an existing character's color:** edit the registry file directly. Don't rely on per-scene overrides.
- **To override color in one scene only:** edit the scene's converted JSON cast block after conversion, and don't re-run the converter on that scene. Or accept the canonical color.

4.5 Registry behavior

The cast registry The `cast` dict is constructed at scene start from the `cast` action block. Each entry follows the structure:

```
cast[name] = {
    "fig":         <live figure instance>,  # None before fade_in
    "figure_type": "...",
    "build":       "...",
    "gender":      "...",
    "pose":        "...",
    "offset":      [x, y, z],
    "scale":       {"sy": ..., "sx": ..., "anchor": "..."},
    "style":       {...},
    "color":       "...",
    "torso_color": "...",
    "tic_profile": [...],
    "uniforms":    {...},
    "facing":      "...",     # dog only
```

```
    "opacity":     1.0,       # set by focus / focus_reset
}
```

`fade_in` writes the live figure into `cast[name]["fig"]`; `fade_out` clears it to `None`. Actions that need the live figure (almost all of them) read `cast[name]["fig"]` via `_resolve_speaker` (§4.1).

The prop registry `props` is a `PropRegistry` instance. Most relevant methods for character-side authoring:

- `props.get(name)` — fetches the prop's `VGroup`, prints a warning on miss.
- `props.get_raw(name)` — same, but silent on miss (used for membership probes).
- `props.add(name, vgroup)` — registers a prop. Used by `spawn_prop` and by `fade_in`'s dual-registration branch.
- `props.world_pos(name)` — returns the `(x, y)` world position of the prop, accounting for any `pam_follows` parent chain.

Dual-registered dogs A `DogGraph` faded in via the cast block appears in both `cast["chekov"]["fig"]` and `props.get("chekov")` (the latter being the `dog.group` `VGroup` with `pam_dog = fig`). Actions that resolve via `_resolve_speaker` see the same `DogGraph` from either side.

The `remove_prop` symmetric-clear lesson `remove_prop` removes a prop from the prop registry *and* clears any cast entry of the same name (for Path C dogs). Before the symmetric-clear fix, removing a dog cleared only the prop side, leaving `cast["chekov"]["fig"]` pointing at a detached figure that no longer rendered. Subsequent `say` / `trot_to` would resolve to the orphan and crash or silently no-op.

Lesson for any future dual-registered figure type: registry maintenance must be symmetric. Add to both registries on creation; remove from both on cleanup. A helper like `_register_dual(cast, props, name, fig, group)` and `_unregister_dual(cast, props, name)` is on the back-burner list.

4.6 Common gotchas

Consolidated footguns from the prior sections. Each entry cross-references the section where the full detail lives.

Identity

- **`figure_type` defaults to `"human"`.** Always set explicitly for alien and dog characters; the default-to-human fallthrough silently rebuilds an alien character as a human skeleton. See §1.1.
- **`build` must match `figure_type` for aliens.** `figure_type: "alien"` + `build: "default"` produces a human-proportioned alien that breaks on morphs to alien-only poses. Use `build: "alien"` or `build: "alien_female"`. See §1.2.
- **`gender` defaults to `"male"` with a warning.** Set explicitly to avoid the warning, especially for female and child characters. See §1.3.

Color

- **The canonical registry is authoritative for color.** Per-scene `color` and `style.edge_color` overrides are stomped on the next `fountain2pam.py` conversion. Edit the registry file directly. See §4.4.

Pose and scale

- **`lying_flat_right` / `lying_flat_left` are required for correct bubble anchoring on horizontal characters.** An ad-hoc `rotate` to horizontal mis-anchors `say` bubbles. See §1.7 and §3.1.1.
- **The `scale` action replaces, doesn't multiply.** `{"action": "scale", "who": "chava", "sy": 0.5}` on a character currently at 0.7 results in 0.5, not 0.35. See §2.4.
- **`rotate` on a character is visual-only.** The figure's internal `pose` and `offset` don't update. A subsequent `morph` snaps the character back to upright unless you counter-rotate first. See §2.5.

Locomotion

- **`walk_to` doesn't update the cast spec.** After fade-out + fade-in, the character returns to the cast-spec offset, not the walked-to x. See §1.8 and §3.2.1.
- **`walk_to_prop` lands *at* the prop x, not next to it.** Follow with a small `walk_to` for offset positioning. See §3.2.6.

Speech bubbles

- **`say` is not parallel-safe at the screenplay level.** The player refuses `say` inside a `parallel` block. See §3.1.1.
- **Persistent speech bubbles anchor world-space, not head-space.** A character who walks while a `persist` bubble is up leaves it behind. Use interleaved short non-persist `say` steps for walk-and-talk. See §3.1.1.
- **`focus_reset` can paint over persistent bubbles** (v0.9.10 gotcha). Issue `clear_bubble` or `clear_all_bubbles` *before* `focus_reset`. See §2.13.

Face attachment (v0.9.15)

- **Faces do not persist across `fade_out` + `fade_in`.** A character that fades out loses their face cleanly; re-attach explicitly after fade-in. See §2.7.
- **The face stays upright during `rotate`.** A character laid flat keeps an upright face by design. See §2.5 and §2.7.
- **Manim CE 0.20.1 (Cairo backend) doesn't composite RGBA alpha correctly** — placeholder PNGs need opaque backgrounds matching the scene `BG_COLOR`. See §2.7.

Tics

- **Tics declared on incompatible body types are silent no-ops** at play time. `tail_wag` on a biped fires harmlessly with no visual. See §1.12 and §3.5.8.
- **`sentence_end` triggers fire on every `say` / `prop_say`.** Set `"suppress_tics": true` on a step to opt out. See §1.12 and §3.1.1.

Resolution

- **`who` and `prop` are separate registries except for Path C dogs.** A human character is never reachable via `prop: name`; a `tv_monitor` prop is never reachable via `who: name`. Dogs are reachable from both sides. See §4.1.
- **Action handlers print a warning and no-op on resolution failure.** A typo in `who` or `prop` produces a console warning but does not crash the render. Watch the log during testing.

Persistent bubble registry

- **Subscene transitions do not auto-clear persistent bubbles.** Explicit `clear_all_bubbles` at beat boundaries is the documented cleanup pattern. See §2.12.
- **`clear_bubble` is a silent no-op on stale clears.** Calling `clear_bubble` for a bubble that's already faded is safe. See §2.11.

Registry maintenance

- **`DogGraph.group` returns a fresh VGroup on every access.** Bind to a local before stamping `pam_*` attributes; otherwise the attributes attach to a throwaway object. See §4.1.
- **Registry cleanup must be symmetric for dual-registered entries.** `remove_prop` clears both the prop registry and the cast entry for Path C dogs. See §4.5.

Status and next steps

This document is intended to evolve into the v1.0.0 frozen reference. At v0.9.15, the following items are still provisional and may shift before the v1.0.0 freeze:

- The `to_x` / `x` locomotion-parameter normalization (the v1.0.0 back-burner item). Currently `walk_to` uses `x`; some older `walk` handlers accept `to_x`. When normalized, the documentation in §3.2 will use whichever key wins.
- The `group_translate` parallel-safety fix (v1.0.0 back-burner). Currently in `_CANNOT_PARALLEL`; if fixed, the §3.8 flag will flip to parallel-safe.
- The `parallel` + `say` constraint (currently tribal knowledge / refused by the player). Either fixed (head-anchored bubbles allowing walk-and-talk) or formally documented with the refusal error.
- The bubble / `focus_reset` ordering gotcha (§2.13). Either fixed in the z-order math or kept as a documented pattern requiring explicit `clear_bubble`.
- Quadruped face attachment (§2.7). Currently `attach_face` no-ops on figures without a `"head"` dot. If `DogGraph` grows a head-positioned face anchor, the action becomes meaningful for dogs.

The companion document `PROP_PROPERTIES.md` (also on the v1.0.0 back-burner list) will mirror this document's structure for prop-side authoring. Topics deferred to that document: prop spawning / despawning lifecycle, the `hidden` flag for props, `DogGraph` as prop, `GovernorGraph`'s autonomous rotation, the `phone` builder's three styles, prop-following lifecycle (`_drag_attached_props`), the `tv_monitor` / `news_anchor` prop-character pattern, and prop-side z-order considerations.

End of Section 4. Section 5 follows as the Face & Appearance reference.

5. Face & Appearance

This section documents PAM's flat-cartoon face system: how character face data is declared in the screenplay JSON, how `attach_face` builds and attaches a face VGroup over a character's head dot, how expression variants are registered and swapped at runtime, and the geometry / layering reference needed to debug face render anomalies.

The face system is implemented in `pam/face_builder.py`. Face data is **separate from skeleton/physics data** — the `"cast"` block (§1) defines the figure (skeleton, build, pose, scale); the `"faces"` block (§5.1) defines the cartoon face that overlays the head dot. The two blocks are independent and may appear in either order in the screenplay JSON, provided both appear before any `attach_face` step that references them.

Faces are built entirely from Manim VMobjects — `Circle`, `Ellipse`, `Rectangle`, `Polygon`, `Line`, `ArcBetweenPoints`. No bitmap or SVG asset loading is involved for character-key faces (the legacy `pam/assets/` path is still supported for file-extension `image` values; see §5.4.1).

- **§5.1** — The `"faces"` block: how project character face data is declared in tntd_characters.json and loaded by pam_player at runtime.
- **§5.2** — The complete FACE_DATA schema: skin, hair, eyes, brows, mouth, clothing, extras, hats, bun geometry.
- **§5.3** — Expression variants: the `"expressions"` key on cast entries, the `{char}_{mouth}` naming convention, compound variants, and `register_variants()`.
- **§5.4** — The `attach_face` action: JSON syntax, scale convention, expression swapping, the `pam_head_ref` anchor fix.
- **§5.5** — Hat geometry reference: triangle / triangle_inv shapes, module-level constants, bun coverage geometry.
- **§5.6** — Layering order reference: front-view and side-view z-order tables.
- **§5.7** — Debugging checklist for missing or mis-anchored faces.

5.1 The "faces" block

Face data for project characters lives in tntd_characters.json (or any screenplay JSON that pam_player loads), in a top-level action block with `"action": "faces"`. The block is a sibling of `"action": "cast"` and structurally mirrors it: a `"characters"` key mapping character keys to per-character face specs.

`face_builder.py` itself ships with only three illustrative entries in `FACE_DATA`: `example_human`, `example_alien`, `example_dog`. Project-specific characters belong in the screenplay JSON, **not** in face_builder.py source code.

JSON shape

```
{
  "action": "faces",
  "characters": {
    "bevers": {
      "name":        "Bevers",
      "note":        "alien · avatar cadet",
```

```
      "skin":        "#8cc47c",
      "hair":        "#3a56a8",
      "hair_style":  "flat_top",
      "eye_type":    "alien",
      "eye_color":   "#c87820",
      "brow_thick":  false,
      "mouth":       "neutral",
      "cloth_color": "#202840",
      "cloth_style": "uniform",
      "ep_color":    "#c8803a",
      "extras":      []
    },
    "freydoon": {
      "name":        "Freydoon",
      "note":        "human · hospital patient",
      "skin":        "#c07848",
      "hair":        "#181818",
      "hair_style":  "short_male",
      "eye_type":    "human",
      "eye_color":   "#6a3808",
      "brow_thick":  true,
      "mouth":       "neutral",
      "cloth_color": "#8090b0",
      "cloth_style": "standard",
      "extras":      []
    }
  }
}
```

Each entry in the `"characters"` dict maps a character key to a face spec dict using the schema in §5.2.

Loading: `load_faces()` When pam_player encounters an `"action": "faces"` step, it calls `face_builder.load_faces(step["characters"])`, which iterates the dict and writes each entry into the module-level `FACE_DATA` dict:

```
def load_faces(characters: dict) -> None:
    for char_key, spec in characters.items():
        FACE_DATA[char_key] = spec
```

Key behaviors:

- **Overwrites** any existing `FACE_DATA` entry with the same key. Re-running `load_faces` with updated data refreshes the entry — useful for iterative tuning during development.
- **Silent no-op** if the `characters` dict is empty.
- **Expression variants are NOT loaded here.** Variants live under the `"expressions"` key in each cast entry and are registered lazily by `register_variants()` on the first `attach_face` for that character (see §5.3.4).

Placement in tntd_characters.json The `"faces"` block must appear **before any scene steps that use `attach_face`**. It can appear before or after the `"cast"` block — the two are independent. Recommended order keeps the character bootstrap at the top of the file:

```
[
  {"action": "title", "text": "..."},
  {"action": "cast",  "characters": { ... }},
  {"action": "faces", "characters": { ... }},
  ... scene steps ...
]
```

5.2 The FACE_DATA schema

Each face spec is a dict. Only `skin` and `hair` are **strictly required** — both are accessed directly via `char["skin"]` and `char["hair"]` in `build_face()` and raise `KeyError` if absent. Every other key uses `char.get(key, default)` and falls back to the documented default when omitted.

5.2.1 Identity

Key	Type	Required	Default	Description
`name`	`str`	no	`"?"`	Display name shown in `list_characters()` output.
`note`	`str`	no	`""`	Free text role/description, also shown in `list_characters()`.

These are documentation fields only — they do not affect rendering.

5.2.2 Skin

Key	Type	Required	Default	Description
`skin`	hex	**yes**	—	Face, neck, and ear fill color. Also used to derive nose and mouth stroke colors (auto-darkened by `_darken()`).

5.2.3 Hair

Key	Type	Required	Default	Description
`hair`	hex	**yes**	—	Hair fill color. Also used to derive eyebrow color (auto-darkened by `0.82`) unless `brow_color` is set explicitly.
`hair_style`	`str`	no	`"short_male"`	One of the named styles in the table below.
`hair_width`	`float`	no	(per style)	*(v0.9.18)* Front-view cap width override in Manim units. Replaces the per-style default. Use to make the hairdo thinner or wider. Not applied in side view (cap width is locked to head depth). See §5.5.5 for per-style defaults and examples.
`hair_height`	`float`	no	(per style)	*(v0.9.18)* Cap height override in Manim units, applied in both front and side views. Use to make the hairdo shorter or taller. Note: cap extends above and below its center, so taller caps lower the hairline unless `hair_offset_y` is also raised.

Key	Type	Required	Default	Description
`hair_offset_y`	`float`	no	(per style)	*(v0.9.18)* Vertical offset from `HEAD_RY` to the cap CENTER, both views. Replaces the per-style default offset. Use to raise or lower the hairline independently of height.

Hair style table:

Value	Description
`"flat_top"`	Flat-topped rectangle. Bevers's signature look.
`"stubble"`	Very thin ellipse cap — barely-there buzz.
`"short_male"`	Low half-ellipse cap with ear visibility.
`"slicked"`	Flat cap with a highlight arc showing the slick.
`"medium_female"`	Cap + side panels hanging to shoulder level.
`"grey_wavy"`	`medium_female` variant with a wavy highlight arc.
`"bob"`	Cap + shorter side panels (chin-length bob).
`"updo"`	Tall dome + top-knot. Thalia's braided updo.
`"bun"`	Cap + round bun circle above. Nona's grey bun.

Styles in the `_HAIR_SHOWS_EARS` set — `short_male`, `stubble`, `slicked`, `flat_top`, `bun`, `updo` — leave the ears visible. The longer styles `medium_female`, `grey_wavy`, `bob` draw side panels that cover the ears. Earring rendering is conditional on this set (see §5.2.8).

5.2.4 Eyes

Key	Type	Required	Default	Description
`eye_type`	`str`	no	`"alien"`	`"alien"` or `"human"`.
`eye_color`	hex	no	`"#c87820"`	Alien: full iris disc color. Human: iris ring color (pupil is always dark).

- **`"alien"`** — single amber/colored disc with a dark pupil and small white catchlight. No white

sclera.
- **"human"** — white sclera circle + small colored iris ring + dark pupil + catchlight.

5.2.5 Eyebrows

Key	Type	Required	Default	Description
brow_color	hex	no	hair darkened by 0.82	Stroke color. Set explicitly for alien characters whose hair-derived brow would be too subtle (e.g. Thalia's "#3a1858" against hair "#5a3080").
brow_thick	bool	no	false	false = standard arch stroke weight. true = heavier brow, reads as more masculine or emphatic. Used for Freydoon.

5.2.6 Mouth

Key	Type	Required	Default	Description
mouth	str	no	"neutral"	See values below.

Value	Description
"neutral"	Very slight upward arc. Resting face.
"smile"	Pronounced upward arc. Friendly.
"flat"	Straight horizontal line. Deadpan.
"wry"	Asymmetric: left corner lower. Ironic or skeptical.
"wry_down"	Same left-corner drop as "wry", but the arc bows downward. Reads as a grimace, resigned displeasure, or "really?".

Side-view note: "wry" maps to "neutral" in profile view — the asymmetry does not read from the side, so the side-view builder silently substitutes a neutral arc. "wry_down" keeps a downward-bowed grimace in profile, so use it when the beat needs visible side-view displeasure rather than a

neutralized skeptical mouth.

5.2.7 Clothing

Key	Type	Required	Default	Description
`cloth_color`	hex	no	`"#202840"`	Main torso/clothing fill.
`cloth_style`	`str`	no	`"standard"`	See values below.
`ep_color`	hex	no	`"#c8a020"`	Epaulette fill. Used when `extras` includes `"epaulettes"` OR when `cloth_style` is `"military"`.
`panel_length`	float	no	`0.38`	Height of a `cloth_style: "panel"` bib. Larger values hang the bib lower while preserving the top anchor.
`panel_width`	float	no	`0.32`	Width of a panel bib.
`panel_color`	hex	no	`cloth_color`	Optional panel fill override.
`panel_border_color`	hex	no	`OUTLINE_COLOR`	Optional panel outline color; useful for ceremonial rims or rank coding.
`panel_border_width`	float	no	`STROKE_WIDTH`	Optional panel outline width; use 4–6 for a visible decorative border.

`cloth_style` value	Description
`"standard"`	Plain trapezoid torso, no detail.
`"blazer"`	Adds a V-neck collar crease (two short angled strokes).
`"military"`	Adds rectangular epaulette slots (filled with `ep_color`). Implicitly enables epaulettes.

`cloth_style` value	Description
`"uniform"`	Adds a center-front line (PAM cadet uniform suggestion).
`"panel"` (v0.9.X)	Rectangular chest bib replacing the trapezoid. See exclusivity note and §5.5.4.

Epaulettes are triggered by either route — explicit `"epaulettes"` in extras, or `cloth_style: "military"`. The internal check is `ep_color if ("epaulettes" in extras or cloth_style == "military") else None`.

Panel exclusivity: `cloth_style: "panel"` is exclusive with all the other decorations. When `cloth_style == "panel"`, the blazer V-neck crease, military/uniform center seam, and shoulder epaulettes are all skipped — even if `"epaulettes"` appears in `extras` or another decoration would otherwise apply. For rank insignia or a name tag on a panel, attach a badge prop using `get_panel_badge_anchor()` (see §5.5.4) rather than relying on `extras: ["epaulettes"]`.

5.2.8 Extras

Key	Type	Required	Default	Description
`extras`	`list[str]`	no	`[]`	Zero or more of the values below.
`blush_color`	hex	no	`"#d04040"`	Blush fill. Use a muted red-pink for aliens; warmer peach for humans.

`extras` value	Description
`"blush"`	Soft ellipses on cheeks.
`"epaulettes"`	Shoulder rank bars (same visual effect as `cloth_style: "military"`).
`"gold_earrings"`	Gold oval earrings at earlobes.
`"pearl_earrings"`	White/cream disc earrings.

Earrings render in front view at both earlobes only for hair styles in `_HAIR_SHOWS_EARS` (see §5.2.3). In side view, the earring appears at the far-side ear (back of head) for the same set of short-hair styles; longer styles (`medium_female`, `grey_wavy`, `bob`) suppress the earring entirely because the side panels would cover it.

Pearl earrings take precedence over gold if both are listed (mutually exclusive via `if/elif` in the builder).

5.2.9 Hats Hats are optional. Omit `hat_style` entirely for a bare-headed character.

Key	Type	Required	Default	Description
`hat_style`	`str`	no	(no hat)	`"triangle"` or `"triangle_inv"`. See §5.5 for full geometry.
`hat_color`	hex	no	`"#303030"`	Hat fill color. Outline uses `OUTLINE_COLOR` at `STROKE_WIDTH`.

The hat is drawn **last** — on top of all hair and face elements. Hair underneath is **not** suppressed; it remains visible at the sides where it extends beyond the hat outline. Because the hat is embedded in the face VGroup, it persists automatically through expression swaps that target the same base character (an `attach_face` with `image: "factor_flat"` keeps Factor's triangle hat).

To show a character **without** their hat mid-scene, define an expression variant in the `"expressions"` block that points to a separate FACE_DATA entry omitting `hat_style`, or define a parallel hat-less base entry under a different key. There is no `"attach_hat"` action; hats are not separate mobjects.

5.2.10 Bun geometry Only relevant when `hair_style == "bun"`.

Key	Type	Required	Default	Description
`bun_offset_y`	`float`	no	`0.155`	Vertical offset from `HEAD_RY` to bun center in front view. Side-view offset is derived automatically as `bun_offset_y * 0.71`.

Larger values push the bun higher above the head. The default `0.155` places the bun close to the cap; if a `triangle_inv` hat is worn, the default leaves enough clearance to cover the bun without clipping (see §5.5.3 for the clearance arithmetic).

Complete schema example: Factor (alien · professor, with triangle hat)

```
{
  "factor": {
    "name":        "Factor",
    "note":        "alien · professor",
    "skin":        "#90aa58",
```

```
    "hair":        "#787878",
    "hair_style":  "slicked",
    "eye_type":    "alien",
    "eye_color":   "#c87820",
    "brow_thick":  false,
    "mouth":       "flat",
    "cloth_color": "#282828",
    "cloth_style": "blazer",
    "extras":      [],
    "hat_style":   "triangle",
    "hat_color":   "#2a6830"
  }
}
```

Complete schema example: Nona (alien · elder, bun + triangle_inv hat + blush)

```
{
  "nona": {
    "name":         "Nona",
    "note":         "alien · elder",
    "skin":         "#809040",
    "hair":         "#909898",
    "hair_style":   "bun",
    "bun_offset_y": 0.155,
    "eye_type":     "alien",
    "eye_color":    "#3a5028",
    "brow_thick":   false,
    "mouth":        "smile",
    "cloth_color":  "#8a6030",
    "cloth_style":  "standard",
    "extras":       ["blush"],
    "blush_color":  "#c03828",
    "hat_style":    "triangle_inv",
    "hat_color":    "#3a4858"
  }
}
```

5.3 Expression variants

Expression variants are per-character face overrides — typically a mouth swap, sometimes a brow change or a blush toggle — used to drive moment-to-moment emotional beats without redefining the full face spec. They live in the `"expressions"` key on each character's **cast** entry (not the `"faces"` block) and are registered into `FACE_DATA` lazily by `register_variants()` on the first `attach_face` step for that character.

You do **not** edit face_builder.py to add expressions, and you do not declare them in the `"faces"` block. All expression data lives in tntd_characters.json under the cast entry for the character.

5.3.1 The `"expressions"` key on cast entries In a character's cast spec, add an `"expressions"` dict alongside `style`, `scale`, `pose`, etc.:

```
{
  "action": "cast",
  "characters": {
    "sidel": {
      "figure_type": "alien",
      "build":       "alien",
      "gender":      "male",
      "pose":        "standing_front",
      "scale":       { "sy": 0.7, "sx": 0.7, "anchor": "lankle" },
      "style":       { "head_label": "Sidel", "edge_color": "#c04010" },
      "expressions": {
        "sidel_smile":   {"mouth": "smile"},
        "sidel_flat":    {"mouth": "flat"},
        "sidel_neutral": {"mouth": "neutral"},
        "sidel_wry":     {"mouth": "wry"},
        "sidel_worried": {"mouth": "flat", "brow_thick": true}
      }
    }
  }
}
```

Each variant key (`"sidel_smile"`, `"sidel_worried"`, ...) is the literal string used as the `"image"` value in subsequent `attach_face` steps. The override dict lists only the fields that change relative to the base face spec — everything else (skin, hair, eyes, clothing, hat) is inherited automatically.

pam_player must copy `"expressions"` from the cast spec into the live cast dict. If `pam_player.py` is missing the line

```
"expressions": spec.get("expressions", {}),
```

in its cast handler, the expressions block never reaches `act_attach_face` and variants will not register. See §5.7, item 1.

5.3.2 Naming convention **Simple variants** (one mouth value, no other overrides) use the pattern `{character}_{mouth_value}`:

```
sidel_smile    sidel_flat    sidel_neutral    sidel_wry
nona_smile     nona_flat     nona_neutral     nona_wry
bevers_smile   bevers_flat   bevers_neutral   bevers_wry
```

The variant key suffix matches the `"mouth"` parameter value directly — no translation needed.

Compound variants combine two parameters and have no single mouth-value equivalent, so they use a descriptive suffix:

Variant suffix	Composition
`_worried`	`"mouth": "flat", "brow_thick": true`
`_flustered`	`"mouth": "flat", "blush": true`

Add new compound names ad-hoc as scenes require them; the naming is convention only, not enforced anywhere in code. Once you settle on a new compound name (e.g. `_smug` = `wry` + `brow_thick`), use it consistently across all characters so a scene's expression beats read uniformly.

5.3.3 The override knobs Only these three fields differ between a base face and its variants. Everything else is inherited from the base FACE_DATA entry — including hats, hair style, clothing, and `bun_offset_y`.

Key	Type	Description
`mouth`	`str`	`"smile"`, `"flat"`, `"neutral"`, or `"wry"`. (`"wry"` maps to `"neutral"` in side view; see §5.2.6.)
`brow_thick`	`bool`	`true` → heavier brow stroke. Reads as concern or anger depending on mouth pairing. Omit (default `false`) for simple mouth-only variants.
`blush`	`bool`	`true` → add `"blush"` to extras. `false` → remove `"blush"` from extras even if the base face has it. Do **not** pass `"extras"` directly in an expression — `register_variants()` handles the list merge.

Any other keys present in an override dict are silently ignored. If you find yourself wanting to override `cloth_color` or `hair_style` mid-scene, you've left the expression-variant lane — define a separate base face under a new key in the `"faces"` block instead.

5.3.4 How `register_variants()` works `act_attach_face` calls `register_variants(char_key, expressions)` the first time a face is attached for a character, **before** the FACE_DATA lookup runs. The function logic:

1. If `char_key` is not in `FACE_DATA`, return early (silent skip — the base face was never loaded).
2. Read the base spec: `base = FACE_DATA[char_key]`.
3. For each `variant_key` → `overrides` pair:
   - **Idempotent skip:** if `variant_key` is already in `FACE_DATA`, leave it alone.
   - Build `merged = {**base}` (a shallow copy of the base).
   - Apply `mouth` and `brow_thick` overrides verbatim.
   - For the `blush` convenience bool: copy `base["extras"]` into a fresh list and add or remove `"blush"` to match the override.
   - Write `merged` into `FACE_DATA[variant_key]`.

Because the function is idempotent, `act_attach_face` calls it on every face swap without harm — second and later calls do nothing for already-registered variants.

`register_variants()` is also exposed as public API for standalone testing:

```python
from pam.face_builder import register_variants
register_variants("sidel", {
    "sidel_smile":   {"mouth": "smile"},
    "sidel_worried": {"mouth": "flat", "brow_thick": True},
})
```

5.4 `attach_face` — the action

Parallel-safe: yes · *Animation:* instant (no tween) · *Required keys:* `who`, `image`

Attach a flat-cartoon face VGroup over a character's head dot. The face is built fresh from FACE_DATA on every call (no caching), positioned via the `pam_head_ref` anchor (§5.4.4), and attached with an updater so it follows the head dot through subsequent motion.

5.4.1 JSON syntax

```
{"action": "attach_face", "who": "freydoon",
 "image": "freydoon", "scale": 0.35}
```

Key	Type	Required	Default	Description
`who`	`str`	yes	—	Character key from the cast block.
`image`	`str`	yes	—	Either a `FACE_DATA` key (no file extension) or a filename in `pam/assets/` (`.png` or `.svg` extension).
`scale`	`float`	no	per-build default (typically `0.35`)	Scale applied to the built VGroup before alignment.
`view`	`str`	no	`"front"`	`"front"`, `"lside"`, or `"rside"` (see §5.4.5).

Source dispatch on `"image"`:

- An `image` value with **no file extension** is treated as a `FACE_DATA` key. `act_attach_face` calls `face_builder.build_face(image, view=view)` to build a fresh VGroup, then attaches it. Unknown keys raise `KeyError` from `build_face()` with a list of valid options.
- An `image` value with a **`.png` or `.svg` extension** is loaded from `pam/assets/` as a bitmap or SVG mobject. This is the legacy path; no FACE_DATA lookup occurs, and `pam_head_ref` does not apply (see §5.4.4).

A character key is valid if it has been registered in `FACE_DATA` via the `"faces"` block (§5.1), **or** is an expression variant lazily registered by `register_variants()` (§5.3.4), **or** is one of the three illustrative example entries (`example_human`, `example_alien`, `example_dog`). Project characters not yet declared in the `"faces"` block fail with `KeyError` on first attach.

5.4.2 Scale convention A scale of **0.30–0.40** places the face at approximately the right visual weight relative to the skeleton figure at default character scale. Tune per scene if a character is unusually large or small on screen.

Keep `scale` identical across all `attach_face` calls for the same character. A scale change between calls produces a visible size jump because each `attach_face` builds a fresh VGroup and rescales it from the default. If a per-scene scale change is genuinely needed, accept the jump and consider hiding it behind a cut or a fade.

5.4.3 Expression swapping `act_attach_face` tears down the prior face automatically before attaching the new one — no `detach_face` step is needed between swaps. Typical mid-scene swap pattern:

```
{"action": "attach_face", "who": "sidel",
 "image": "sidel", "scale": 0.35},

{"action": "say", "who": "sidel",
 "text": "The Council's decision is final."},

{"action": "attach_face", "who": "sidel",
 "image": "sidel_worried", "scale": 0.35},

{"action": "say", "who": "sidel",
 "text": "This will not go well for us."},

{"action": "attach_face", "who": "sidel",
 "image": "sidel_smile", "scale": 0.35}
```

Because hats are embedded in the face VGroup (§5.2.9), they persist across swaps that resolve to variants of the same base character. Factor's `triangle` hat stays in place through every `factor_*` swap.

5.4.4 The `pam_head_ref` anchor fix This is the alignment trick that makes tall hair and hats render correctly. Understanding it is essential for debugging mis-anchored faces.

`build_face()` tags the returned VGroup with an attribute pointing at the head oval Ellipse (layer 5 in front view, layer 6 in side view):

```
face.pam_head_ref = head    # the head oval Ellipse
```

`act_attach_face` uses this attribute to align the face by the **head oval center**, not the VGroup bounding-box center. The shift is:

```
face.shift(head_dot_center - face.pam_head_ref.get_center())
```

Why this exists: without `pam_head_ref`, alignment uses `face.move_to(head_dot_center)`, which moves the VGroup's bounding box center to the head dot. Tall hair (buns, updos) and hats extend the bounding box upward, so `move_to()` pushes the face **downward** until the bounding box center coincides with the head dot — leaving the head oval below the head dot, with the hair or hat occupying the space where the face should be.

`pam_head_ref.get_center()` returns the **live world position** of the head oval Ellipse after any prior scale or shift, so the formula stays correct after scaling — the scale operation moves the Ellipse along with the rest of the VGroup, and `get_center()` reads the post-scale position.

Fallback for bitmap / SVG faces: PNG and SVG assets loaded from `pam/assets/` have no `pam_head_ref` attribute. `act_attach_face` falls back to the legacy `move_to(head_dot_center)` behavior in that case. If a bitmap face mis-anchors, the fix is to crop or compose the source image so its center matches the intended head center — there is no `pam_head_ref` equivalent for bitmap sources.

Related anchor: `cloth_style: "panel"` (v0.9.X) exposes a parallel attribute `face.pam_panel_ref` pointing at the panel Rectangle, used by `get_panel_badge_anchor()` to compute the name-tag / badge attachment point. Same scale-survival pattern as `pam_head_ref`. See §5.5.4.

5.4.5 Side views

`build_face()` accepts a `view` parameter passed through from `attach_face`:

```
{"action": "attach_face", "who": "thalia",
 "image": "thalia", "view": "lside", "scale": 0.35}
```

Value	Direction
`"front"` (default)	Front-facing portrait.
`"lside"`	Profile facing LEFT (-x direction).
`"rside"`	Profile facing RIGHT (+x direction).

Use `view: "lside"` / `"rside"` whenever the character's skeleton is in a side pose (e.g. during a `walk_to`, or while standing in a side-facing pose). The profile face is built from a separate code path — wider head ellipse (`_SIDE_HEAD_RX = 0.30` vs front `HEAD_RX = 0.25`), single eye/brow/mouth on the face side, a nose polygon protruding beyond the head edge, a single far-side ear behind the head oval, and a far-side hair back panel for longer styles. See §5.6 for the full side-view z-order.

Unknown `view` values raise `ValueError` from `build_face()`.

5.5 Hat and hair geometry reference

This section documents the constants used by the hat and hair builders, and the per-character override fields that can adjust hair geometry without editing source. When a hat clips the hair, fails to cover a bun, or sits at the wrong height, the fix is usually to adjust `bun_offset_y` (§5.2.10) or the hair geometry overrides (§5.5.5) rather than to change the constants — but the constants are listed here for completeness so anomalies can be diagnosed without reading source.

All dimensions are in Manim units at scale=1.0.

5.5.1 Hat shapes `"triangle"` — Triangle hat. Base at `HEAD_RY` (top of the head oval), tip pointing up. Defaults are equilateral; both dimensions are overridable per character (v0.9.18).

Dimension	Default	Override
Base y	`HEAD_RY = 0.28`	(anchored, not overridable)
Base width	`0.52`	`hat_width`
Tip y above base	`0.45` (equilateral for default base)	`hat_height` (absolute, in Manim units)

When only `hat_width` is given, the tip height auto-derives as the equilateral value for the new base (`hat_width * 0.866`), so width-only changes preserve the equilateral silhouette. When only `hat_height` is given, the base width stays at `0.52`. Specifying both gives a fully free triangle (e.g., a flat wide cone or a tall narrow spike). Factor's hat. Pairs with Bevers's `flat_top` rectangle as a father-son shape family (square → triangle).

`"triangle_inv"` — Inverted triangle hat. Wide base at the top, tip pointing down to just above the hairline. Defaults are tuned to fully contain a bun; both dimensions are overridable per character (v0.9.18).

Dimension	Default	Override
Tip y (anchor point)	`HEAD_RY - 0.10 = 0.18`	(anchored, not overridable — keeps the hat seated on the head crown)
Total vertical span	`0.20` (so top is at `HEAD_RY + 0.10 = 0.38`)	`hat_height` (sets total span; top y = tip y + `hat_height`)
Base width (at top)	`0.80` (wide enough to fully contain a bun)	`hat_width`

The tip stays anchored at `HEAD_RY - 0.10` regardless of override, so the hat keeps its bun-covering relationship to the head when only width is tweaked. Nona's hat.

Side-view triangle hats: The same `hat_width` and `hat_height` values apply directly in side view (no front/side scaling — unlike `"square"` which scales front width down for the profile). Side-view defaults are slightly different: `triangle` default base in profile is `2 * _SIDE_HEAD_RX + 0.04` ≈ `0.64` to match the head's front-to-back depth; `triangle_inv` default base in profile is `0.76`. Specifying `hat_width` overrides both views to the same absolute value, so the hat looks identical from front and side when overridden — this is usually what you want for a recognizable silhouette.

Worked syntax examples:

Example 1 — the original Factor case (flat wide triangle). The exact configuration that motivated the v0.9.18 parameterization:

```
{
  "factor": {
    "hat_style":  "triangle",
    "hat_color":  "#2a6830",
    "hat_width":  0.72,
```

```
    "hat_height": 0.03
  }
}
```

Produces a base `0.72` wide at `HEAD_RY`, tip `0.03` above — effectively a flat triangular ridge. Same shape in front and side view.

Example 2 — width-only override (auto-equilateral). A wider equilateral triangle. The tip auto-derives at `hat_width * 0.866` ≈ `0.52` above the base:

```
{
  "factor": {
    "hat_style":  "triangle",
    "hat_color":  "#2a6830",
    "hat_width":  0.60
  }
}
```

Example 3 — tall narrow spike.

```
{
  "factor": {
    "hat_style":  "triangle",
    "hat_color":  "#2a6830",
    "hat_width":  0.30,
    "hat_height": 0.80
  }
}
```

Example 4 — narrower `triangle_inv` keeping default height. The base narrows but the hat still seats at the same y relative to the head:

```
{
  "nona": {
    "hat_style": "triangle_inv",
    "hat_color": "#5a4030",
    "hat_width": 0.60
  }
}
```

Example 5 — taller `triangle_inv` (taller dome over a bigger bun). Tip stays at `HEAD_RY - 0.10`; top extends up to `HEAD_RY - 0.10 + 0.30 = 0.48`:

```
{
  "nona": {
    "hat_style":  "triangle_inv",
    "hat_color":  "#5a4030",
    "hat_height": 0.30
  }
}
```

Example 6 — baseline (no overrides, current equilateral default). Both fields omitted; renders exactly as before v0.9.18:

```
{
  "factor": {
    "hat_style": "triangle",
    "hat_color": "#2a6830"
  }
}
```

5.5.2 Module-level geometry constants These constants live in `face_builder.py` at module scope and can be overridden at runtime to re-tune face geometry globally. Per-character tuning should use FACE_DATA fields instead; modifying the constants affects every face.

Constant	Value	Description
`OUTLINE_COLOR`	`"#18120c"`	Default stroke color for all elements (near-black warm).
`STROKE_WIDTH`	`2.2`	Outline stroke weight (Manim units × 100).
`HEAD_RX`	`0.25`	Head oval x-radius (front view).
`HEAD_RY`	`0.28`	Head oval y-radius (slightly taller than wide).
`_SIDE_HEAD_RX`	`0.30`	Profile head front-to-back radius (side view).
`EYE_Y`	`0.05`	Eye center y (above head center).
`EYE_DX`	`0.09`	Eye center x offset from vertical centerline.
`BROW_Y`	`0.14`	Eyebrow arc centerline y.
`NOSE_Y`	`-0.04`	Nose arc centerline y.
`MOUTH_Y`	`-0.13`	Mouth arc/line centerline y.
`NECK_W`, `NECK_H`	`0.12`, `0.12`	Neck rectangle dimensions.
`EYE_R_OUTER`	`0.038`	Outer eye circle radius (sclera or alien iris).
`EYE_R_IRIS`	`0.020`	Human iris ring radius.
`EYE_R_PUPIL`	`0.010`	Pupil dark circle radius.
`EYE_R_LIGHT`	`0.005`	Catchlight specular dot radius.
`PANEL_WIDTH` (v0.9.X)	`0.32`	Width of the `cloth_style: "panel"` rectangular bib.
`PANEL_TOP_Y` (v0.9.X)	`-0.22`	Top edge of the panel — the symbolic "neck_bottom" reference.
`PANEL_BOTTOM_Y` (v0.9.X)	`-0.60`	Bottom edge of the default panel.
`PANEL_DEFAULT_LENGTH` (v0.9.X)	`0.38`	Derived bib height, `PANEL_TOP_Y - PANEL_BOTTOM_Y`.
`PANEL_SIDE_OFFSET_X` (v0.9.X)	`0.04`	Side-view shift onto face side (multiplied by `xs`).

Constant	Value	Description
`PANEL_BADGE_Y_OFFSET` (v0.9.X)	`0.08`	Badge anchor distance below `PANEL_TOP_Y`.

Approximate bounding box of a built face at scale=1.0:

Feature	y
Top of tallest updo hair	≈ `+0.86`
Top of typical short cap	≈ `+0.50`
Head center	`0.00`
Chin	≈ `-0.28`
Bottom of clothing shape	≈ `-0.45`

Side-view bounding box differs slightly:

Feature	y or x
Top of updo	`y` ≈ `+0.87`
Top of `flat_top`	`y` ≈ `+0.54`
Head top	`y` ≈ `+0.28`
Nose tip (lside)	`x` ≈ `-0.38` (rside: `x` ≈ `+0.38`)
Back of head	`x` ≈ `±0.30`
Clothing bottom	`y` ≈ `-0.45`

5.5.3 Bun coverage by `triangle_inv` The default `bun_offset_y = 0.155` places Nona's bun so that:

Feature	y
Bun center	`HEAD_RY + 0.155 = 0.435`
Bun top (radius `0.15`)	`0.585`
`triangle_inv` base (top of hat)	`HEAD_RY + 0.52 = 0.800`
Clearance	`0.215`

If a custom `bun_offset_y` pushes the bun top above `0.65`, the `triangle_inv` will clip the bun crown. Either lower the bun (decrease `bun_offset_y`) or accept the clip — the hat builder does not currently expose a "rise" parameter for the hat itself.

In side view the bun shifts toward the far side of the head, and the bun y-offset is recomputed as `bun_offset_y * 0.71`. A bun that just barely clears the hat in front view will have additional clearance in side view because the side-view multiplier brings the bun lower.

5.5.4 `cloth_style: "panel"` geometry (v0.9.X) When `cloth_style == "panel"`, the default trapezoid torso is replaced by a Rectangle of dimensions `PANEL_WIDTH × PANEL_DEFAULT_LENGTH = 0.32 × 0.38`.

Dimension	Value
Width	`PANEL_WIDTH = 0.32`
Height	`PANEL_DEFAULT_LENGTH = 0.38`
Top edge y	`PANEL_TOP_Y = -0.22`
Bottom edge y	`PANEL_BOTTOM_Y = -0.60`
Center y	`PANEL_TOP_Y - PANEL_DEFAULT_LENGTH / 2 = -0.41`
Front-view center x	`0`
Side-view center x	`xs * PANEL_SIDE_OFFSET_X = xs * 0.04`

`PANEL_TOP_Y = -0.22` is the symbolic "neck_bottom" reference — it matches the y-coordinate of the inner top edge of the default trapezoid (the torso's neck opening). The panel hangs from this y down to `PANEL_BOTTOM_Y = -0.60`. The top edge stays fixed at the neck anchor; changing the panel length pushes only the bottom edge downward.

Side view: the same Rectangle dimensions are used; only the x-shift differs. Far-side arm occlusion when a character is in a side pose comes from skeleton draw order — `face_builder.py` does no special handling for the far-side arm crossing the panel.

Badge / name-tag anchor. `build_face()` tags the panel Rectangle as `face.pam_panel_ref` (analogous to `face.pam_head_ref` for the head oval — see §5.4.4). Consumers should retrieve the world-space anchor via the public helper:

```
from pam.face_builder import get_panel_badge_anchor
anchor = get_panel_badge_anchor(face)    # np.ndarray (x, y, z), or None
```

The helper returns the live world-space point at `PANEL_BADGE_Y_OFFSET = 0.08` below the panel top, computed from `panel.get_top()` and `panel.get_bottom()` so the anchor scales correctly with the face — at scale=1.0 the offset is `0.08`; at scale=0.35 it returns `0.028`. Returns `None` for non-panel faces (no `pam_panel_ref` attribute).

In face-local coordinates the anchor is `(0, -0.30, 0)` front view and `(xs * 0.04, -0.30, 0)` side view, but you should not hardcode these — use `get_panel_badge_anchor()` so the value stays correct after `act_attach_face` has applied scale and shift.

Exclusivity. Setting `cloth_style: "panel"` skips all the other torso decorations: blazer crease, military/uniform seam, and shoulder epaulettes (even when `"epaulettes"` is listed in `extras` or `cloth_style == "military"` would otherwise apply — but those two values are mutually exclusive with `"panel"` anyway). Attach badges and name tags as separate props using the anchor above.

5.5.5 Hair geometry overrides (v0.9.18) The defaults built into `_build_hair_front` and `_build_hair_side_front` are tuned for the original eleven-or-so TNTD characters; on other faces they often read slightly too big or too small. Three optional FACE_DATA fields let you tune the *primary cap shape* of any `hair_style` in place, without editing the builder.

Field	Applies to	Effect
`hair_width`	Front view only	Cap width in Manim units. Replaces the per-style default. Smaller = thinner hair, larger = wider. Not applied in side view because the side cap width is locked to the head's front-to-back depth (`2 * _SIDE_HEAD_RX + 0.04` ≈ `0.64`), which is a different physical dimension.
`hair_height`	Both views	Cap height in Manim units. Replaces the per-style default. Smaller = shorter hair, larger = taller. Cap extends both above and below its center, so a taller cap lowers the apparent hairline unless paired with `hair_offset_y`.
`hair_offset_y`	Both views	Vertical offset from `HEAD_RY` to the cap CENTER. Replaces the per-style default offset. Use to raise or lower the hairline independently of height. Positive = higher, negative = lower.

All three default to absent (i.e., to the per-style hardcoded value), so omitting them preserves existing rendering exactly. Setting them replaces the default outright (absolute override, not a multiplier — same semantic as the existing `hat_width` / `hat_height` fields).

Scope — primary cap shape only. The overrides affect the single largest hair element (the cap ellipse, or for `flat_top` the cap rectangle). Secondary shapes are *not* affected:

- `"updo"` — the dome (`0.38 × 0.46`) and the top knot (`radius 0.10`) keep their hardcoded geometry.
- `"bun"` — the bun ball (`radius 0.15`) keeps its hardcoded geometry. Use `bun_offset_y` (§5.2.10) to move it vertically.
- `"slicked"` — the highlight arc keeps its hardcoded geometry.
- `"grey_wavy"` — the wave highlight arc keeps its hardcoded geometry.
- `"medium_female"`, `"grey_wavy"`, `"bob"` — the back panels drawn behind the head oval (the longer-hair side panels) keep their hardcoded geometry. `hair_back_width` / `hair_back_height` fields may follow in a later release.

If you need to tune those secondary shapes, edit `face_builder.py` directly for now.

5.5.6 Per-style hair defaults (v0.9.18) Defaults the overrides replace, by style and view. Front-view defaults matter for both width and height tuning; side-view defaults matter only for height (width is locked).

Front view — primary cap shape:

`hair_style`	Default width	Default height	Default offset y	Shape
`"flat_top"`	`0.54`	`0.29`	`+0.06`	Rectangle
`"stubble"`	`0.35`	`0.10`	`-0.01`	Ellipse
`"short_male"`	`0.52`	`0.22`	`+0.03`	Ellipse
`"slicked"`	`0.54`	`0.18`	`+0.01`	Ellipse + arc

hair_style	Default width	Default height	Default offset y	Shape
"medium_female"	0.52	0.26	+0.05	Ellipse + back panels
"grey_wavy"	0.52	0.26	+0.05	Ellipse + back panels + wave arc
"bob"	0.52	0.26	+0.05	Ellipse + shorter back panels
"updo"	0.52	0.26	+0.05	Ellipse + dome + knot
"bun"	0.50	0.22	+0.02	Ellipse + bun ball
(fallback)	0.50	0.22	+0.02	Ellipse

Side view — primary cap shape (height & offset only):

hair_style	Default height	Default offset y
"flat_top"	0.26	+0.08
"stubble"	0.13	+0.01
"short_male"	0.22	+0.03
"slicked"	0.17	+0.01
"medium_female" / "grey_wavy" / "bob"	0.24	+0.04
"updo"	0.24	+0.04
"bun"	0.20	+0.02
(fallback)	0.20	+0.02

Side-view heights differ slightly from front-view (most are 0.02–0.04 smaller because the profile head is taller relative to its width). The same hair_height value applies to both views; if you tune for front-view, the side view scales by the same absolute amount, which generally reads correctly. Width is locked in side view — the cap always spans the head's front-to-back depth, regardless of hair_width.

5.5.7 Syntax examples (v0.9.18) Example 1 — baseline (no overrides, current default). Factor keeps the per-style slicked defaults (0.54 × 0.18 cap at HEAD_RY + 0.01):

```
{
  "factor": {
    "hair":       "#a0a0a0",
    "hair_style": "slicked"
  }
}
```

Example 2 — narrower slicked hair. Override width only, leave height and offset alone. Useful when the default 0.54-wide cap overhangs a narrower head:

```
{
  "factor": {
    "hair":        "#a0a0a0",
```

```
    "hair_style":  "slicked",
    "hair_width":  0.46
  }
}
```

Example 3 — taller cap, hairline preserved. If you just bump `hair_height` the cap extends both up and down from its center, dropping the hairline by half the height delta. Pair with `hair_offset_y` to lift the cap by the same amount so the hairline stays put. Slicked default is height `0.18` at offset `+0.01`; adding `0.10` to both gives a cap of height `0.28` sitting `0.05` higher than before, with the hairline unchanged:

```
{
  "factor": {
    "hair":           "#a0a0a0",
    "hair_style":     "slicked",
    "hair_height":    0.28,
    "hair_offset_y":  0.06
  }
}
```

Example 4 — shorter cap, hairline preserved. Mirror of Example 3. To shrink the cap by `0.06` (from `0.18` default to `0.12`) while keeping the hairline at the original position, lower `hair_offset_y` by half the height reduction (`0.01 - 0.03 = -0.02`):

```
{
  "factor": {
    "hair":           "#a0a0a0",
    "hair_style":     "slicked",
    "hair_height":    0.12,
    "hair_offset_y": -0.02
  }
}
```

Example 5 — "barely-there" stubble. The `"stubble"` style is already minimal (`0.35 × 0.10`), but you can push it further:

```
{
  "yannos": {
    "hair":        "#403028",
    "hair_style":  "stubble",
    "hair_width":  0.28,
    "hair_height": 0.06
  }
}
```

Example 6 — wider `flat_top`. Bevers's defining shape, made noticeably blockier:

```
{
  "bevers": {
    "hair":        "#e8c870",
    "hair_style":  "flat_top",
    "hair_width":  0.66,
```

```
    "hair_height": 0.34
  }
}
```

Example 7 — tighter `bun` cap, bun ball unaffected. The cap shrinks; the bun ball (radius `0.15`) ignores the override because it's a secondary shape. Use `bun_offset_y` to reposition the ball if needed:

```
{
  "nona": {
    "hair":           "#a8a8a8",
    "hair_style":     "bun",
    "hair_width":     0.44,
    "hair_height":    0.18,
    "bun_offset_y":   0.18
  }
}
```

Example 8 — `updo` with low base cap. The dome and knot keep their hardcoded geometry (Thalia's silhouette stays recognizable), but the base cap can be tuned independently:

```
{
  "thalia": {
    "hair":           "#5a3080",
    "hair_style":     "updo",
    "hair_height":    0.20,
    "hair_offset_y":  0.02
  }
}
```

Example 9 — offset-only tweak. If the hairline is the only thing off (the cap shape is fine, it just sits too high or too low), use `hair_offset_y` alone. Here `short_male` sits `0.04` higher than its default `+0.03`:

```
{
  "freydoon": {
    "hair":           "#2c2018",
    "hair_style":     "short_male",
    "hair_offset_y": 0.07
  }
}
```

Example 10 — the same override on multiple characters with different defaults. The override is absolute, not a multiplier — so the same `hair_width: 0.46` produces different effects on characters whose defaults differ. On `flat_top` (default `0.54`) it narrows by `0.08`; on `bun` (default `0.50`) it narrows by `0.04`; on `stubble` (default `0.35`) it widens by `0.11`. Always cross-reference §5.5.6 before applying the same value across characters.

5.6 Layering order reference

Both views use a fixed z-order — face elements are added to the VGroup in the order below, so later elements draw on top of earlier ones.

5.6.1 Front view (back to front)

Layer	Element	Notes
1	Clothing	Trapezoid torso + any collar / epaulette detail.
2	Neck	Skin-colored rectangle.
3	Ears	Visible only for hair styles in `_HAIR_SHOWS_EARS`.
4	Hair back panels	For `medium_female`, `bob`, `grey_wavy` — drawn BEHIND head oval.
5	**Head oval**	Skin-colored Ellipse. Covers inner portions of hair panels. Tagged as `face.pam_head_ref` (§5.4.4).
6	Hair front cap	Drawn ON TOP of head oval.
7	Blush	Low-opacity ellipses, if extras includes `"blush"`.
8	Eyes	Sclera + iris + pupil + catchlight (human) or disc + pupil + catchlight (alien).
9	Eyebrows	Stroke-only arcs.
10	Nose	Skin-colored arc.
11	Mouth	Arc or line per `mouth` value.
12	Earrings	If extras includes `"gold_earrings"` or `"pearl_earrings"`.
13	Hat	If `hat_style` is set. Drawn LAST — on top of all hair and face.

5.6.2 Side view (back to front)

Layer	Element	Notes
1	Clothing	Profile trapezoid + optional far-shoulder epaulette.
2	Neck	Offset slightly toward face side.
3	Far-side ear	Drawn BEFORE head oval — head oval covers the inner half.
4	Far-side hair back panel	For `medium_female`, `bob`, `grey_wavy` only.

Layer	Element	Notes
5	Nose polygon	Drawn BEFORE head oval — tip protrudes beyond face edge; head oval covers the base.
6	**Head oval**	Wider profile Ellipse. Covers ear/panel/nose bases. Tagged as `face.pam_head_ref`.
7	Hair front cap	Drawn ON TOP of head oval.
8	Blush	Single spot, near/face side only.
9	Single eye	Near/face side.
10	Single eyebrow	Near/face side.
11	Mouth	Near/face side, shorter span than front view.
12	Earring	Far-side, short-hair styles only.
13	Hat	If `hat_style` is set. Drawn LAST.

The position of layer 5 (nose polygon, side view) is the key trick that makes the side-view nose work — drawing the polygon *before* the head oval lets the head oval mask the polygon's base, leaving only the protruding tip visible beyond the face edge.

5.7 Debugging checklist — when faces don't appear or mis-anchor

Ordered by frequency of occurrence in past renders.

1. Console warning: `PAMPlayer: attach_face - unknown face_builder key 'sidel_smile'.`

The variant key is not in `FACE_DATA` at lookup time. Check, in order:

- The base character (`"sidel"`) is declared in the `"faces"` block in tntd_characters.json. Without the base, `register_variants()` returns early (silent skip) and the variant never enters `FACE_DATA`.
- The `"expressions"` block on the cast entry for `"sidel"` lists `"sidel_smile"` with the expected override fields.
- `pam_player.py` is copying `"expressions"` from the cast spec into the live cast dict:

  ```
  "expressions": spec.get("expressions", {}),
  ```

  If this line is absent from the cast handler, `act_attach_face` receives an empty expressions dict and `register_variants()` registers nothing.
- `face_builder.py` is v0.9.17 or later (has `register_variants()`).
- Variant key spelling matches exactly — `FACE_DATA` keys are case-sensitive. `"Sidel_Smile"` ≠ `"sidel_smile"`.

2. Face appears but is positioned wrong — head dot visible behind/below the face, or face floats above the body.

- The face has tall hair (bun, updo) or a hat, and `pam_head_ref` is not being honored. This means either (a) the face was built from a bitmap/SVG asset (no `pam_head_ref` attribute), or (b) `act_attach_face` has fallen through to the legacy `move_to()` path. Verify which branch fires in `act_attach_face`.
- The character has been moved (`walk_to`, `group_translate`, etc.) and the head-dot follower updater has lagged or been removed. Re-attach the face after motion completes.

3. Hat clips the bun.

- Decrease `bun_offset_y` (e.g. `0.155` → `0.140`) or switch from `triangle` to `triangle_inv`. See §5.5.3 for the clearance arithmetic.

4. Expression swap produces a visible size jump.

- `scale` differs between the two `attach_face` calls. Make them identical. See §5.4.2.

5. Hat disappears on expression swap.

- The expression variant was declared in `"expressions"` (so it should inherit the base face's hat), but at some point a separate FACE_DATA entry without `hat_style` got loaded under the variant's key. Check the `"faces"` block — only the base character should be declared there; variants must come through `"expressions"`, not through duplicate face entries.

6. Side-view face has front-view geometry.

- `view` parameter missing from the `attach_face` step. Default is `"front"`. Pass `"view": "lside"` or `"view": "rside"` explicitly when the figure is in a side pose.

7. Earrings missing in front view despite `"gold_earrings"` in extras.

- The character has a long hair style (`medium_female`, `grey_wavy`, or `bob`) which suppresses earring rendering. Either switch to a hair style in `_HAIR_SHOWS_EARS` (`short_male`, `stubble`, `slicked`, `flat_top`, `bun`, `updo`), or accept the suppression — earrings will still appear at the far-side ear in side view if the hair style is in `_HAIR_SHOWS_EARS`.

8. Epaulettes set in `extras` (or `cloth_style: "military"`) but not rendering.

- Character has `cloth_style: "panel"`, which silently drops shoulder epaulettes — the panel replaces the trapezoid, so the shoulder coordinates the epaulette builder uses (`x = ±0.30, y = -0.26`) fall outside the panel's geometry. This is intentional exclusivity (§5.5.4). Move rank insignia to a badge prop attached at `get_panel_badge_anchor(face)`.

9. Brow color barely visible against hair.

- The default brow is hair darkened by `0.82`, which is too subtle for dark base hair. Set `brow_color` explicitly to a darker tone (e.g. Thalia's `"#3a1858"` against hair `"#5a3080"`).

10. Wry mouth looks neutral on a side-pose beat.

- This is expected behavior — `"wry"` maps to `"neutral"` in side view because the asymmetry does not read from the side. See §5.2.6. If the wry read is critical for the beat, either rework the blocking so the character is front-facing, or accept the neutral substitution.

11. List all registered face keys for verification.

After a render, dump the FACE_DATA registry to see what's actually registered (including lazily-registered variants):

```
from pam.face_builder import list_characters
list_characters()
```

Run this immediately after a scene's first `attach_face` step to verify both base characters and expression variants made it into `FACE_DATA`.

End of Section 5.

6. Wardrobe overlays, badges, and name tags

This section covers decorative mobjects that attach to an already-live figure. These are not pose data, prop registry entries, or skeleton joints. They are small Manim mobjects with updaters that follow an anchor point on the character every frame.

The authoring pattern mirrors `attach_face`: attach once after `fade_in`, let the updater track walks and morphs, and either detach explicitly or allow `fade_out` to clean up the attachment bundle.

6.1 Common lifecycle rules

All attached wardrobe overlays share the same lifecycle conventions:

Rule	Consequence
Idempotent attach	Re-attaching the same kind of item first removes the previous one.
Decorative only	They do not change pose dictionaries, joint positions, hit detection, or action targeting.
Updater-driven	They follow live joint locations through `walk_to`, `run_to`, `morph`, `turn`, and scale changes.
Explicit detach available	Each attach action has a matching `detach_*` action.
Fade-out cleanup	`fade_out` removes faces, torso icons, gloves, shoes, name tags, and dog harnesses in the same concurrent fade bundle as the body.
Joint-dependent no-op	If the figure lacks the needed joints, the handler prints a diagnostic or skips cleanly.

These overlays are best used for readable character coding: uniforms, delivery gloves, shoes, badges, harness labels, or temporary disguise markers.

6.2 `attach_torso_icon` / `detach_torso_icon`

`attach_torso_icon` anchors a small icon to the figure's torso midpoint. In the attached source, the JSON-facing preset factory currently ships one preset: `name_tag`. The low-level `HumanGraph.attach_torso_icon(icon, scene)` method can attach any pre-built VMobject or VGroup, but the JSON action exposes only named presets.

Key	Type	Required	Default	Meaning

Key	Type	Required	Default	Meaning
`who`	string	yes	-	Character key.
`preset`	string	yes	-	Currently `"name_tag"`.
`text`	string	no	`"NAME"`	Text for the `name_tag` preset.
`fill`	hex	no	`"#f0e4b8"`	Plate fill color.
`stroke`	hex	no	`"#3a2a14"`	Plate outline and text color.
`scale`	float	no	`1.0`	Scale applied to the built icon.

```
{"action": "attach_torso_icon", "who": "yannos",
 "preset": "name_tag", "text": "Y. YANNOS",
 "fill": "#f0e4b8", "stroke": "#3a2a14", "scale": 1.0}

{"action": "detach_torso_icon", "who": "yannos"}
```

The torso anchor uses the midpoint between `lshoulder` and `lhip`, with the x-coordinate forced to the figure centerline. This keeps the icon centered on both ordinary humans and split-torso alien builds.

6.3 `attach_gloves` / `detach_gloves`

`attach_gloves` creates one flat-cartoon mitten ellipse at each wrist. It is intentionally low-detail: a colored mass with a dark outline, not articulated fingers.

Key	Type	Required	Default	Meaning
`who`	string	yes	-	Character key.
`color`	hex	yes	-	Glove fill color.
`size`	float	no	`0.22 * fig._scale_sy`	Glove width in world units.
`stroke`	hex	no	`"#18120c"`	Outline color.

```
{"action": "attach_gloves", "who": "bevers", "color": "#cc3333"}
{"action": "attach_gloves", "who": "freydoon", "color": "#3a2a14", "size": 0.18}
{"action": "detach_gloves", "who": "bevers"}
```

The method requires `lwrist` and `rwrist`. Quadrupeds and non-humanoid figures do not have the required wrist joints, so the action is not useful for dogs or the Governor.

6.4 `attach_shoes` / `detach_shoes`

`attach_shoes` creates one elongated ellipse at each ankle. The shoe updater tracks both ankle position and the figure's facing direction. When `walk_to` or `run_to` changes `fig.facing`, the shoes mirror so their long axis still points in the direction of travel.

Key	Type	Required	Default	Meaning
`who`	string	yes	-	Character key.

Key	Type	Required	Default	Meaning
`color`	hex	yes	-	Shoe fill color.
`size`	float	no	`0.32 * fig._scale_sy`	Shoe length in world units.
`stroke`	hex	no	`"#18120c"`	Outline color.

```
{"action": "attach_shoes", "who": "bevers", "color": "#1a1a1a"}
{"action": "attach_shoes", "who": "thalia", "color": "#3a2a14", "size": 0.30}
{"action": "detach_shoes", "who": "thalia"}
```

Shoe height is about 40% of the requested length, giving a readable 3:1 flat shoe silhouette.

6.5 `attach_name_tag` / `detach_name_tag`

`attach_name_tag` attaches a text-only label at a semantic badge anchor. It is different from the `attach_torso_icon name_tag` preset: the torso icon is a rectangular plate centered on the torso; `attach_name_tag` is a free text mobject attached to either a dog harness or a panel-clothed face.

Anchor resolution order:

1. `fig.get_harness_nametag_anchor()` for a DogGraph with a harness.
2. `face_builder.get_panel_badge_anchor(fig.head_face)` for a human/alien face with `cloth_style: "panel"`.
3. No anchor available: diagnostic and no-op.

Key	Type	Required	Default	Meaning
`who`	string	yes	-	Character key.
`text`	string	yes	-	Label text; embedded `\n` is supported.
`font_size`	float	no	`18`	Manim text font size.
`color`	hex	no	`"#ffffff"`	Text color.
`dy`	float	no	`0.0`	Extra vertical nudge in world units.

```
{"action": "attach_name_tag", "who": "chekov", "text": "CHEKOV"}
```

```
{"action": "attach_face", "who": "sidel", "image": "sidel", "scale": 0.35}
{"action": "attach_name_tag", "who": "sidel",
 "text": "CMDR\nSIDEL", "font_size": 14, "color": "#c8a020", "dy": 0.02}
```

For panel-clothed humans and aliens, `attach_face` must happen before `attach_name_tag`, because the panel badge anchor lives on the attached face VGroup as `face.pam_panel_ref`. For dogs, the harness must be present in the dog style before fade-in.

6.6 Dog harnesses and service labels

The attached `DogGraph` supports an optional triangular harness icon. It is controlled by style keys on the dog figure, not by a separate attach action.

Style key	Type	Default	Meaning
`harness_style`	string	omitted	Enables the harness. `"standard"` is implemented; `"service"` is reserved.
`harness_color`	hex	`edge_color`	Fill color for the triangular harness.

```
{
  "action": "cast",
  "characters": {
    "chekov": {
      "figure_type": "dog",
      "offset": [-2, 0, 0],
      "style": {
        "harness_style": "standard",
        "harness_color": "#2d6a4f"
      }
    }
  }
}
```

The harness is a rigid right-triangle badge anchored to `spine_mid`. It tracks the dog through trot cycles and flips its head-side/tail-side geometry when the dog faces left. `fade_in` draws it after the dog skeleton so it sits visually above the body; `fade_out` includes it in the cleanup bundle.

7. Figure style additions

This section documents figure-level style fields added in the attached `figure.py`. These live in the character's `style` dict, not in the `faces` block.

7.1 Clothed limbs

`style["clothed_limbs"]` replaces selected arm and leg edges with filled polygon bands. The rest of PAM still treats those bands as edges: they live in `self.lines`, belong to `edge_group`, and animate through `become(new_band)` whenever the pose changes.

```
{
  "action": "cast",
  "characters": {
    "driver": {
      "build": "broad",
      "offset": [-2, 0, 0],
      "style": {
        "edge_color": "#304050",
        "clothed_limbs": {
          "mode": "tapered",
          "arm_proximal_width": 0.12,
```

```
          "arm_distal_width": 0.07,
          "leg_proximal_width": 0.16,
          "leg_distal_width": 0.10,
          "fill_color": "#304050",
          "stroke_color": "#18120c",
          "stroke_width": 1.8
        }
      }
    }
  }
}
```

Key	Values / type	Default	Meaning
`mode`	`"uniform"` or `"tapered"`	`"uniform"`	In uniform mode, each limb segment uses one width; in tapered mode, proximal joints are thicker than distal joints.
`arm_proximal_width`	float	`0.10`	Full band width near shoulder.
`arm_distal_width`	float	`0.06`	Full band width near wrist.
`leg_proximal_width`	float	`0.14`	Full band width near hip.
`leg_distal_width`	float	`0.09`	Full band width near ankle.
`fill_color`	hex	`edge_color`	Band fill color.
`stroke_color`	hex	`"#18120c"`	Band outline color.
`stroke_width`	float	`1.8`	Band outline width.

The taper is continuous across elbows and knees: the width at an elbow or knee is the arithmetic mean of proximal and distal widths. Coincident endpoints produce invisible placeholder polygons, matching the existing convention for degenerate lines.

7.2 Character speech-bubble colors

Human and alien figures support two style-level bubble keys:

Style key	Default	Meaning
`bubble_color`	`None`, then fallback to `edge_color`	Drives the saturated border and the pale tinted-white bubble fill.
`bubble_text_color`	`None`, then fallback to the dark head color	Overrides text color when the automatic dark text clashes.

```
{
  "style": {
    "edge_color": "#cc3333",
    "bubble_color": "#cc3333",
    "bubble_text_color": "#301010"
  }
}
```

At runtime, the implementation blends `bubble_color` about 10% toward the source color and 90% toward white for the fill. The result is still visibly associated with the character, but pale enough for dark text.

7.3 Facing direction

`HumanGraph` now tracks `self.facing` as `"right"` or `"left"`. `walk_to` and `run_to` update this field from the sign of horizontal motion. This matters for attached shoes, side-view face selection, and any later accessory whose silhouette needs to mirror with travel direction.

```
[
  {"action": "attach_shoes", "who": "bevers", "color": "#1a1a1a"},
  {"action": "walk_to", "who": "bevers", "x": 3.0},
  {"action": "walk_to", "who": "bevers", "x": -2.0}
]
```

After the second walk, the shoe ellipses have flipped to preserve the direction-of-travel read.

Prop Properties Reference

PAM v0.9.13 — props reference

Companion document to `CHARACTER_PROPERTIES.md`. Complete reference for everything that can be created, modified, or done with a prop in a PAM screenplay. Grouped by lifecycle: creation-time fields, prop metadata, registry behavior, action targets, and meta concerns. Heavy on syntax examples — like the `to_x`/`x` bite on the character side, the `tv_monitor` "missing `type` defaults to `hat`" bite came from sparse examples.

---

1. Creation-time fields

A prop is defined by an entry in either the manifest `props` block (for props present at scene start) or a `spawn_prop` action (for props that appear mid-scene). Both share the same field set.

1.1 Required: `type`, `x`

Every prop needs a `type` (the builder key) and an `x` (world-space horizontal position, in Manim units; screen center is `x: 0`).

```
{"type": "chair", "x": -2.0}
```

In a manifest `props` block, each prop is keyed by a **registry name** (unique, used in later actions):

```
{"action": "props", "items": {
  "alice_chair": {"type": "chair", "x": -2.0}
}}
```

In `spawn_prop`, the registry name is given as `prop`:

```
{"action": "spawn_prop", "prop": "alice_chair",
 "type": "chair", "x": -2.0}
```

Gotcha: if `spawn_prop` omits `type`, `pam_player.py` (line 1732 area) falls back to `"hat"`. Always include `type` even when you've also set it in the manifest. See §5.4.

1.2 Common positional: `y`, `color`, `accent`, `label`

Field	Type	Default	Notes
`y`	float	varies by type	World-space vertical. Furniture defaults to floor (-2.5ish); carried/wall props need explicit `y`.
`color`	hex string	per-type	Primary fill / stroke color. Builders style accents automatically.
`accent`	hex string	per-type	Secondary color (dodecahedron faces, monitor frames).
`label`	string	none	Optional text label drawn on the prop. Used heavily for "Elevator", "PolymerCorp", etc.

```
{"alice_chair": {"type": "chair", "x": -2.0,
                 "color": "#ff9999", "label": "A"}}
```

1.3 Style variants: `style`, `size`, `animate`, `closed`

A handful of builders take additional shape/state parameters.

`style` — selects a builder variant. Most useful on `phone`:

```
{"phone1": {"type": "phone", "x": 5.3, "y": -1.7,
            "style": "landline_flat"}}
```

Builder	Allowed `style` values
`phone`	`cellphone` (default), `landline_flat`, `landline_wall`
`cell_phone` / `smartphone`	aliases for `phone` with `style="cellphone"`

`size` — used by `floral_arrangement`:

Value	Use
`"carry"` (default)	Small bouquet held at chest height
`"large"`	Set-down arrangement on desk/table surface

```
{"bouquet": {"type": "floral_arrangement", "size": "carry",
             "parent": "chava", "attach": "rhand"}}
```

`animate` — currently used only by `dodecahedron`:

Value	Effect
(omitted)	Static
`"spin"`	Continuous rotation

`closed` — added in v0.9.13 to `build_laptop`:

Value	Effect
`false` (default)	Lid open, screen visible
`true`	Lid closed flat (storage / pre-reveal state)

1.4 Scene-graph: `parent`, `attach`, `prop_registry`

Any prop can be positioned relative to another prop or a character joint instead of in world space. Three fields together do this:

Field	Meaning
`parent`	Registry name of the parent prop (or character).
`attach`	Named attachment point on the parent (see §2.4).
`prop_registry`	The live registry dict — required only when building a child prop programmatically.

```
{"action": "spawn_prop", "prop": "monitor",
 "type": "dodecahedron", "radius": 0.12,
 "parent": "main_desk", "attach": "surface",
 "attrs": {"inclination": 10}}
```

A child prop snaps its origin to the parent's named attachment point; any explicit `x`/`y` is then applied as a local offset.

For character-parent attachment (e.g. carrying), see §4.2 and §4.6.

1.5 Visual attrs dict: `scale`, `inclination`

The `attrs` dict carries scale and rotation overrides that apply after the prop is built but before it's placed:

```
{"mon1": {"type": "dodecahedron", "radius": 0.12,
          "attrs": {"scale": 0.8, "inclination": 15}}}
```

Key	Type	Units	Default
`scale`	float	multiplier	1.0
`inclination`	float	degrees	0

1.6 Per-type parameter tables

A summary of the type-specific extras most often used. See §6 for the full prop catalog and §1.4 of `CHARACTER_PROPERTIES.md` analog notes on parameter discoverability.

`desk`

Param	Type	Default	Notes
`width`	float	3.0	Desk top width
`monitor`	bool	false	Adds a small dodecahedron monitor
`monitor_color`	hex	accent	Color of the added monitor

`dodecahedron`

Param	Default	Notes
`radius`	0.4	Geometric size
`accent`	per-type	Face accent color
`animate`	(none)	`"spin"` for rotation

`tv_monitor`

Param	Default	Notes
`screen_text`	" "	On-screen text
`screen_color`	dark	Background fill
`width`	2.2	Frame width

`avatar_pod`

Param	Default	Notes
`occupied`	false	Use occupied color palette

`elevator`

Param	Default	Notes
`label`	"Elevator"	Sign text
`capacity_label`	"Max Capacity: 4"	Secondary sign

`backpack` (v0.9.7)

Param	Default	Notes
`accent`	teal	Edge color of the Fano-plane graph

2. Prop metadata

Every prop VGroup carries metadata in three forms, all set by the `_attach_pam_attrs` helper in `props_core.py`.

2.1 Flat attributes (backward-compat)

The original metadata layer, preserved so older code in `pam_player.py` keeps working unchanged.

Attribute	Type	Meaning
`.pam_name`	str	Registry name (unique key used in screenplay actions)
`.pam_type`	str	Type string (`"chair"`, `"desk"`, etc.)
`.pam_x`	float	World x-coordinate (center of the prop)
`.pam_y`	float	World y-coordinate
`.pam_surface_y`	float	Y of the top surface (desk top, chair seat, etc.)

```
prop = registry["main_desk"]
print(prop.pam_x, prop.pam_y, prop.pam_surface_y)
```

Action handlers in `pam_player.py` rely on these heavily — anything that needs to know where a prop is reads `.pam_x` / `.pam_y` directly.

2.2 The `.pam_node` scene-graph dict

Added in the scene-graph pass. Carries the relational metadata.

```
prop.pam_node = {
    "name":   "monitor",
    "kind":   "dodecahedron",
    "parent": "main_desk",      # or None for world coords
    "attach": "surface",        # name of attachment point on parent
    "attrs":  {"scale": 1.0, "inclination": 10},
    "x":      0.0,
    "y":      0.06,
}
```

This is the canonical place to read parent/attach relationships. The flat attrs (§2.1) are the legacy mirror — `pam_node` is authoritative when they disagree.

2.3 The `.pam_attachments` dict

A dict of named 3-D positions (numpy arrays) on the prop geometry where child props or characters may connect.

```
desk.pam_attachments
# {
#   "surface":    [0.0, -0.4, 0.0],
#   "left-edge":  [-1.5, -0.4, 0.0],
#   "right-edge": [ 1.5, -0.4, 0.0],
#   "floor":      [0.0, -2.5, 0.0],
# }
```

Used by `resolve_position(node, registry)` to compute the world position of any child prop given its parent and attach point.

2.4 Per-type attachment-point catalog

A reference for which attach points each prop kind exposes.

Prop type	Attachment points
`chair`	`surface` (seat), `back-top`, `floor`, `left-edge`, `right-edge`
`desk`	`surface`, `left-edge`, `right-edge`, `floor`
`door` / `pocket_door`	`surface`, `handle`, `threshold`, `floor`, `left-edge`, `right-edge`
`elevator`	`surface`, `floor`, `threshold`, `centre`, `left-edge`, `right-edge`, `button_panel`
`avatar_pod`	`surface` (top of lid when closed), `floor`, `centre`, `lid_hinge`
`dodecahedron`	`surface` (top of polygon), `centre`, `floor`
`tv_monitor`	`surface` (wall-mounted top), `centre`, `screen_center`
`desk_lamp`	`bulb`, `base`, `arm_top` (bug target)
`phone` (cell)	`screen`, `top`
`phone` (landline_flat)	`cradle`, `receiver`, `dial_pad`
`backpack`	`top_strap`, `bottom`, `interior`
`flower` / `floral_arrangement`	`stem_base`, `bloom_center`

A child prop with `attach` set to a name not in this dict raises a `KeyError` at build time; check the builder source if uncertain.

---

3. Registry behavior

The prop registry is a dict-like object on the player, maintaining which props are currently live in the scene and where to find them.

3.1 The prop registry

Method	Returns	Notes
`registry.get(name)`	the prop VGroup, or `None`	Safe lookup
`registry.get_raw(name)`	the prop VGroup or raises	Strict — for code paths that must have the prop
`registry.add(name, prop)`	None	Used by `spawn_prop` internals
`registry.world_pos(name, attach)`	numpy array	Resolved world position of a named attachment point

Most action handlers in `pam_player.py` go through `get` and fall through quietly if a prop isn't found — see §3.3 for why this matters.

3.2 `spawn_prop` — mid-scene appearance

Adds a new prop to the registry and fades it in.

```
{"action": "spawn_prop", "prop": "bouquet",
 "type": "floral_arrangement", "size": "carry",
 "parent": "chava", "attach": "rhand",
 "color": "#ffcc88"}
```

Required fields: `prop` (registry name), `type`. All other fields match the manifest `props` block (§1).

Authoring tip: `spawn_prop` is the right action whenever the prop's existence is itself a story beat (a phone appears in a pocket, a folder materializes on the desk). For props that are simply present from the beginning of the scene, use the manifest `props` block.

3.3 `remove_prop` — despawn and symmetric-clear

Removes a prop from the registry and fades it out.

```
{"action": "remove_prop", "prop": "bouquet"}
```

Symmetric-clear gotcha: if a prop has been `parent`-attached as a child of another prop (or character), `remove_prop` clears it from the prop registry but does **not** automatically clear the parent's attachment-point reservation. This can leave a stale entry that affects later `spawn_prop` calls reusing the same attach point. v1.0.0 will add a `clear_attachments=True` helper; until then, the safest pattern is:

```
{"action": "remove_prop", "prop": "old_monitor"},
{"action": "spawn_prop", "prop": "new_monitor",
 "type": "dodecahedron",
 "parent": "main_desk", "attach": "surface"}
```

. . . where the new spawn explicitly overwrites the attach.

3.4 The `hidden` flag

A prop registered with `hidden: true` is added to the registry at full opacity 0 — present but invisible. Used for pre-positioning props that will appear later via `spawn_prop` or a custom reveal animation.

```
{"laptop": {"type": "laptop", "x": -1.5, "y": 0.8,
            "closed": true, "hidden": true}}
```

Later in the scene:

```
{"action": "reveal_laptop", "prop": "laptop", "from": "backpack"}
```

Scope note: `hidden` is a *prop* concept. An early v1.0.0 draft of `CHARACTER_PROPERTIES.md` referenced a character-side `hidden` flag, but that implementation did not land — characters use `fade_in` / `fade_out` and the cast-block manifest controls visibility. Props are the only entities where `hidden: true` is meaningful at construction.

3.5 Manifest `props` block vs `spawn_prop`

Question	Manifest `props`	`spawn_prop`
When does it run?	At scene start, before any other action	At its position in the action sequence
Visible immediately?	Yes (opacity 1) unless `hidden: true`	Fades in with `rt` (default 0.4s)
Use for?	Furniture and props present from the start	Story-driven appearances
Audit-friendly?	Yes — single block, easy to scan	Distributed through scene

A useful convention: list every prop in the manifest with appropriate `hidden: true` flags, then use `spawn_prop` only for animation timing of reveals. This keeps the at-a-glance prop manifest complete.

---

4. Action targets

The set of JSON actions that operate on props. Grouped by interaction type. Every action here resolves its target via the prop registry (§3.1).

4.1 Pointing and reaching

`reach_for` — extend one arm toward a prop, hold, then retract.

```
{"action": "reach_for", "who": "sidel", "target": "button_panel",
 "arm": "r", "hold": 0.6}
```

Key	Type	Default	Notes
`who`	str	required	Character key
`target`	str	required	Prop registry name
`arm`	`"r"` / `"l"`	`"r"`	Which arm to extend
`hold`	float	0.4	Seconds at full extension
`offset_x`	float	0	Adjust reach landing point
`offset_y`	float	0	Adjust reach landing point

Key	Type	Default	Notes

punch_button — sharp jab and retract.

```
{"action": "punch_button", "who": "sidel", "target": "button_panel"}
```

Same key set as **reach_for** but with snappier timing (shorter **hold**, faster extend/retract). v0.9.5 patch added **offset_x**/**offset_y** for fine targeting within a multi-button panel; an earlier bug where the arm failed to retract was fixed by deep-copying the rest pose before the jab so **morph_to** couldn't overwrite it.

peel_from_hand (v0.9.5) — open-palm gesture that slides a tiny prop off the palm, leaving the prop in **fig._held_prop**, ready for **stick_to**.

```
{"action": "peel_from_hand", "who": "sidel", "prop": "bug", "arm": "r"}
```

Best paired with **FRAMING=insert** — the prop is tiny in wide shots.

4.2 Carrying

grab — decisive reach + retract with the prop now in the character's hand. Faster than **pick_up**.

```
{"action": "grab", "who": "nona", "prop": "folder", "arm": "r"}
```

Sets **fig._held_prop = prop** and reparents the prop to the relevant hand joint. Subsequent **walk_to** actions will drag the held prop along (see §4.6).

place_on — set a carried prop onto a target surface.

```
{"action": "place_on", "who": "nona",
 "prop": "bouquet", "target": "main_desk"}
```

Reparents the prop to the target's **surface** attachment point and clears **fig._held_prop**.

move_aside — push a prop laterally without picking it up.

```
{"action": "move_aside", "who": "sidel",
 "prop": "desk_lamp", "direction": "left", "distance": 0.4}
```

Key	Type	Default
direction	**"left"** / **"right"**	required
distance	float	0.3
arm	**"r"** / **"l"**	inferred from **direction**

stick_to — attach a tiny prop to a target prop's surface.

```
{"action": "stick_to", "who": "sidel",
 "prop": "bug", "target": "desk_lamp"}
```

Reparents the prop and consumes **fig._held_prop**.

snap_photo — point a phone at a target and flash.

```
{"action": "snap_photo", "who": "nona", "target": "governor"}
```

Combines `reach_for` + `flash` (§4.4) with phone-specific framing.

4.3 Stateful props: open/close

Some prop types carry their own animation methods, bound to the VGroup at build time. Action handlers dispatch by method name.

`open_doors` / `close_doors` — used by both `pocket_door` and `elevator`.

```
{"action": "open_doors", "prop": "elev_1"}
{"action": "close_doors", "prop": "elev_1", "run_time": 0.6}
```

Key	Type	Default
`prop`	str	required
`run_time`	float	0.6

The prop's own `is_open` flag prevents redundant animations.

`open_lid` / `close_lid` — used by `avatar_pod`.

```
{"action": "open_lid", "prop": "pod_1"}
{"action": "close_lid", "prop": "pod_1"}
```

The lid hinges open along a configured axis and swaps in the "occupied" color palette when closed over a character.

Generic dispatch. Since `pocket_door` and `elevator` share method names but are different prop types, the action handler dispatches by `getattr(prop, "open_doors")` rather than checking `pam_type`. This means **any future prop with an `.open_doors` method automatically inherits the action** — which is both a feature and a small footgun (see §5.5).

4.4 `flash` — temporary recolor

Recolor a prop, character, or both for a brief duration, then restore.

```
{"action": "flash", "prop": "monitor",
 "color": "#44aaff", "duration": 0.3}
```

Key	Type	Default	Notes
`prop`	str	(one of)	Target prop
`who`	str	(one of)	Target character
`color`	hex	`"#44aaff"`	Flash color
`duration`	float	0.3	Total time in flashed state

At least one of `prop` or `who` is required; both may be set to flash both simultaneously. Used for camera flashes (`snap_photo`), highlight beats, and avatar-transfer effects (`highlight_edges` was an earlier specialized variant; `flash` superseded it).

4.5 `prop_say` — speech from non-character props

Make a prop speak via a bubble anchored to the prop position.

```
{"action": "prop_say", "prop": "phone1",
 "text": "Hello, is anyone there?",
 "side": "right", "hold": 1.4}
```

Key	Type	Default
`prop`	str	required
`text`	str	required
`side`	`"left"` / `"right"`	`"right"`
`hold`	float	1.4
`font_size`	int	18
`rt_in`	float	0.35
`rt_out`	float	0.25
`persist`	bool	false
`duration`	float	none

The bubble appears above the prop with a tail pointing at it. Used heavily for phones, TVs, and the GovernorGraph autonomous prop-char.

Persistent variant. Setting `persist: true` or a `duration: t` leaves the bubble on screen until explicitly cleared by `clear_bubble` or `clear_all_bubbles`. Watch for the focus_reset gotcha — see §5.2.

4.6 Walk-following via `_drag_attached_props`

When a character holds a prop (set by `grab`, `pick_up`, or manifest-level `parent: <character>`), `walk_to` / `run_to` / `rush_to` automatically drag the prop along.

The mechanism is `_drag_attached_props(fig)`, called inside the locomotion handlers (v0.9.7+). It iterates the global prop registry, finds every prop whose `pam_node["parent"]` matches the character's key, and applies the same per-frame translation that the character is undergoing.

Carry positions. Held props snap to the character's hand joint (`rhand` or `lhand`) by default. For carry styles that need a different anchor (chest, hip, head), set the `attach` explicitly when spawning:

```
{"action": "spawn_prop", "prop": "bouquet",
 "type": "floral_arrangement",
 "parent": "chava", "attach": "rhand"}
```

Known gotcha — `say`/`prop_say` with `persist` was rejected during walk-and-talk design because speech bubbles anchor to world-space coordinates at the moment of creation, not to the figure's head joint during movement. A walking character with a persistent bubble leaves the bubble behind. For walk-and-talk dialogue, use short non-persisted `say` calls per beat or design the dialogue around standing/bench rest points.

4.7 Prop-characters: DogGraph, GovernorGraph

Some entities are simultaneously **characters** (have figures, can pose and speak) and **props** (can be spawned, removed, parented). They're dual-registered in both the cast registry and the prop registry. See also `CHARACTER_PROPERTIES.md` §4.1 for the dual-registration mechanics.

DogGraph. Built via `build_dog(...)` in `characters.py` but also exposed as a prop type for spawning convenience. Has a `facing` parameter and supports `trot_to`, `prop_say`, and the usual locomotion verbs. Chekov (the robot dog) is the canonical example.

```
{"action": "spawn_prop", "prop": "chekov",
 "type": "dog", "x": -3.0, "facing": "right"}
{"action": "trot_to", "who": "chekov", "x": 2.0}
{"action": "prop_say", "prop": "chekov", "text": "Beep boop!"}
```

`trot_to` uses `step["x"]` (note: `x`, not `to_x`) for the destination — a known inconsistency with `walk_to` flagged for normalization in v1.0.0.

GovernorGraph. A dodecahedral autonomous prop-char that pulses gold/red and rotates on its own. Designed to be a background presence ("the Governor of Venus") that doesn't need explicit `morph`/`pose` actions. Built via `build_governor(...)`.

```
{"action": "spawn_prop", "prop": "governor",
 "type": "governor", "x": 0.0, "y": 1.5}
{"action": "prop_say", "prop": "governor",
 "text": "PLEASE WAIT...", "persist": true}
```

GovernorGraph's rotation runs continuously; it does not need a per-frame action to keep moving.

4.8 The `tv_monitor` / `news_anchor` pattern

A stateful display prop pattern for when a prop's contents change over time but its position does not.

The pattern. A `tv_monitor` prop holds `screen_text` and `screen_color`. To "change channels," the scene removes the prop and respawns it with new params at the same position.

```
{"action": "remove_prop", "prop": "monitor"},
{"action": "spawn_prop", "prop": "monitor",
 "type": "tv_monitor", "x": 4.0, "y": 1.8,
 "screen_text": "BREAKING NEWS", "screen_color": "#cc2222"}
```

The footgun. A `spawn_prop` block that omits `type` defaults to `"hat"` (see §5.4). When iterating on tv_monitor scenes, it's easy to copy a `spawn_prop` block and forget the `type`. The fix is to always include `type` even on the re-spawn — see the canonical example above.

A v1.0.0 candidate is a `set_prop_state` action that mutates the prop in-place instead of remove+respawn, with `tv_monitor` as the first client. Not yet scoped.

---

5. Meta and gotchas

5.1 Z-order for props

Manim's compositing follows the order in which mobjects are added to the scene. For props this typically gives:

1. Backdrop / building props (lowest)
2. Floor-level furniture (desk, chair, bench, sofa)
3. Wall-mounted props (tv_monitor, signs)
4. Characters
5. Hand-held props attached to characters
6. Mid-scene spawned props (often above characters intentionally)
7. Speech bubbles (highest, set by `bring_to_front`)

Footgun. A `focus_reset` re-plays characters into the scene at restored opacity, which can push them above props/bubbles added earlier in the scene. See §5.2.

PAM does not currently expose explicit z-order overrides for props. If a prop must sit above a character (e.g. a foreground bouquet during a wide shot), the practical solution today is to `remove_prop` + later `spawn_prop` after the character is staged, so the prop is added last.

5.2 Speech-bubble interactions: `clear_bubble`, `focus_reset`

Persistent bubbles (`say` or `prop_say` with `persist: true` or `duration: t`) survive across actions until explicitly cleared.

The `focus_reset` gotcha. When a `focus_reset` runs after a persistent bubble is on screen, the reset restores characters to full opacity and z-order, which can paint character mobjects on top of the bubble — visually overlaying it. Two workarounds:

```
{"action": "clear_bubble", "who": "thalia"},
{"action": "focus_reset"}
```

. . . or:

```
{"action": "say", "who": "thalia", "text": "...",
 "duration": 2.5},
{"action": "wait", "t": 3.0},
{"action": "focus_reset"}
```

. . . where `duration` is tuned to expire before the reset.

Action reference:

Action	Effect
`clear_bubble`	Dismiss one persistent bubble. Sub-key: `who` or `prop`. Optional `rt`.
`clear_prop_bubble`	v0.9.9 alias for `clear_bubble`. Identical semantics.
`clear_all_bubbles`	Dismiss every persistent bubble. Optional uniform `rt`.

5.3 Bubble world-space anchoring during walks

Speech bubbles attach to a fixed world-space position at the moment of creation. They do **not** track the character or prop during subsequent motion. Implications:

- `say` with `persist: true` followed by `walk_to` leaves the bubble behind at the character's starting position.
- `prop_say` followed by reparenting / moving the prop leaves the bubble behind at the prop's starting position.

Authoring pattern for walk-and-talk. Break dialogue into short non-persisted `say` beats per step of the walk, or design the dialogue to occur during a standing rest point (treadmill-style: walk-and-talk -> bench rest -> group-translate reset).

5.4 The `tv_monitor` type-field requirement

When `spawn_prop` is used to swap a stateful prop (typically `tv_monitor`), the block must include `type`. Omitting it triggers a fallback in `pam_player.py` (around line 1732) that defaults to `"hat"`, producing a silent visual bug — the new "channel" appears as a small hat at the wall position.

```
// WRONG - defaults to hat
{"action": "spawn_prop", "prop": "monitor",
 "x": 4.0, "y": 1.8, "screen_text": "BREAKING"}

// RIGHT
{"action": "spawn_prop", "prop": "monitor",
 "type": "tv_monitor",
 "x": 4.0, "y": 1.8, "screen_text": "BREAKING"}
```

Also worth checking the `prop` key matches the **manifest registry name**, not the type. If the manifest has `"monitor": {"type": "tv_monitor", ...}`, all later references must say `"prop": "monitor"` — not `"prop": "tv_monitor"`.

5.5 Stateful prop method-name overloading

Because `open_doors` / `close_doors` are dispatched by `getattr` on the prop VGroup (§4.3), any prop type that exposes those methods inherits the action. This is mostly fine — `pocket_door` and `elevator` work identically from the screenplay's point of view — but means an authoring error like specifying a `pocket_door` action on an `avatar_pod` (which has `open_lid` instead) produces a silent no-op rather than an error.

The unification candidate flagged for v1.0.0+ is a `StatefulProp` mixin with a generic `set_state(scene, name)` method. Each stateful prop would declare its states (`open`, `closed`, `partial`) and the per-state visual config; one JSON action `{"action": "prop_set_state", "prop": "elev_1", "state": "open"}` covers all four current stateful prop types. Not yet scoped; useful past five or six stateful props.

5.6 Back-burner

Items relevant to props, drawn from `BACK_BURNER.md` and the cleanup-pass surface:

- **`bench` prop missing from `props_furniture.py`.** Currently flagged for the walk-and-talk -> bench rest design. Build pattern follows `chair` with widened seat and no back.
- **`props_furniture.py` refactor.** Duplicated stateful-prop logic across `tv_monitor`, `avatar_pod`, `pocket_door`, and `elevator` is ripe for the `StatefulProp` mixin in §5.5.
- **`to_x` / `x` normalization.** `run_to` uses `step["x"]`, `walk_to` uses `step["to_x"]`, `trot_to` uses `step["x"]`. Inconsistent across locomotion handlers that drive walk-following. Cross-referenced in `CHARACTER_PROPERTIES.md` §4.6.
- **Elevator door animation approach.** Implemented in an earlier scene; the specific approach (slide direction, timing) is worth documenting in a `pocket_door` / `elevator` section of the pam_player manual when that's redone.
- **Carried-prop visibility in pan-up shots.** When a character carries flowers (or any chest-attached prop) and the camera pans up, the prop may be partially out of frame depending on `attach` choice. Worth experimenting with carry-point alternatives.
- **Prop gallery.** Analogous to `character_gallery.py`. Useful for visual regression and documentation. Not yet scoped.

---

6. Prop type catalog

Full registry of prop types as of v0.9.13. Aliases route to the same builder. Asterisk (*) = stateful (carries animation methods).

6.1 Furniture (`props_furniture.py`)

Type	Aliases	Builder	Notes
`chair`	—	`build_chair`	`surface`, `back-top`, `floor` attach points
`desk`	`table`, `console`, `computer`, `workstation`, `terminal`	`build_desk`	Optional `monitor=true`
`door`	—	`build_door`	Static panel
`pocket_door`*	—	`build_pocket_door`	`open_doors` / `close_doors`
`elevator`*	—	`build_elevator`	`open_doors` / `close_doors` / `partial_close`; exposes `button_panel` sub-prop
`desk_lamp`	`lamp`	`build_desk_lamp`	L-shaped, bug target
`bench`	—	*(missing — back-burner)*	—

6.2 Carried (`props_carried.py`)

Type	Aliases	Builder	Notes
`hat`	—	`build_hat`	Default `spawn_prop` fallback type (gotcha)
`briefcase`	—	`build_briefcase`	Carried at side
`folder`	—	`build_folder`	Flat, carried at chest
`phone`	—	`build_phone`	`style`: `cellphone` / `landline_flat` / `landline_wall`
`cell_phone`	`smartphone`	`build_phone(style="cellphone")`	Alias
`audio_video_bug`	`bug`	`build_audio_video_bug`	Tiny dot prop
`backpack`*	—	`build_backpack` (v0.9.7)	Fano-plane graph; `reveal_laptop(scene, laptop)`
`laptop`	—	`build_laptop` (v0.9.7)	`closed` param (v0.9.13)

6.3 Flora (`props_flora.py`)

Type	Aliases	Builder	Notes
`flower`	—	`build_flower`	Single stem
`floral_arrangement`	`bouquet`	`build_floral_arrangement`	`size`: `carry`/`large`

6.4 Environment (`props_environment.py`)

Type	Aliases	Builder	Notes
`dodecahedron`	—	`build_dodecahedron`	`radius`, `accent`, `animate="spin"`
`building`	—	`build_building`	`pam_height` for pan-up camera moves
`sun`	—	`build_sun`	Background
`moon`	—	`build_moon`	Background
`solar_panel`	—	`build_solar_panel`	Background
`wire`	—	`build_wire`	Background
`backdrop`	—	`build_backdrop`	Full-frame background
`tv_monitor`*	`monitor`	`build_tv_monitor`	`screen_text`, `screen_color`
`avatar_pod`*	`pod`	`build_avatar_pod`	`open_lid` / `close_lid`; `occupied` palette
`governor`	—	`build_governor` (in characters.py — dual-registered)	Autonomous rotation, pulses gold/red

6.5 Accessories (`props_accessories.py`)

These were originally scoped as character accessories but per v0.9.5 design landed as CAPTION text rather than built props. Listed here for completeness; the build functions exist but are not registered for `spawn_prop`.

Type	Notes
`name_tag`	Caption-only. E.g. *"Chava name_tag: Eve Smith"*
`delivery_cap`	Caption-only
`cheap_suit`	Caption-only
`silver_hair`	Builder exists (`build_silver_hair`) — accessible via `spawn_prop` for Mrs Bosch and similar. Render-check on `narrow`/`female` figures recommended.

End of PROP_PROPERTIES.md v0.9.13 draft. Companion to `CHARACTER_PROPERTIES.md`. Revision pass planned after the `props_furniture.py` stateful-prop refactor lands.

References and Source Documents

Source Markdown files

The book was assembled from the following source Markdown files, each folded into the single-file LaTeX output:

- README-pam0914.md
- pam_player-manual_v0914.md
- CHARACTER_PROPERTIES_section_1.md
- CHARACTER_PROPERTIES_section_2.md
- CHARACTER_PROPERTIES_section_3.md
- CHARACTER_PROPERTIES_section_4.md
- CHARACTER_PROPERTIES_section_4b.md
- CHARACTER_PROPERTIES_section_4c.md
- face_builder.py
- actions.py
- figure.py
- pam_player.py
- PROP_PROPERTIES-pam0914.md

External links

These links appeared in the source Markdown and are preserved as URL references in the body text where relevant.

- Manim Community: https://www.manim.community/
- screenplain: https://pypi.org/project/screenplain/